



RS6x_7x HIF Reference Manual

Number of Pages:121

Document Version:1.2

Release Date:2026-09-04

Revision History

Version	Revision Description	Date
1.2	1、 Modified the formatting; 2、 Updated description of point cloud.	2026-09-04
1.1	3、 Corrected the C1 and C2 data formats; 4、 The DevHw_Set_DefaultParam function is recommended for configuring DevHw_t parameters; 5、 Added the data_len parameter to Host_General_Command to obtain the response payload length; 6、 Added HIF frame examples.	2026-08-14
1.0	Initial release.	2026-06-30

Table of Contents

Revision History	ii
Table of Contents	iii
List of Figures	vii
List of Tables	viii
1 Introduction	1
2 Hardware Design	2
2.1 Framework	2
2.2 I/O Selection	2
2.2.1 SPI	2
Table 2-1 SPI I/O Definitions	2
2.2.2 UART	2
Table 2-2 UART I/O Definitions	2
2.2.3 I ² C	3
Table 2-3 I ² C I/O Definitions	3
2.2.4 Wake-Up I/O	3
Table 2-4 Notify I/O Definitions	3
2.2.5 Multiple Slaves	3
2.2.5.1 SPI	3
2.2.5.2 UART	4
2.2.5.3 I ² C	4
2.3 Hardware Design Considerations	4
2.3.1 Reference Schematic	4
2.3.2 Layout Description	4
3 Basic Communication Flow	5
3.1 HIF Protocol Frame Format	5
Figure 3-1 HIF Frame Format	5
3.1.1 Header	5
3.1.2 CheckSum	7
3.1.3 Payload	7
3.1.3.1 Command-Response	7
3.1.3.2 Report Data	9
3.1.3.3 Fragmentation	14
3.1.4 HIF Protocol Frame Example	15
3.2 Host-Device Interaction	16
3.2.1 Wake-Up and Sleep	16
3.2.1.1 Wake-Up	16
3.2.1.2 Sleep	16
3.2.1.3 SPI	17
1. The Host sends 55 FF 55 FF;	17
3.2.1.3.2 Sleep	18
3.2.1.4 UART	19
3.2.1.4.1 Wake-Up	19
3.2.1.4.2 Sleep	19
3.2.1.5 I ² C Communication	19
3.2.2 Command-Response	20
3.2.2.1 SPI	20
3. The Host reads the data;	20
3.2.2.2 UART	20

3.2.2.3 I ² C	20
3.2.3 Data Reporting	20
3.2.3.1 SPI	21
3.2.3.2 UART	22
3.2.3.3 I ² C	22
3.2.3.4 Transport-Layer Retransmission	22
3.2.4 Exception Recovery	22
3.2.4.1 The Host Cannot Receive the Message Header When Receiving Report Data	22
4 Software Framework	24
4.1 Protocol Architecture	24
4.1.1 Message Layer	24
4.1.2 Transport Layer	25
4.1.3 Link Layer	25
4.1.4 Physical Layer	25
4.2 Software Architecture	25
4.2.1 Host	25
4.2.2 Device	26
5 Host Development	27
5.1 Host Driver Porting	27
5.1.1 File Structure	27
5.1.2 Porting Requirements	29
5.1.2.1 System Interfaces	29
5.1.2.2 I/O Interface	35
5.1.2.3 Storage-Media Interface	38
5.1.2.4 Logging Interface	39
5.1.2.5 COM Interface	39
5.2 Host Driver Usage Flow	48
5.2.1 Driver and Device Initialization	49
5.2.1.1 Initialization Flow	49
5.2.1.2 Device Parameter Configuration	49
5.2.1.3 API Usage	51
Table 5-42 Host_Driver_Deinit	51
5.2.2 Data Reporting	52
5.2.2.1 C1/C2/C3 Data Usage Flow	53
5.2.2.2 Other Data Processing Flows	55
5.2.2.3 API Usage	56
Table 5-47 Report_CB	56
5.2.3 Command Transmission	59
5.2.3.1 Usage Flow	59
5.2.3.2 API Usage	60
5.2.4 Firmware Download	69
5.2.4.1 Usage Flow	69
5.2.4.2 API Usage	70
Table 5-75 Host_BurnMode_Enter	70
5.3 Configuration and Compilation	71
5.3.1 Feature Trimming	71
5.3.2 Adaptation-Layer Feature Switches	71
5.3.3 Firmware Download Function	71
5.3.4 Message-Layer Functions	72
5.3.5 Other	73
Table 5-82 Other Configuration Macros	73
5.4 Simple Porting Test Code	73
5.4.1 Data Reporting	73

5.4.1.1 UART	73
5.4.1.2 SPI	74
5.4.2 Configuration Transmission	77
5.4.2.1 UART	77
5.5 Typical Debugging Process	78
5.5.1 Data Path	78
5.5.1.1 Initialization Process	78
5.5.1.2 LLC_Send and LLC_Recv Paths	78
5.5.2 Data Reporting	78
5.5.2.1 HIF_TL_DataNotify_CB	78
5.5.2.3 Protocol Logic	79
5.5.3 Command Sending	79
5.5.3.1 Path for Sending a Command Request to the HIF_TL_Transport_Task Thread	79
5.5.3.2 Protocol Logic	79
5.5.4 Firmware Download	79
5.5.4.1 Synchronization Failure	79
5.6 Notes	79
5.6.1 Automatic C1, C2, and C3 Packet Assembly at the Application Layer	79
5.6.2 Sleep Function	79
6 Device Development	80
6.1 Code Directory Structure	80
6.2 Usage Process	81
2. Initialize HIF;	81
4. Report data.	81
6.3 Configuration Description	82
6.3.1 Configuration Structure and Global Variable	82
6.3.2 Default Configuration	84
6.3.2.1 Basic Configuration	84
6.3.2.2 Feature Configuration	85
6.3.3 Other Configuration	85
6.3.4 Configuration and Usage	85
6.3.4.1 Using the Default Configuration	85
6.3.4.2 Static Project Configuration	86
6.3.4.3 Dynamic Configuration	87
6.4 API Usage Instructions	87
6.4.1 Status Codes	87
Table 6-3 Status Codes	87
6.4.2 API	88
6.4.2.1 HIF_DefaultInitParam	88
6.4.2.2 HIF_Init	88
6.4.2.3 HIF_ExtControl	88
6.4.2.4 HIF_MsgReport_ListPushData	89
6.4.2.5 HIF_MsgReport_ListPopData	89
6.4.2.6 HIF_MsgReport_ListStart	90
6.4.2.7 HIF_MsgReport	90
6.4.2.8 HIF_MsgHdl_Regist	91
6.4.2.9 HIF_MsgResp	92
6.4.2.10 HIF_InitStatusReport	92
6.4.2.11 HIF_MsgReport_Lock	93
6.4.2.12 HIF_MsgReport_Unlock	93
6.4.2.13 HIF_PM_WakeupDevice	93
6.4.2.14 HIF_PM_SuspendDevice	93
6.4.3 Data Reporting	94
6.4.3.1 Blocking Linked-List Transmission	94

6.4.3.2 Callback-Based Linked-List Transmission	95
6.4.3.3 Blocking Non-Linked-List Transmission	97
6.4.3.4 Callback-Based Non-Linked-List Transmission	97
6.4.4 Command-Response	98
3. Register the callback	98
6.4.4.1 Message-Processing Callback Prototype	98
6.4.4.2 Response Description	98
6.4.4.3 Receive and Reply Example	98
6.5 Radar Data Reporting Process	99
6.5.1 0xC2 Radar FFT Data	100
6.5.2 0xC3 Radar Point-Cloud Data	100
6.5.3 0xC6 "PSIC Debug Protocol" Data	100
7 Custom Msg IDs	101
7.1 Permitted Custom Msg ID Ranges	101
7.2 Custom Commands	101
7.2.1 Host-Side Development Steps	101
7.2.2 Device-Side Development Steps	102
7.3 Custom Report Data	102
7.3.1 Host-Side Development Steps	102
7.3.2 Device-Side Development Steps	102
8 Q&A	103
8.1 Communication Issues	103
8.1.1 A checksum validation problem occurs when the HOST sends a message with a Payload of 1024 bytes or more	103
8.1.2 Why does the Host driver receive one packet but then continuously miss multiple packets when active reporting uses COM_IRQ_THREAD notification?	103
8.1.3 Unable to Receive Report Packets That Pass checksum Validation	103
8.1.4 Commands and Responses	103
8.2 Configuration Issues	103
8.2.1 Modifying Tool/Host PC Configuration Through the Host	103
8.3 Abnormal Radar Operation	104
8.3.1 The radar analysis sample suddenly stops reporting when the Frame period is set to a very small value	104
Appendix	106
Existing Msg ID Definitions	106
Common Command-Response Data Formats	108
START_CTRL(0x07)	110
Disclaimer and Copyright Notice	112

List of Figures

Figure 2-1	Hardware Connection for HIF Communication Between the Device and Host	2
Figure 3-1	HIF Frame Format	5
Figure 3-2	HIF Frame Header Format (Little-Endian)	5
Figure 3-3	HIF Checksum Calculation Area	7
Figure 3-4	Command Frame MsgHeader	7
Figure 3-5	Response Frame MsgHeader	7
Figure 3-6	Response Frame Payload Field Definitions	8
Figure 3-7	C1 FFT Data Message	9
Figure 3-8	C2 FFT Data Message	10
Figure 3-9	Point-Cloud Data Message	11
Figure 3-10	PSIC Debug Protocol 0xC6	13
Figure 3-11	Payload Structure of a Fragmented Frame	14
Figure 3-12	Example of Fragmenting Data to Be Sent	14
Figure 3-13	Fragmented-Frame Payload Example	15
Figure 3-14	HIF Header Example (Little-Endian)	16
Figure 3-15	Wake-Up and Sleep Interaction Flow	16
Figure 3-16	SPI Wake-Up Timing	17
Figure 3-17	SPI Timing Diagram for Data 55 FF 55 FF	17
Figure 3-18	SPI Timing Diagram for Data 79 79 79 79	18
Figure 3-19	SPI Timing Diagram for Command-Response	18
Figure 3-20	SPI Timing for Sleep Command Data	18
Figure 3-21	SPI Timing for Response Data Header	19
Figure 3-22	SPI Timing for Response Data Payload + CheckSum	19
Figure 3-23	UART Wake-Up Timing	19
Figure 3-24	UART Timing Diagram for Command-Response	19
Figure 3-25	SPI Timing Diagram for Command-Response	20
Figure 3-26	Data Reporting Interaction Flow	21
Figure 3-27	SPI Report-Data Timing	22
Figure 3-28	UART Report-Data Timing	22
Figure 3-29	Host-Device State Synchronization	23
Figure 4-1	Protocol Architecture	24
Figure 4-2	Host Software Architecture	26
Figure 4-3	Device Software Architecture	26
Figure 5-1	EDGE Notification-Type Reported-Data Reception Flow	40
Figure 5-2	COM_ISR Notification-Type Reported-Data Reception Flow (Thread Calls LLC_Recv Before the Cache Is Full)41	
Figure 5-3	COM_ISR Notification-Type Reported-Data Reception Flow (Thread Calls LLC_Recv After the Cache Is Full)42	
Figure 5-4	COM_IRQ_THREAD Notification-Type Reported-Data Reception Flow	42
Figure 5-5	Overall Host Driver Usage Flow	49
Figure 5-6	Host Driver and Device Initialization Flow	49
Figure 5-7	C1/C2/C3 Reported-Data Callback Setup Flow	53
Figure 5-8	Custom Data Processing Scheme	55
Figure 5-9	Command Transmission Flow	59
Figure 5-10	Firmware Download Flow	69
Figure 6-1	Linked-List Data Reporting Example Scenario 1	94
Figure 6-2	Linked-List Data Reporting Example Scenario 2	94

List of Tables

Table 2-1	SPI I/O Definitions	2
Table 2-2	UART I/O Definitions	2
Table 2-3	I ² C I/O Definitions	3
Table 2-4	Notify I/O Definitions	3
Table 2-5	SPI Host I/O Requirements for N Slaves over Communication	3
Table 2-6	UART Host I/O Requirements for N Slaves over Communication	4
Table 2-7	I ² C Host I/O Requirements for N Slaves over Communication	4
Table 3-1	HIF Header Field Definitions	5
Table 3-2	HIF Response and Report Status Definitions	8
Table 3-3	HIF Command-Response Common MsgID Definitions	8
Table 3-4	C1 FFT Data Message Payload Description	9
Table 3-5	C2 FFT Data Message Payload Description	10
Table 3-6	Point-Cloud Data Message Payload Description	11
Table 3-7	PSIC Debug Protocol 0xC6 Payload Field Description	13
Table 3-8	Frag Header Description	14
Table 5-1	host_os_enter_critical	29
Table 5-2	host_os_exit_critical	29
Table 5-3	host_os_timestamp_us	30
Table 5-4	host_os_malloc	30
Table 5-5	host_os_free	30
Table 5-6	host_os_memset	30
Table 5-7	host_os_memcpy	31
Table 5-8	host_os_memcmp	31
Table 5-9	host_os_thread_create	32
Table 5-10	host_os_thread_delete	32
Table 5-11	host_os_thread_is_valid	32
Table 5-12	host_os_sem_create	33
Table 5-13	host_os_sem_delete	33
Table 5-14	host_os_sem_is_valid	33
Table 5-15	host_os_sem_take	33
Table 5-16	host_os_sem_give	34
Table 5-17	host_os_delayms	34
Table 5-18	host_os_delayus	34
Table 5-19	host_os_pm_lock	34
Table 5-20	host_os_pm_unlock	35
Table 5-21	host_io_mode_set	35
Table 5-22	host_io_irq_cfg	35
Table 5-23	host_io_irq_enable	36
Table 5-24	host_io_irq_disable	36
Table 5-25	host_io_write	37
Table 5-26	host_io_read	37
Table 5-27	host_store_open	38
Table 5-28	host_store_close	38
Table 5-29	host_store_read	38
Table 5-30	host_log_print	39
Table 5-31	Interface Requirements for Different LLC Notification Types	42
Table 5-32	host_com_init_t	43
Table 5-33	host_com_deinit_t	43
Table 5-34	host_com_open_t	44

Table 5-35	host_com_close_t	44
Table 5-36	host_com_write_t	44
Table 5-37	host_com_read_t	45
Table 5-38	host_com_interrupt_ctrl_t	45
Table 5-39	host_com_interrupt_handle_regist_t	46
Table 5-40	host_com_read_finish_t	46
Table 5-41	Host_Driver_Init	51
Table 5-42	Host_Driver_Deinit	51
Table 5-43	Host_Device_Regist	51
Table 5-44	Host_Device_Unregist	52
Table 5-45	Host_Device_Open	52
Table 5-46	Host_Device_Close	52
Table 5-47	Report_CB	56
Table 5-48	Mmw_Cx_FUNCTION_Handle_Set	57
Table 5-49	Mapping Between Cx and FUNCTION	57
Table 5-50	Mmw_Report_Handle_Set_General	57
Table 5-51	Mmw_Report_Handle_FreeAll	59
Table 5-52	HostCmd_Version_Get	60
Table 5-53	HostCmd_Device_Reboot	60
Table 5-54	Host_General_Command	61
Table 5-55	MmwCmd_General_StartCtrl_Cfg	62
Table 5-56	MmwCmd_General_MimoMode_Cfg	62
Table 5-57	MmwCmd_General_FrameType_Cfg	63
Table 5-58	MmwCmd_General_StartFreq_Cfg	63
Table 5-59	MmwCmd_General_TriggerRange_Cfg	63
Table 5-60	MmwCmd_General_RangeResolution_Cfg	63
Table 5-61	MmwCmd_General_MaxVelocity_Cfg	64
Table 5-62	MmwCmd_General_VelResolution_Cfg	64
Table 5-63	MmwCmd_General_FramePeriod_Cfg	64
Table 5-64	MmwCmd_RadarAnalysis_Start	65
Table 5-65	MmwCmd_RadarAnalysis_Stop	65
Table 5-66	MmwCmd_RadarAnalysis_Mode_Cfg	65
Table 5-67	MmwCmd_RadarAnalysis_Freq_Cfg	65
Table 5-68	MmwCmd_RadarAnalysis_Range_Cfg	66
Table 5-69	MmwCmd_RadarAnalysis_Veloc_Cfg	66
Table 5-70	MmwCmd_RadarAnalysis_Frame_Cfg	66
Table 5-71	MmwCmd_RadarAnalysis_Intv_Cfg	67
Table 5-72	MmwCmd_RadarAnalysis_ChirpNum_Cfg	67
Table 5-73	MmwCmd_RadarAnalysis_Report_Cfg	67
Table 5-74	MmwCmd_RadarAnalysis_DopFft_Cfg	68
Table 5-75	Host_BurnMode_Enter	70
Table 5-76	Host_Burn_Image	70
Table 5-77	Host_BurnMode_Exit	70
Table 5-78	Global Macros for Feature Trimming	71
Table 5-79	Global Macros for Enabling/Disabling Adaptation-Layer Features	71
Table 5-80	Detailed Firmware Download Configuration	72
Table 5-81	Global Macros for Configuring the Message Layer	72
Table 5-82	Other Configuration Macros	73
Table 6-1	HIF Basic Configuration	84
Table 6-2	HIF Feature Configuration Items	85
Table 6-3	Status Codes	87
Table 8-1	Radar Configuration TLVs (Partial)	111

1 Introduction

As an independent intelligent sensor, PSIC reports the coordinates of a detected object to the Host through a communication interface when an application requirement is met, for example, when a target is detected. For different application scenarios, the Host also sends radar configuration parameters through the communication interface. In low-power scenarios, the Host and Device remain asleep most of the time. Because burst communication requires waking the peer first, a wake-up notification I/O is required.

Data actively sent from the Device to the Host is referred to as a "Report" in this document. Data actively sent from the Host to the Device is referred to as a "Command", and the Device reply to a "Command" is referred to as a "Response." Except for several special "Commands", the protocol specifies that each "Command" has a corresponding "Response". Only one "Command" can be processed at a time; that is, the Host can send the next "Command" only after receiving the "Response" to the current "Command". Reports support active and passive reporting. Passive reporting is commonly used with communication interfaces such as SPI and I²C. Because the Device is a communication slave, before transferring data to the Host, the Device must first use an I/O to indicate that data is available. The Host sends a special POLL frame to the Device. After preparing the data, the Device indicates readiness again through the I/O, and the Host reads the Device data, thereby transferring the Device Report data to the Host. Active reporting is commonly used with UART, where the Device sends Report data directly to the Host through UART TX. Passive reporting is commonly used with master-slave interfaces such as SPI or I²C.

HIF supports UART, SPI, and I²C by default. One interface can be selected dynamically through parameters, or configuration macros can disable selected interfaces to reduce code size. UART is easy to use, SPI is suitable for high-bandwidth data transmission, and I²C is suitable for multi-slave communication. SPI and I²C support only passive reporting, whereas UART supports both active and passive reporting.

Note:

1. The radar application communication protocol uses HIF, for example, when the radar reports detected-target position information or when the Host/master controller sends configuration parameters.
2. The communication protocol used for radar firmware upgrade may not be HIF and must be determined according to the specific device model. Radar firmware upgrade supports BURN and OTA. For BURN, the Host communicates directly with the radar IC BOOT ROM to download and burn the firmware; the protocol used by some device models is not HIF. For OTA, the application first downloads the firmware to the backup area and then swaps the images to complete the upgrade; HIF is used by default.

2 Hardware Design

2.1 Framework

The following hardware resources are required for HIF communication between the Device and Host:

Physical communication interface: SPI, UART, I²C, etc.

Notify I/O: Used by the Device to notify the Host that the Device has data to report and is in the Ready state, allowing the Host to retrieve the data.

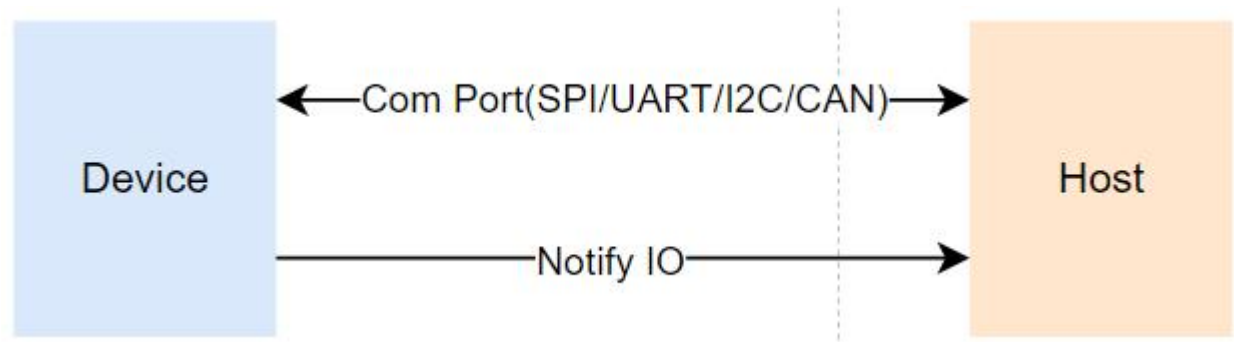


Figure 2-1 Hardware Connection for HIF Communication Between the Device and Host

2.2 I/O Selection

2.2.1 SPI

When SPI is used as the physical communication interface, the Device operates as the slave (SPI0 by default) and the Host operates as the master. The I/O definitions are as follows:

Table 2-1 SPI I/O Definitions

IO	Device	Host
SPI0-CS	PA0	User-defined
SPI0-CLK	PA1	User-defined
SPI0-MOSI	PA2	User-defined
SPI0-MISO	PA3	User-defined

Note: When SPI is selected as the communication port, UART0 (PB0 and PB1) is recommended for the Debug (UART Print) port. PB0 can also be multiplexed as UPD (Burn Control I/O), thereby saving I/O resources.

2.2.2 UART

When UART is used as the physical communication interface, the Device selects UART0 or UART1 by default. Because PB0 can serve as both UART0-TX and the Burn Control I/O, UART0 is recommended to save I/O resources.

The I/O definitions are as follows:

Table 2-2 UART I/O Definitions

IO	Device	Host
UART0-TX	PB0	User-defined
UART0-RX	PB1	User-defined

Note: When UART is selected as the communication port, UART1 (PA0 and PA1) is recommended for the Debug (UART Print) port.

2.2.3 I²C

When I²C is used as the physical communication interface, the Device operates as the slave (I²C0 by default) and the Host operates as the master. The I/O definitions are as follows:

Table 2-3 I²C I/O Definitions

IO	Device	Host
I ² C0-SCL	PA5	User-defined
I ² C0-SDA	PA4	User-defined

Note: When I²C is selected as the communication port, UART0 (PB0 and PB1) is recommended for the Debug (UART Print) port. PB0 can also be multiplexed as UPD (Burn Control I/O), thereby saving I/O resources.

2.2.4 Wake-Up I/O

Table 2-4 Notify I/O Definitions

IO	Device	Host
Notify IO	PA6 (output)	User-defined (input)

2.2.5 Multiple Slaves

This section describes the Host I/O resource requirements for a one-master (Host), multiple-slave (Device) application.

The resources mainly include wake-up I/Os and communication-interface I/Os (SPI and I²C).

2.2.5.1 SPI

Table 2-5 SPI Host I/O Requirements for N Slaves over Communication

I/O Category	Host I/O Requirement	Description
Wake-Up I/O	N	One wake-up I/O corresponds to the wake-up I/O of one Device.
SPI-CS	N	
SPI-CLK	1	
SPI-MOSI	1	
SPI-MISO	1	
UPD	1	Used to burn the Device.
NRST	N	Used to reset the Device.

Note:

Using UPD together with NRST can place the Device in burn mode: pull UPD low, then pull NRST low, and then keep UPD low for at least 50 ms to place the Device in burn mode.

SPI multi-slave designs are subject to usage constraints. Therefore:

For the RS6130 solution, refer to Section 7.3, SPI Multi-Radar Hardware Design, in RS6130_Hardware_Design_and_PCB_Layout_Guide_Vxx.pdf.

For the RS6240 solution, refer to Section 7.3, SPI Multi-Radar Hardware Design, in RS6240_Hardware_Design_and_PCB_Layout_Guide_Vxx.pdf.

2.2.5.2 UART

Table 2-6 UART Host I/O Requirements for N Slaves over Communication

I/O Category	Host I/O Requirement	Description
Wake-Up I/O	N	One wake-up I/O corresponds to the wake-up I/O of one Device.
UART-Tx	N	
UART-Rx	N	
UPD	1	Used to burn the Device.
NRST	N	Used to reset the Device.

2.2.5.3 I²C

Table 2-7 I²C Host I/O Requirements for N Slaves over Communication

I/O Category	Host I/O Requirement	Description
Wake-Up I/O	N	One wake-up I/O corresponds to the wake-up I/O of one Device.
I ² C-SCL	1	
I ² C-SDA	1	
UPD	1	Used to burn the Device.
NRST	N	Used to reset the Device.

2.3 Hardware Design Considerations

2.3.1 Reference Schematic

For the RS6130 solution, refer to Section 7, Reference Schematic, in RS6130_Hardware_Design_and_PCB_Layout_Guide_Vxx.pdf.

For the RS6240 solution, refer to Section 7, Reference Schematic, in RS6240_Hardware_Design_and_PCB_Layout_Guide_Vxx.pdf.

2.3.2 Layout Description

UART and I²C operate at relatively low frequencies and can use a conventional design.

SPI supports higher operating frequencies. Therefore, when the frequency exceeds 5 MHz:

For the RS6130 solution, refer to Section 8.8, External SPI Layout Guidelines, in RS6130_Hardware_Design_and_PCB_Layout_Guide_Vxx.pdf.

For the RS6240 solution, refer to Section 8.8, External SPI Layout Guidelines, in RS6240_Hardware_Design_and_PCB_Layout_Guide_Vxx.pdf.

3 Basic Communication Flow

3.1 HIF Protocol Frame Format

This section describes the HIF frame format (referred to as Frame in this document).

A Frame consists of three parts: Header, Payload, and Checksum, as shown in the following figure.

The Header field length is fixed at 6 bytes;

The Payload field length ranges from 0 to 4095 bytes;

The Checksum field length is fixed at 4 bytes.

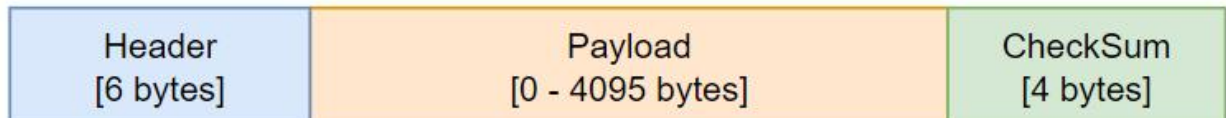


Figure 3-1 HIF Frame Format

3.1.1 Header

HIF

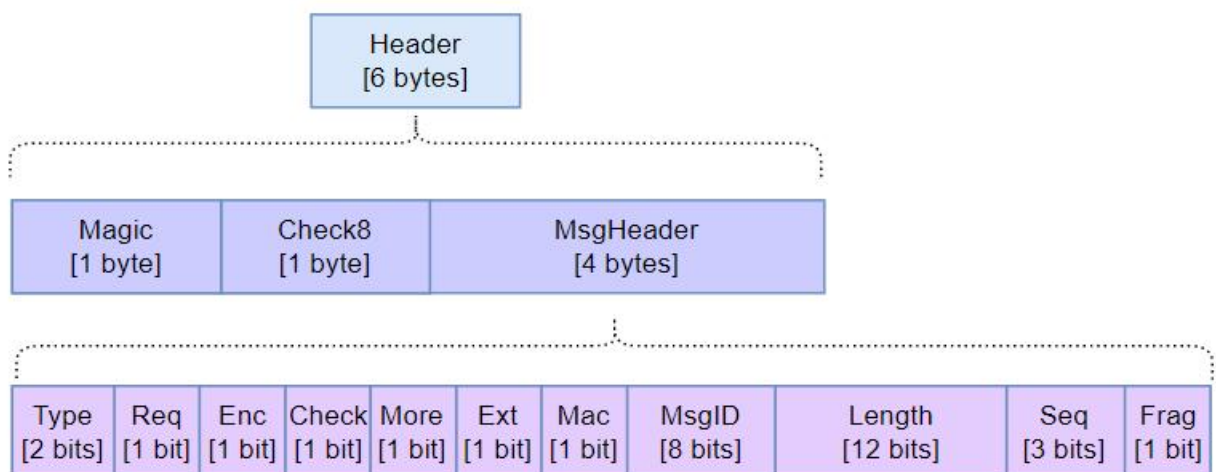


Figure 3-2 HIF Frame Header Format (Little-Endian)

The Header fields are defined as follows:

Table 3-1 HIF Header Field Definitions

Field		Size	Description	Typical Value
Magic		8bit	Signature code. The fixed value is 0xA5.	0xA5
Check8		8bit	Header checksum, calculated over Magic + MsgHeader. The accumulation unit is Byte. The checksum is the bitwise complement of the accumulated sum, that is, check8 = ~sum8.	N/A
MsgHeader	Type	2bit	Type: 0: Reserved; 1:Host message, Host → Device; 2:Device message, Device → Host; 3: Reserved.	Host → Device: 1 Device → Host: 2

Field		Size	Description	Typical Value
	Req	1bit	Whether a response is required: 0: Non-request message or response message; 1: Request message; a response is required.	Host → Device: 1 Device → Host: 0
	Enc	1bit	Whether encryption is required: 0: Unencrypted message; 1: Encrypted message.	0
	Check	1bit	Whether a checksum field is present: 0: No Checksum field; 1: Checksum field present.	1
	More	1bit	Pending-data indication: 0: No more messages pending transmission; 1: More messages pending.	0
	Ext	1bit	Extended-header indication: 0: No extended header; 1: Extended header present.	0
	Mac	1bit	Checksum algorithm type for the Checksum field: 0: Checksum algorithm is Check32; 1: Other checksum algorithm.	0
	MsgID	8bit	Message ID, range 0x00 to 0xFF.	N/A
	Length	12bit	Data (payload) length, range 0 to 4095.	N/A
	Seq	3bit	Sequence number, range 0 to 7. Starting from 0, it increments by 1 for each Frame during transmission and wraps around.	0 to 7
	Frag	1bit	Fragmentation flag, defined as follows: 0: The current Frame is not fragmented; 1: The current Frame is fragmented.	0
Check32		32bit	32-bit checksum covering MsgHeader [2 to 5] + Payload. 1) For check32, the accumulation unit is DWORD (32 bits). If the trailing data is shorter than one DWORD, it is zero-padded for calculation. The checksum is the bitwise complement of the accumulated sum, that is, Check32 = ~sum32; 2) For MAC32 (data integrity check), the checksum algorithm is platform-defined.	Checksum

3.1.2 CheckSum

32-bit checksum value calculated over MsgHeader + Payload.

When Mac in MsgHeader is 0, the checksum algorithm takes the bitwise complement of the accumulated sum. Specifically, using DWORD (32 bits) as the accumulation unit, accumulate from MsgHeader through Payload. If the trailing data is shorter than one DWORD, pad the high-order bits with zeros. The resulting 32-bit accumulated sum is sum32, and the final checksum is CheckSum = $\sim$ sum32.

When Mac in MsgHeader is 1, this field is reserved.

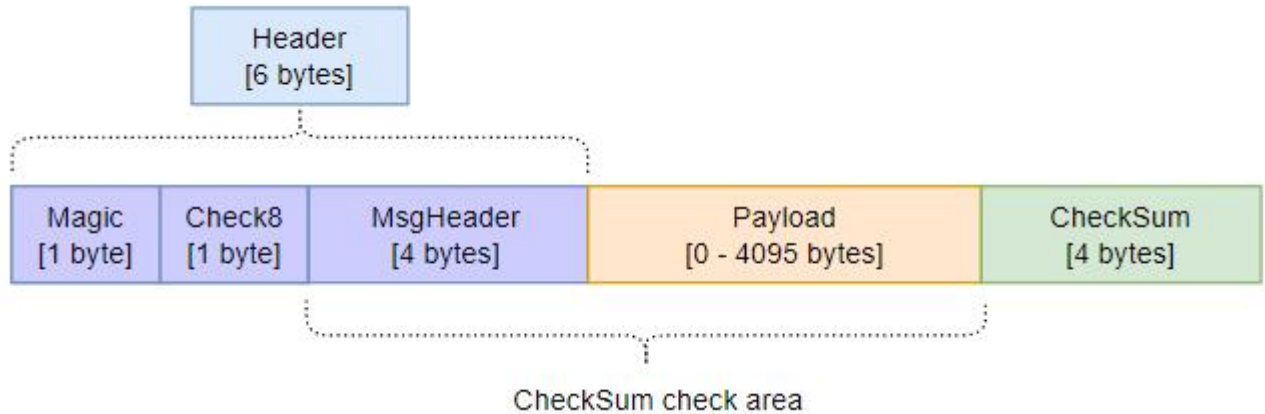


Figure 3-3 HIF Checksum Calculation Area

3.1.3 Payload

This section primarily describes the formats of defined application data, that is, the payload field.

3.1.3.1 Command-Response

Command: A command sent from the Host to the Device.

Response: After the Device receives a Command from the Host, the reply to that Command. Its ID is the same as the Command ID, and the reply includes the execution status, data, and other information.

Command and Response generally occur in pairs.

3.1.3.1.1 Command Format

The format follows the HIF Frame format above. MsgHeader is shown in the following figure.

Type [2 bits]	Req [1 bit]	Enc [1 bit]	Check [1 bit]	More [1 bit]	Ext [1 bit]	Mac [1 bit]	MsgID [8 bits]	Length [12 bits]	Seq [3 bits]	Frag [1 bit]
1	1	0	1	0	N/A	0	Cmd ID	Length of Cmd Data	NA	0

Figure 3-4 Command Frame MsgHeader

3.1.3.1.2 Response Format

The Response Frame follows the HIF Frame format, where:

MsgID is the same as that of the corresponding Command;

The MsgHeader fields are shown in the following figure:

Type [2 bits]	Req [1 bit]	Enc [1 bit]	Check [1 bit]	More [1 bit]	Ext [1 bit]	Mac [1 bit]	MsgID [8 bits]	Length [12 bits]	Seq [3 bits]	Frag [1 bit]
2	0	0	1	0	N/A	0	Cmd ID	Length of Payload	NA	0

Figure 3-5 Response Frame MsgHeader

The first Byte of the Payload is the Status information, which indicates the processing status of the Command. Reply data (optional) follows Status, as shown in the following figure.

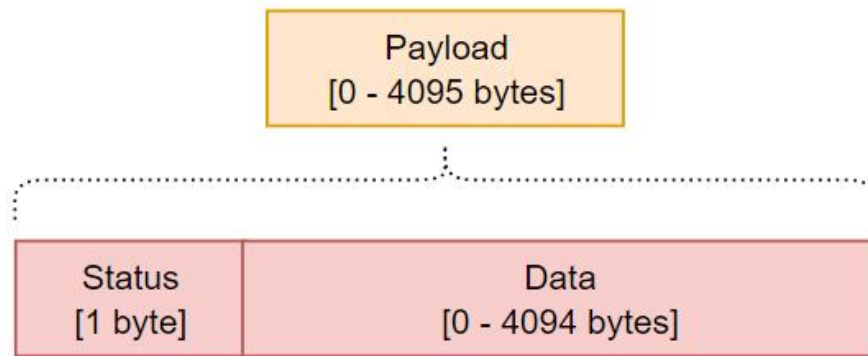


Figure 3-6 Response Frame Payload Field Definitions

Response status is defined as follows:

Table 3-2 HIF Response and Report Status Definitions

Status	Value	Meaning
HIF_CMD_STATUS_SUCCESS	0	Command executed successfully
HIF_CMD_STATUS_UNSUPPORT	1	Unsupported command
HIF_CMD_STATUS_VERSION	2	Version mismatch
HIF_CMD_STATUS_TOOLONG	3	Message length mismatch
HIF_CMD_STATUS_CHECK	4	Header or data checksum failure
HIF_CMD_STATUS_PARAM	5	Invalid parameter
HIF_CMD_STATUS_REPLAY	6	Duplicate message
HIF_CMD_STATUS_AUTH	7	Authentication failed
HIF_CMD_STATUS_LATE	8	Delayed reporting required
HIF_CMD_STATUS_BUSY	9	Device busy
HIF_CMD_STATUS_STATE	10	Current Device state does not allow processing
HIF_CMD_STATUS_SYSERR	11	System error/Flash write error
HIF_CMD_STATUS_IO	12	General error

3.1.3.1.3 Common Commands

Table 3-3 HIF Command-Response Common MsgID Definitions

MsgID	Description
0x00	Obtain the Device version.
0x01	Get the Device Tick clock to perform defensive detection of the Device Active state.
0x04	Configure a radar reset and restart.
0x05	Configure the Device sleep state.
0x07	Configure the radar to start or stop detection.
0x0C	For passive reporting, the Host sends the POLL command to the Device.
0x42	Dynamically obtain Device configuration information. Some samples do not use this ID.

MsgID	Description
0x60	Dynamically configure Device radar or I/O parameters. Some samples do not use this ID.

For the specific Payload definition of each Command, see the appendix Common Command-Response Data Formats.

3.1.3.2 Report Data

This section describes the Payload formats of some defined MsgIDs, primarily radar-related data.

3.1.3.2.1 Cube Data (0xC1)

Radar FFT data is reported using an HIF message with MsgID 0xC1. The Payload is defined in the following figure. The right side shows the complete FFT data for one radar frame, which may be divided into multiple data fragments for transmission. The fragments are transmitted from top to bottom. Each FFT data fragment is prefixed with a report-data header (gray area) and reported as the message Payload, as shown on the left.

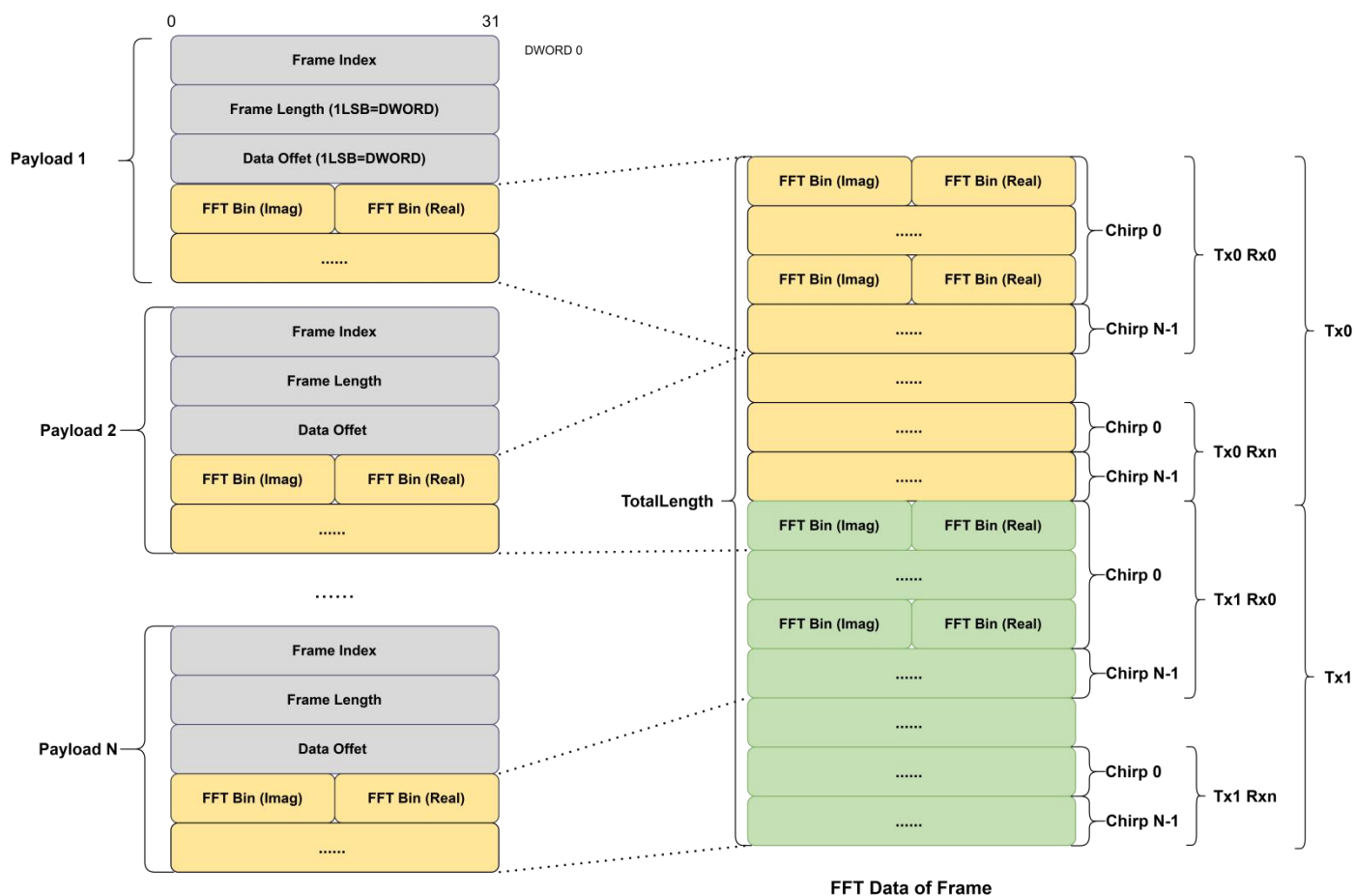


Figure 3-7 C1 FFT Data Message

Table 3-4 C1 FFT Data Message Payload Description

Frame Parameter	Parameter Description
Frame Index	Current frame sequence number. Multiple data Payloads in the same Frame have the same frame sequence number. The type is a 32-bit unsigned integer.
Frame Length	Total length of the complete FFT data for one radar frame, in DWORDs. When the sum of the FFT data lengths in message Payloads with the same frame sequence number (excluding report-data headers) equals this value, transmission of all radar report data for that frame is complete.
Data Offset	Offset of the FFT data in the current packet within the complete data, in DWORDs.

Frame Parameter	Parameter Description
FFT Bin	Each FFT bin is a DWORD (32 bits), with imag in the lower 16 bits and real in the upper 16 bits. Both the real and imaginary values range from -32768 to 32767.

3.1.3.2.2 Multi-Type FFT Data (0xC2)

Radar FFT data is reported using an HIF message with MsgId 0xC2. The Payload is defined in the following figure. The right side shows the complete FFT data for one radar frame, which may be divided into multiple data fragments for transmission. The fragments are transmitted from top to bottom. Each FFT data fragment is prefixed with a report-data header (gray area) and reported as the message Payload, as shown on the left.

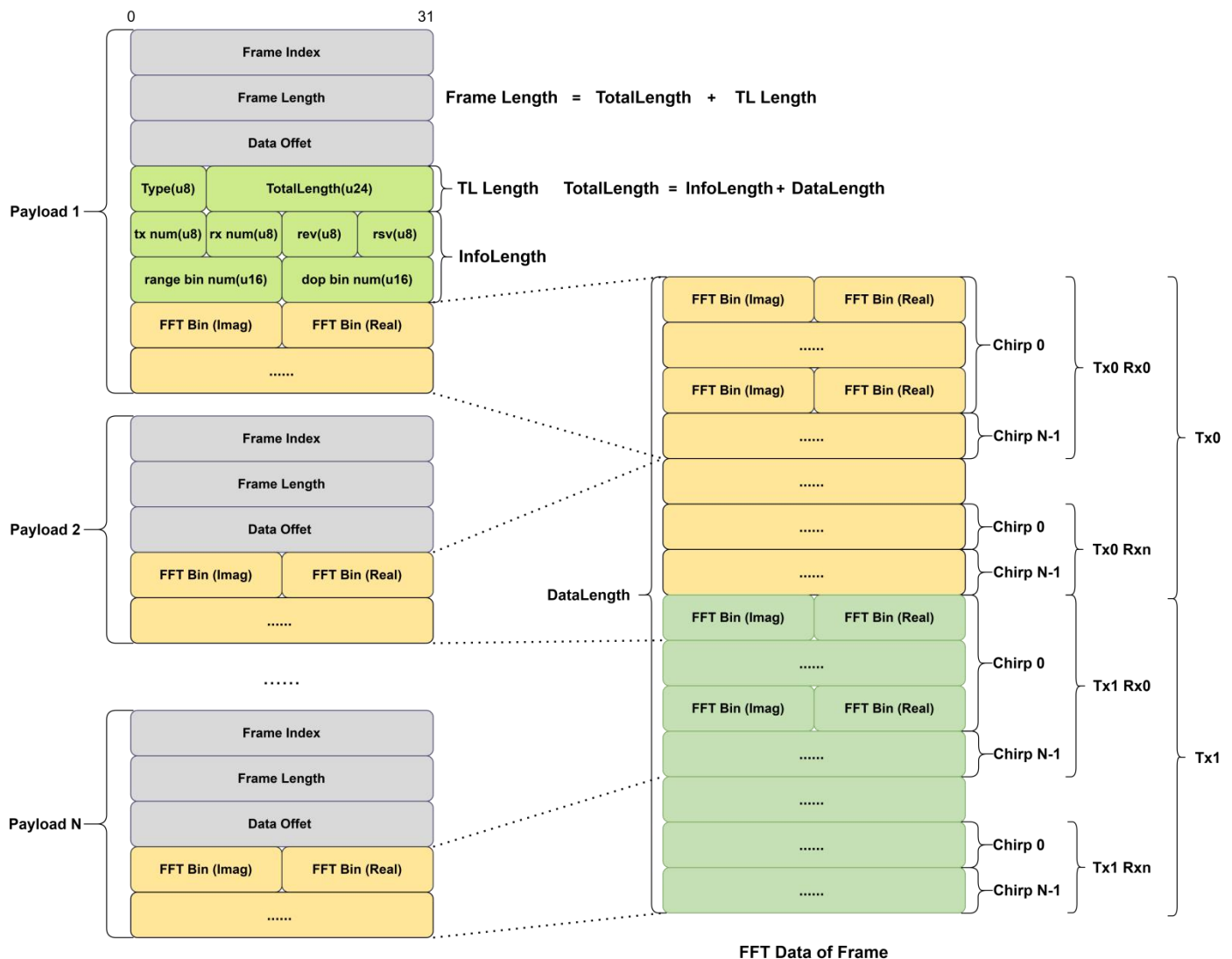


Figure 3-8 C2 FFT Data Message
Table 3-5 C2 FFT Data Message Payload Description

Frame Parameter	Parameter Description
Frame Index	Current frame sequence number. Multiple data Payloads in the same Frame have the same frame sequence number. The type is a 32-bit unsigned integer.
Frame Length	Total length of the complete FFT data for one radar frame, in Bytes. When the sum of the FFT data lengths in message Payloads with the same frame sequence number (including the green Info area but excluding report-data headers) equals this value, transmission of all radar report data for that frame is complete.

Frame Parameter	Parameter Description
Data Offset	Offset of the FFT data in the current packet within the complete data (including the green Info area), in Bytes.
Type(u8)	Packet type (0x0 to 0x2: DataCube data for channels 0 to 2) (little-endian).
TotalLength(u24)	Payload size of the packet, in Bytes. This is the sum of InfoLength and DataLength.
Tx_num(u8)	Number of transmit antennas in the current radar configuration.
Rx_num(u8)	Number of receive antennas in the current radar configuration.
Rev(u8)	Reserved.
Rev(u8)	Reserved.
Range bin num(u16)	FFT Range num.
Dop bin num(u16)	FFT Dop num.
FFT Bin	Each FFT bin is a DWORD (32 bits), with imag in the lower 16 bits and real in the upper 16 bits. Both the real and imaginary values range from -32768 to 32767.

3.1.3.2.3 Point-Cloud Targets (0xC3)

Radar point-cloud data is reported using an HIF message with MsgId 0xC3. The Payload is defined in the following figure. The right side shows the complete point-cloud data for one radar frame, which may be divided into multiple data fragments for transmission. The fragments are transmitted from top to bottom. Each point-cloud data fragment is prefixed with a report-data header (gray area) and reported as the message Payload, as shown on the left.

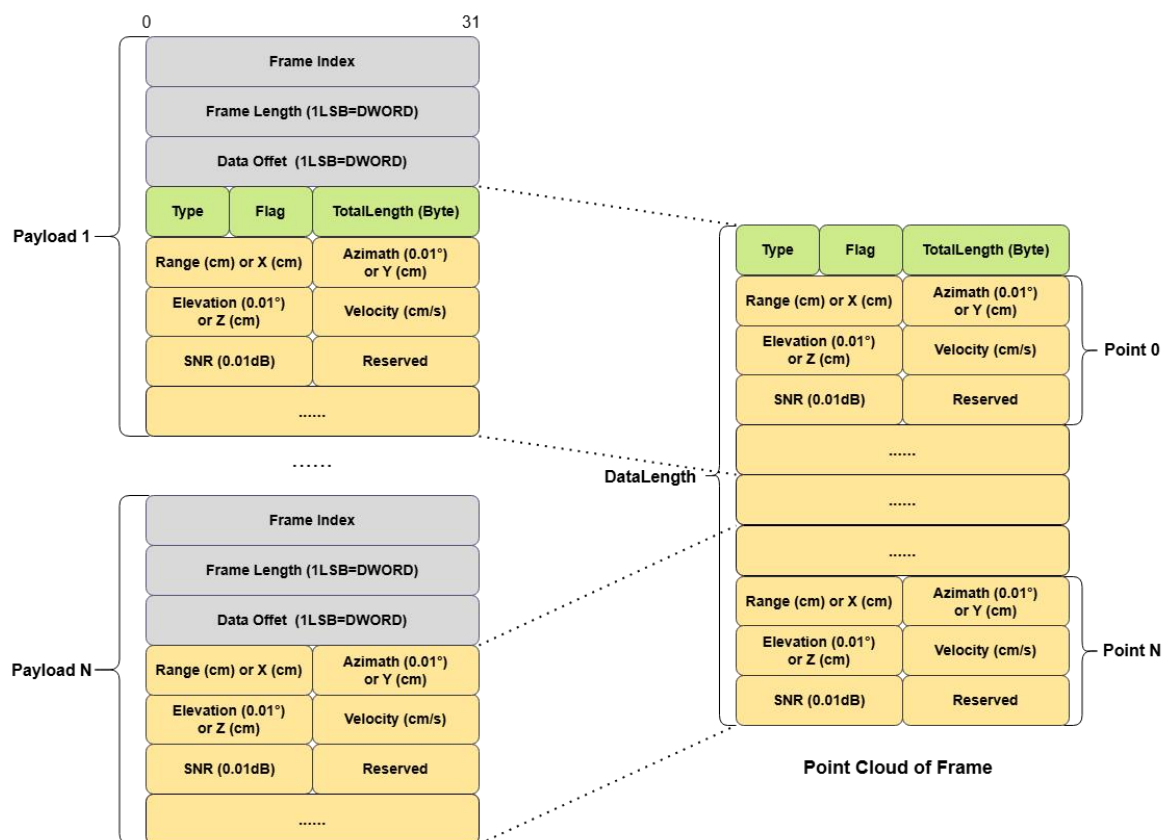


Figure 3-9 Point-Cloud Data Message

Table 3-6 Point-Cloud Data Message Payload Description

Frame Parameter	Parameter Description
-----------------	-----------------------

Frame Parameter	Parameter Description
Frame Index	Current frame sequence number. Multiple data Payloads in the same Frame have the same frame sequence number. The type is a 32-bit unsigned integer.
Frame Length	Total length of the complete point-cloud data for one radar frame, in DWORDs. When the sum of the point-cloud data lengths in message Payloads with the same frame sequence number (excluding report-data headers) equals this value, transmission of all radar report data for that frame is complete.
Data Offset	Offset of the point-cloud data in the current packet within the complete data, in DWORDs.
Type	Data type, an 8-bit unsigned integer. The value is fixed at 4 for point-cloud data.
Flag	Data flag, an 8-bit unsigned integer. When bit0 is 0, the data uses polar coordinates; when bit0 is 1, the data uses Cartesian coordinates; when bit1:bit2 is 00b, the data is motion point cloud; when bit1:bit2 is 00b, the data is presence point cloud.
Total Length	Total point-cloud data length, a 16-bit unsigned integer, in Bytes.
Range(cm) or X(cm)	Range in polar coordinates, in cm; X in Cartesian coordinates, in cm.
Azimuth(0.01°) or Y(cm)	Azimuth in polar coordinates, in 0.01°; Y in Cartesian coordinates, in cm.
Elevation(0.01°) or Z(cm)	Elevation in polar coordinates, in 0.01°; Z in Cartesian coordinates, in cm.
Velocity(cm/s)	The speed of the point cloud, units: cm/s.
SNR	Signal-to-noise ratio of the point, in 0.01 dB.
Reserved	Reserved word

3.1.3.2.4 Debug Data (0xC6)

Within the HIF Payload field, the SDK defines a subprotocol called the "PSIC Debug Protocol" (ID: 0xC6), specifically for reporting flexible debug information. Together with RadarDebugTool, it supports data visualization, storage, and playback.

The PSIC Debug Protocol has the following key features:

- It is a subprotocol within the HIF protocol framework, eliminating the need for developers to create a new application-layer protocol;
- Supports reporting multiple data types, including byte/short/word/float/double, as well as arrays;
- Supports reporting fixed-point bit widths and restoring the fixed-point position on the Host PC;
- Supports defining different data organization methods;
- Supports flexible data labels that distinguish each data type by string;
- Supports reporting up to 4090 bytes of data;
- Supports reporting strings and displaying them in RadarDebugTool.

The relationship between the PSIC Debug Protocol and the HIF Payload field is shown in the following figure:

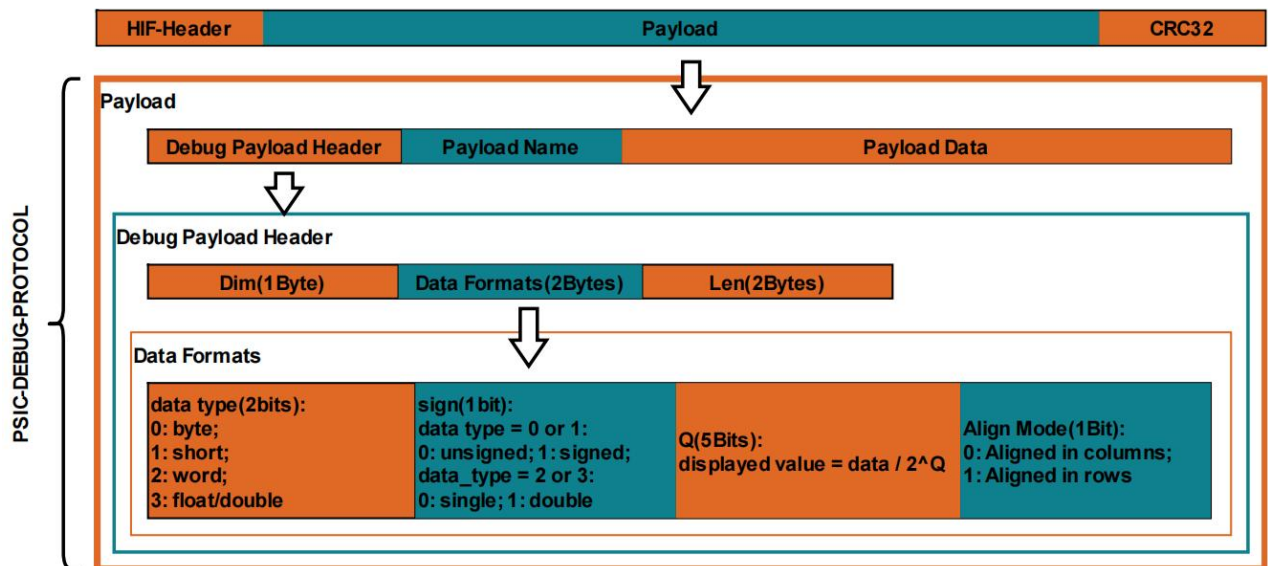


Figure 3-10 PSIC Debug Protocol 0xC6

Table 3-7 PSIC Debug Protocol 0xC6 Payload Field Description

0xC6 Payload Field/Parameter	Description
Dim	Indicates the dimensions of the data to be reported. When Dim = 1/2/3, RadarDebugTool supports data visualization.
Len	Indicates the number of multidimensional data items to be reported. Dim * Len is the total number of reported data items.
Align Mode	Data alignment mode. When Align Mode = 0, Payload Data is populated first with Len values of the same dimension. After one dimension is fully populated, the Len values of the next dimension are populated, until all Dim dimensions are complete; When Align Mode = 1, Payload Data is populated first with Dim values of different dimensions that belong to the same multidimensional data item. The Dim values of the next multidimensional data item are then populated, until all Len multidimensional data items are complete.
Q	Fixed-point bit width: After parsing Payload Data, RadarDebugTool divides the data by 2 ^Q for display and storage. This is useful when the reported data is fixed-point. When the reported data is floating-point, Q should generally be set to 0.
Data Type	Data type. Indicates the data type of each element. Supported data types include byte/short/word/float(double);
Sign	Sign flag and floating-point type. When Data Type is byte/short/word, Sign = 1 indicates that all values in Data Payload are unsigned; Sign = 0 indicates that all data in Data Payload is treated as unsigned by the Host PC; When Data Type is float(double), Sign = 1 indicates that the reported type is double, and Sign = 0 indicates that the reported type is float.
Payload Name	Payload Name indicates the label of the debug data. It is a string and must end with '\0'. This label determines the name used to display and store this data segment in RadarDebugTool.
Payload Data	Populate Align Mode/Dim/Len with data

3.1.3.3 Fragmentation

When the frag field in MsgHeader is 1, the Frame is a fragmented Frame derived from a large data packet. The Host must reassemble all fragmented Frames belonging to the same large packet. The Payload structure of a fragmented Frame is as follows:

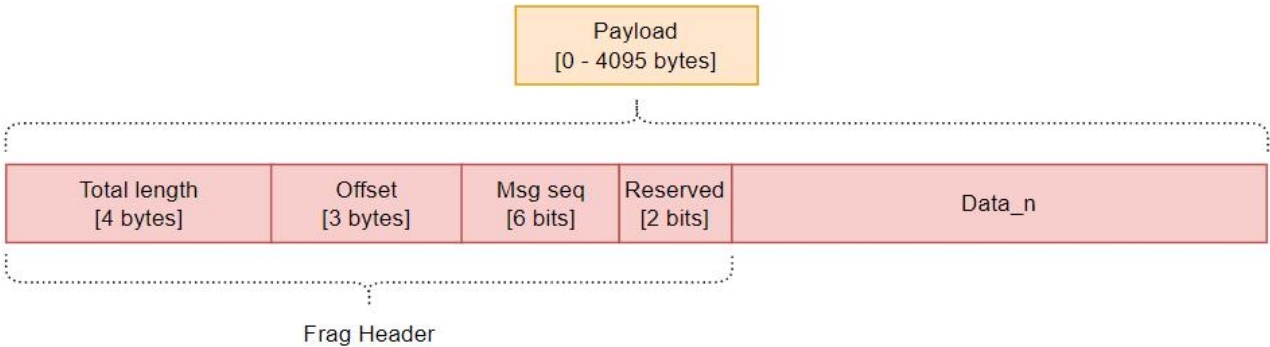


Figure 3-11 Payload Structure of a Fragmented Frame

Frag Header, that is, the fragmentation header, is used for fragmentation control. The fields are defined as follows:

Table 3-8 Frag Header Description

Frag Header Field	Description
Total length	Total length of all fragmented data (Data_n). This field is the same in each fragment.
Offset	Offset/position of the current fragmented data (Data_n).
Msg seq	Message sequence number. This field is the same in each fragment. The MsgID field in MsgHeader together with Msg seq identifies the fragmented Frame to which this fragment belongs.
reserved	Reserved bit.

3.1.3.3.1 Fragmentation Example

Assume that the maximum Payload length of one HIF Frame is 4095 bytes and the data to be sent is 8190 bytes. The data must be divided into three fragments, each of which is encapsulated in a Frame according to the protocol.

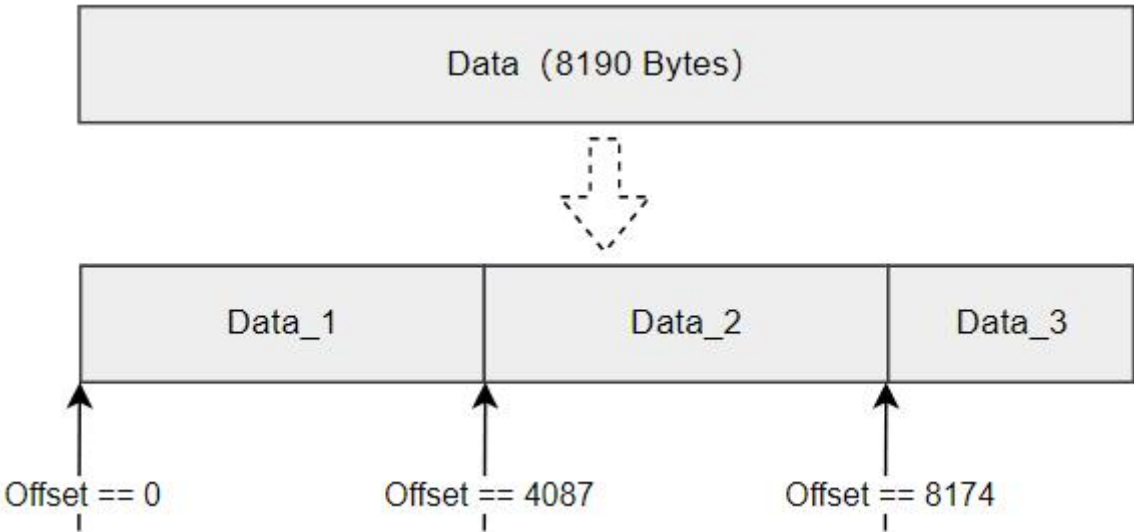


Figure 3-12 Example of Fragmenting Data to Be Sent

The Payload of each fragmented Frame is shown in the following figure:

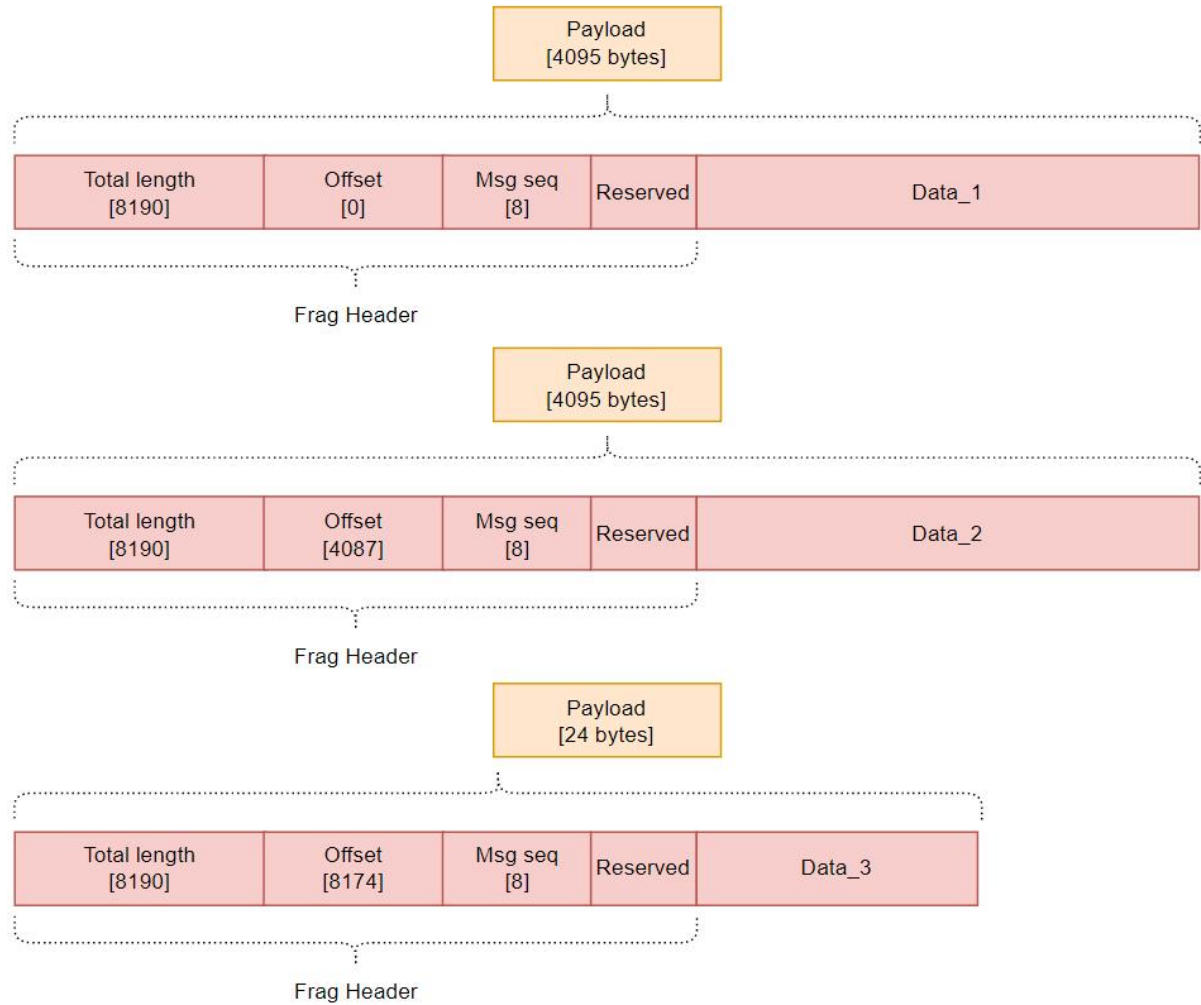


Figure 3-13 Fragmented-Frame Payload Example

3.1.4 HIF Protocol Frame Example

A message that conforms to the HIF protocol frame format:

```
A5 2B 32 C2 0C 2F 00 00 00 0C ... 00 00 01 00 E8 F3 FA FF
```

Header: A5 2B 32 C2 0C 2F. The fields are parsed as follows:

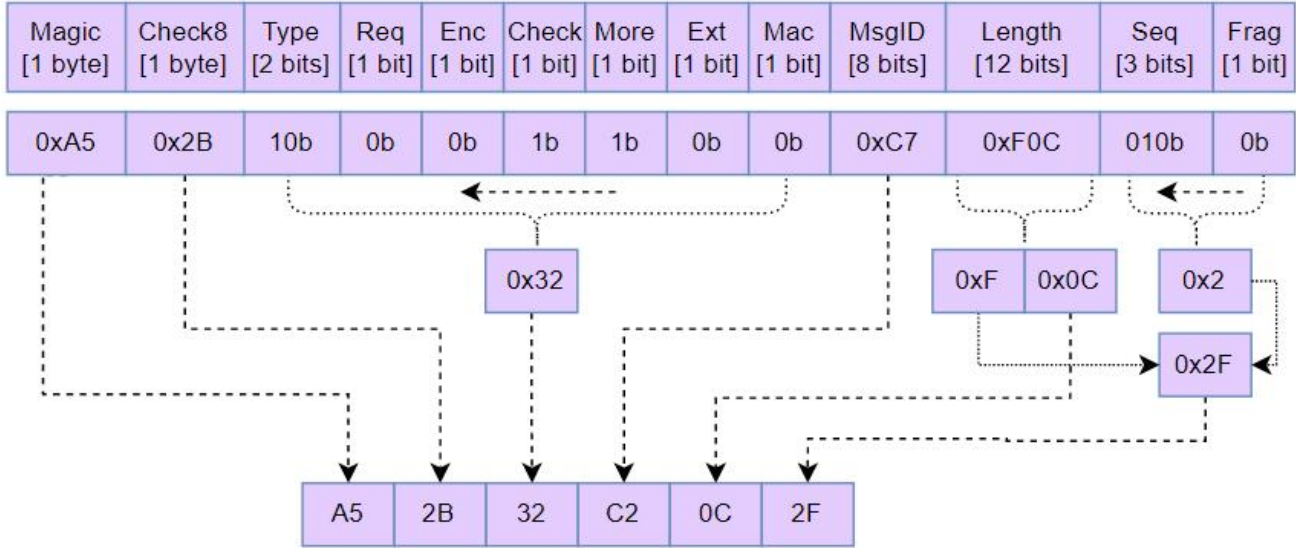


Figure 3-14 HIF Header Example (Little-Endian)

Payload: 00 00 00 0C ... 00 00 01 00.

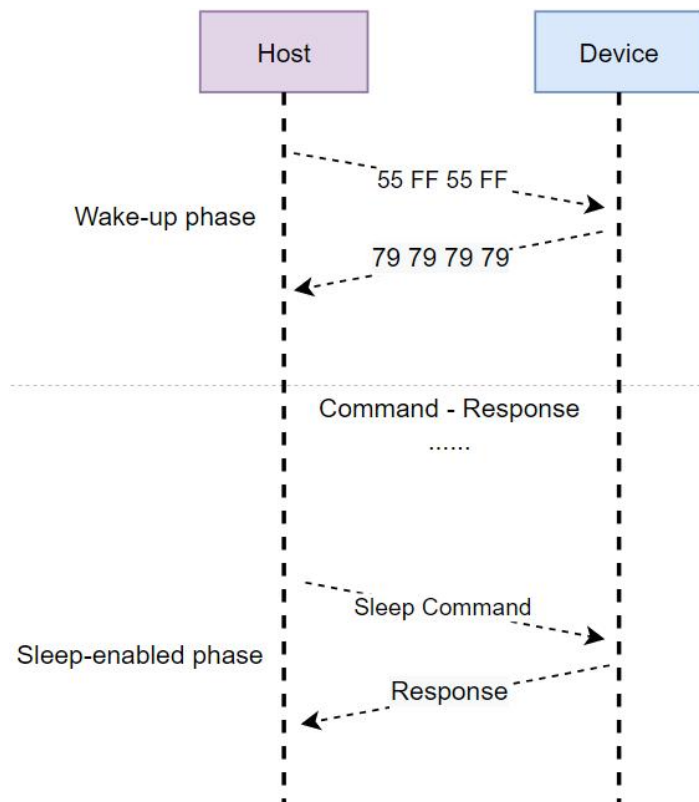
Checksum: E8 F3 FA FF. Converted to uint32, the value is 0xFFFFAF3E8 (little-endian).

3.2 Host-Device Interaction

3.2.1 Wake-Up and Sleep

Before a Command-Response interaction between the Host and Device, the Host cannot determine whether the Device is asleep. Therefore, the Host must wake the Device before the interaction; this is the wake-up substage. After the Command-Response interaction is complete, the Device must be allowed to sleep through an HIF Command (MsgID = 0x05); this is the sleep-release substage.

The wake-up and sleep interaction flow is as follows:

**Figure 3-15 Wake-Up and Sleep Interaction Flow**

3.2.1.1 Wake-Up

The Host wakes the Device through the communication interface. The wake-up data is as follows:

Wake sequence:

55 FF 55 FF

After waking up, the Device replies 79 79 79 79 to the Host.

ACK sequence:

79 79 79 79

Because the Device is asleep, it may fail to receive wake-up data sent by the Host. The Host may therefore need to retry until the Device replies 79 79 79 79, confirming that wake-up succeeded.

3.2.1.2 Sleep

Sleep Command: ID - 0x05; Payload (1 Byte) - 0x01, which allows sleep. Other fields follow the HIF Frame format. For example:

A5 CF 15 05 01 70 01 E9 FA FE 8F

Response: ID - 0x05; Payload (1 Byte) - execution status; see [Command-Response](#); other fields follow the HIF Frame format. For example:

A5 02 12 05 01 00 00 ED FA FE BF

Here, 00 indicates an execution status of Success.

The timing flows for wake-up and sleep over different physical communication interfaces are as follows.

3.2.1.3 SPI

3.2.1.3.1 Wake-Up

Wake-up flow:

1. The Host sends 55 FF 55 FF;
2. The Host waits 1 ms and reads four data bytes. If all four bytes are 0x79, wake-up is successful, the Device is in the Active state, and the wake-up flow ends.
3. If the four bytes read are not 0x79, wait 10 ms and repeat Steps 1 and 2.

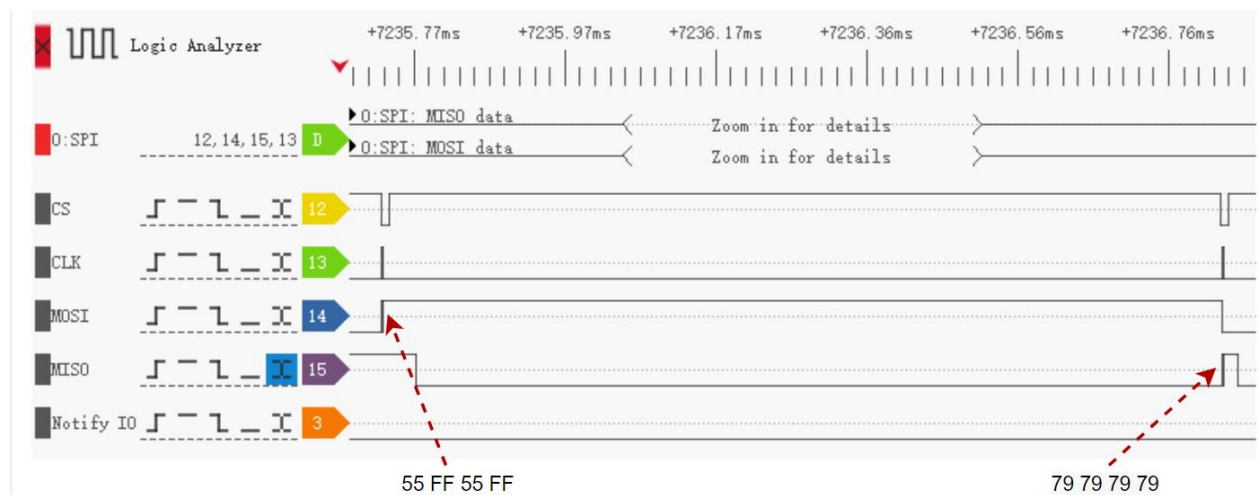


Figure 3-16 SPI Wake-Up Timing

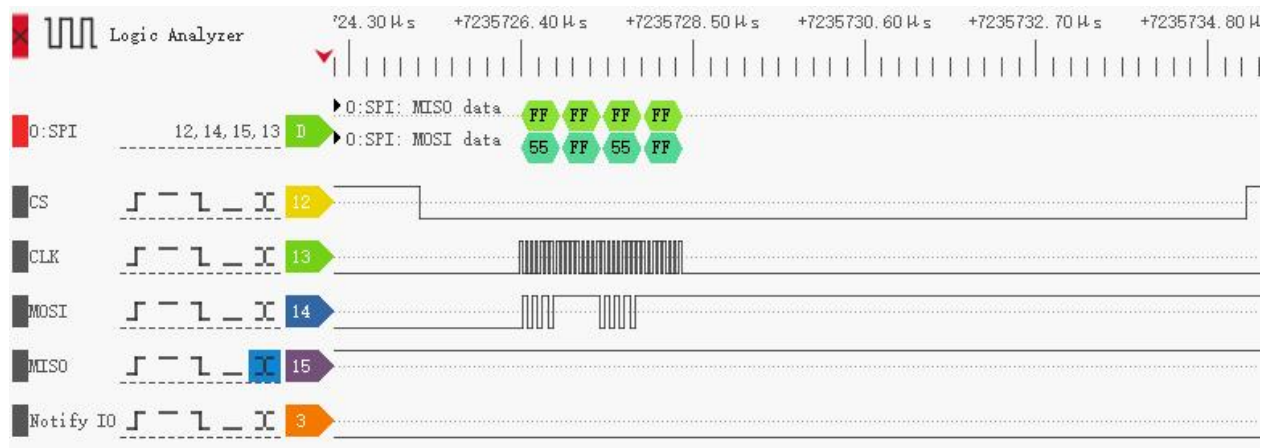


Figure 3-17 SPI Timing Diagram for Data 55 FF 55 FF



Figure 3-18 SPI Timing Diagram for Data 79 79 79 79

3.2.1.3.2 Sleep

The sleep flow is effectively a Command-Response flow.

This section describes the SPI data timing for the sleep Command-Response. For the complete interaction flow, refer to the Command-Response section below.

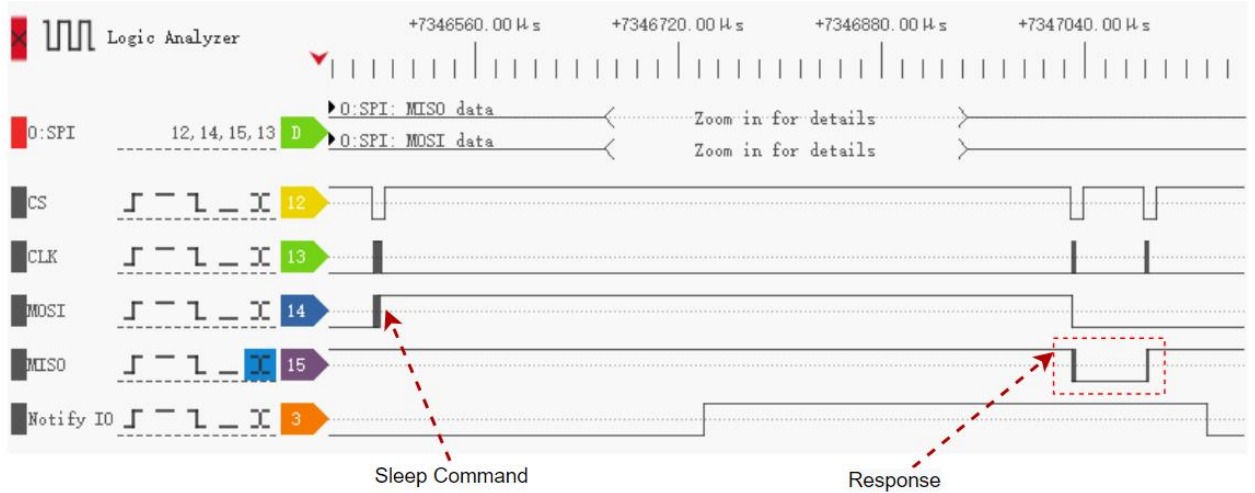


Figure 3-19 SPI Timing Diagram for Command-Response

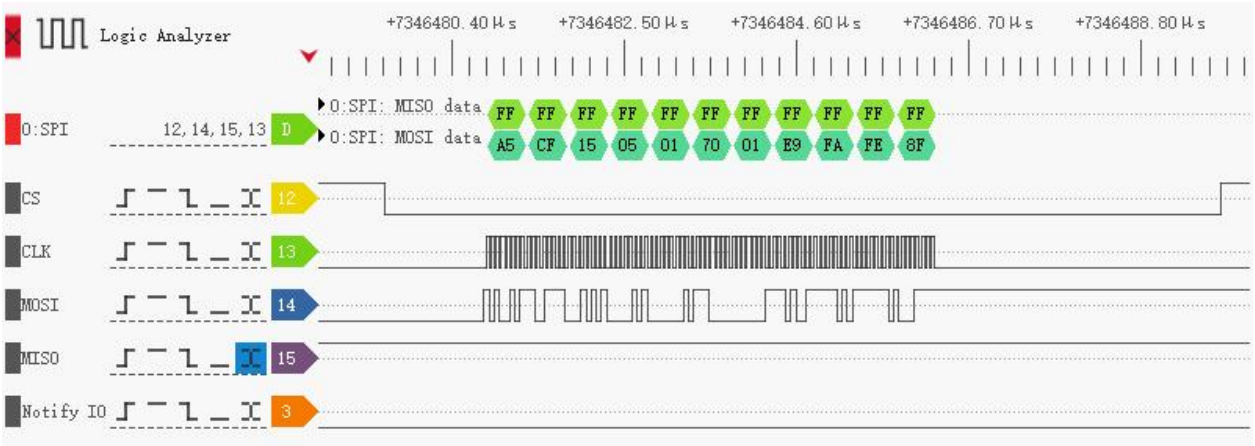


Figure 3-20 SPI Timing for Sleep Command Data

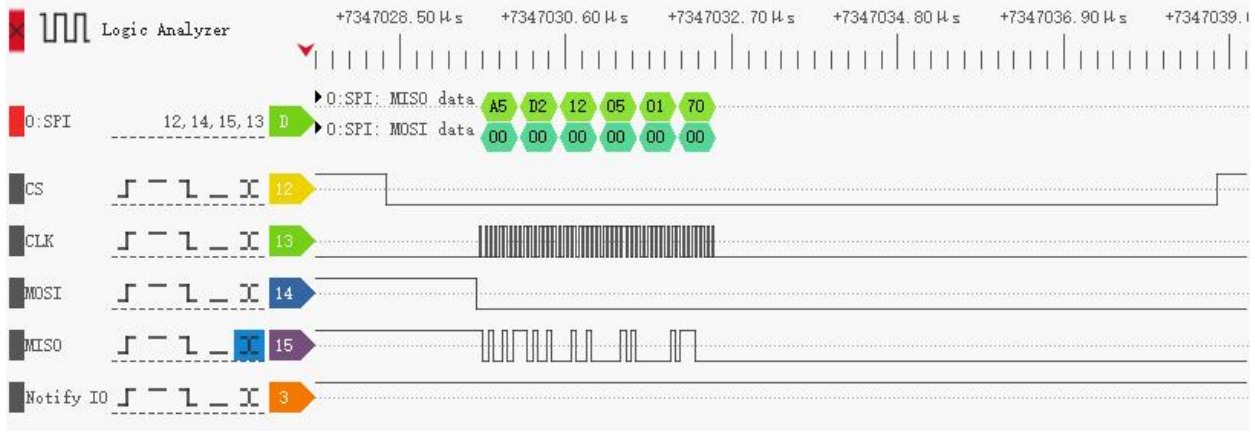


Figure 3-21 SPI Timing for Response Data Header

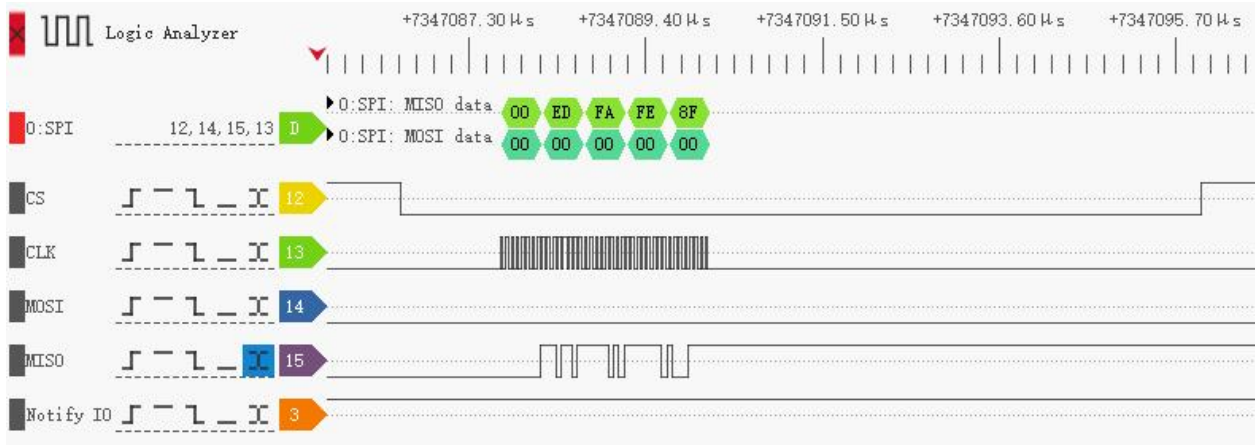


Figure 3-22 SPI Timing for Response Data Payload + CheckSum

3.2.1.4 UART

3.2.1.4.1 Wake-Up

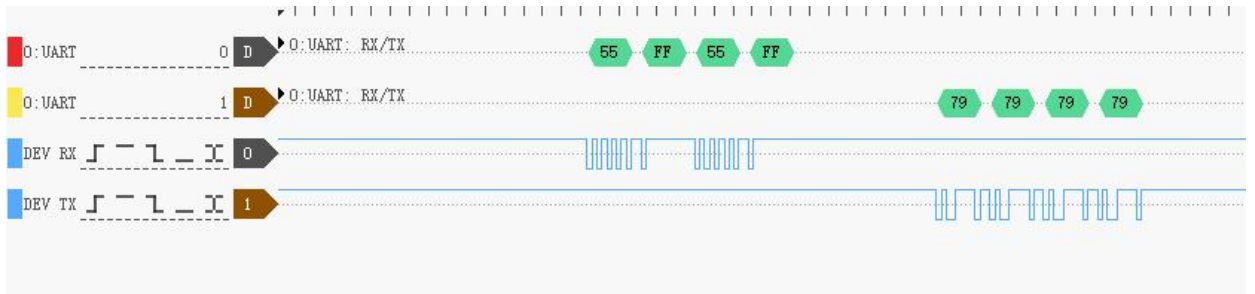


Figure 3-23 UART Wake-Up Timing

3.2.1.4.2 Sleep



Figure 3-24 UART Timing Diagram for Command-Response

3.2.1.5 I²C Communication

The wake-up and sleep flows are the same as those for SPI.

3.2.2 Command-Response

Prerequisite: The Device is in the Active state. If the Device is asleep, the Host must first wake the Device. See [Wake-Up and Sleep](#) in the preceding section.

Format: See [HIF Protocol Frame Format](#) under [Command-Response](#) in the preceding section.

Interaction: In general, Command-Response interactions occur in pairs, with one transmission and one reply. The Host sends a Command to the Device, and the Device replies with the execution status and data after receiving it. Commands cannot be processed concurrently. The Device receives and processes only one Command at a time and starts receiving the next Command only after processing of the current Command and transmission of its Response are complete.

Command-Response timing diagrams for different physical communication types are described below.

3.2.2.1 SPI

The interaction flow is as follows:

1. The Device receives a Command from the Host;
- ② The Device processes the data. When processing is complete, it drives the I/O high to notify the Host that the Response data can be read;
3. The Host reads the data;
- ④ When data reading is complete, the Device drives the I/O low.

If the Host does not read the data after Step ②, the Device discards the data after a configurable timeout.

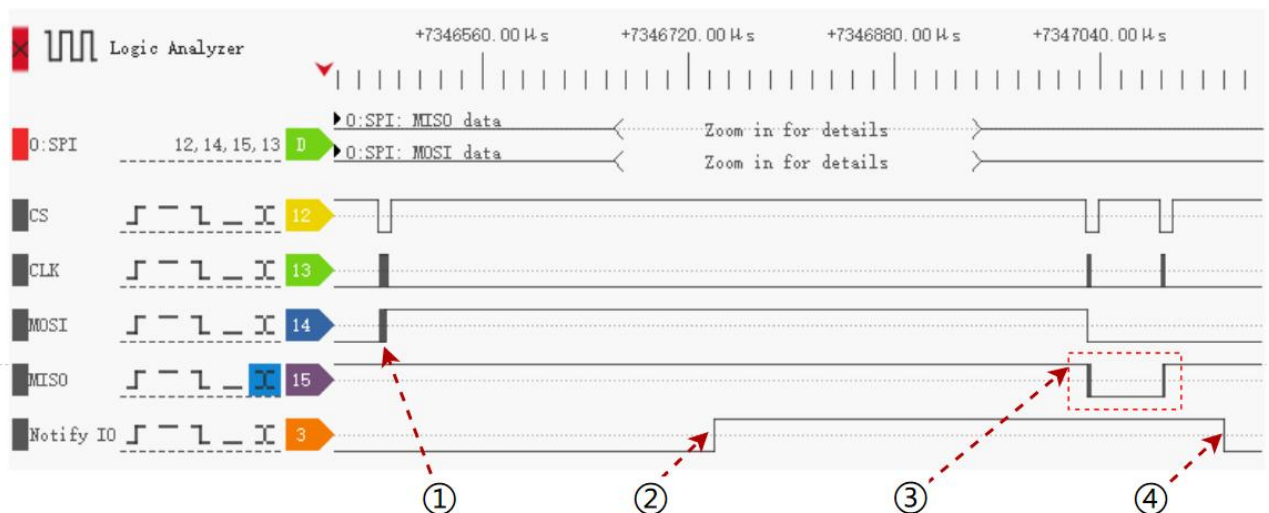


Figure 3-25 SPI Timing Diagram for Command-Response

3.2.2.2 UART

The interaction flow is the same as that described in the UART sleep section above.

3.2.2.3 I²C

The flow is the same as that for SPI.

3.2.3 Data Reporting

Data reporting is divided into active and passive reporting.

Active reporting applies to active communication interfaces such as UART. During active reporting, the Device sends data directly to the Host with minimal interaction.

Passive reporting applies to passive communication interfaces such as SPI and I²C. Passive reporting can also be used in UART mode. The flow is as follows:

1. The Device drives Notify I/O high to wake the Host, indicating that the Device has data to report;
2. The Host sends a Poll Command to the Device;

3. After receiving the Poll Command from the Host, the Device drives Notify I/O low;
4. The Device prepares the data. When the data is ready, the Device drives Notify I/O high to notify the Host that it can retrieve the data;
5. The Host reads one data Frame and checks the Header more field and Notify I/O:
 - (1) If more is 1 and Notify I/O is high, repeat Step 5 and subsequent steps;
 - (2) If more is 1 and Notify I/O is low, return an error;
 - (3) If more is 0 and Notify I/O is high after a delay of 50 μ s, return to Step 2;
 - (4) If more is 0 and Notify I/O is low after a delay of 50 μ s, end the transmission.

If the Host does not read the data after Step 4, the Device discards the data after a timeout.

After the Device sends all data, it sends a NullFrame (empty packet) at the end.

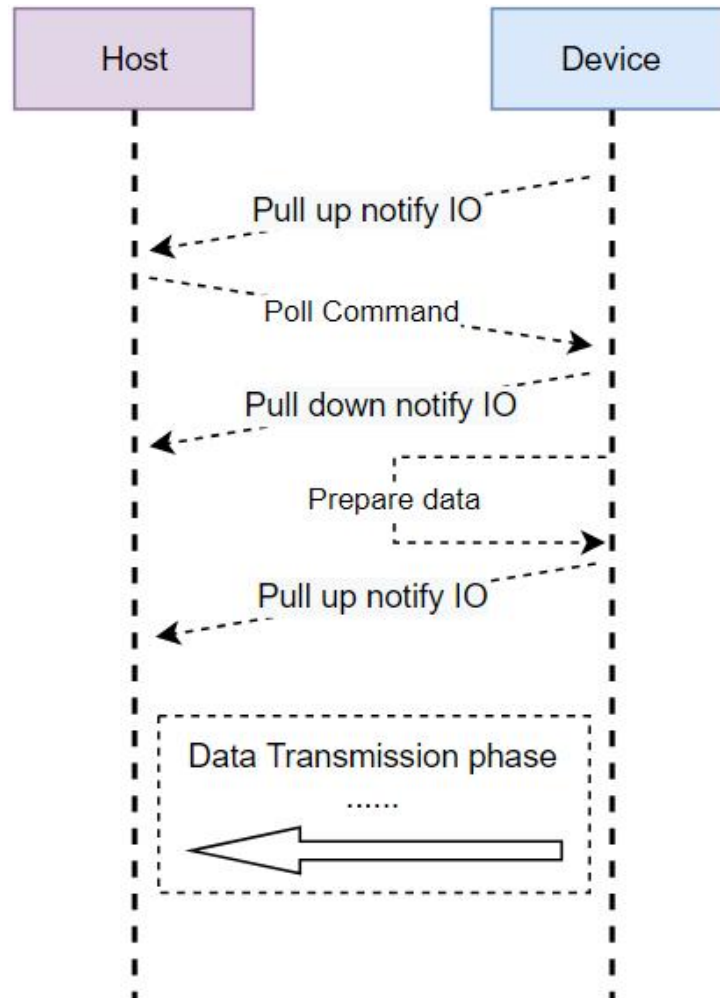


Figure 3-26 Data Reporting Interaction Flow

3.2.3.1 SPI

SPI report-data timing:

- ① The Device drives Notify I/O high to wake the Host, indicating that the Device has data to report;
- ② The Host sends a Poll Command to the Device;
- ③ After receiving the Poll Command from the Host, the Device drives Notify I/O low;
- ④ The Device prepares the data. When the data is ready, the Device drives Notify I/O high to notify the Host that it can retrieve the data;
- ⑤ The Host reads one data Frame and checks the Header more field and Notify I/O;
- ⑥ After the Device finishes sending data, it drives the I/O low.

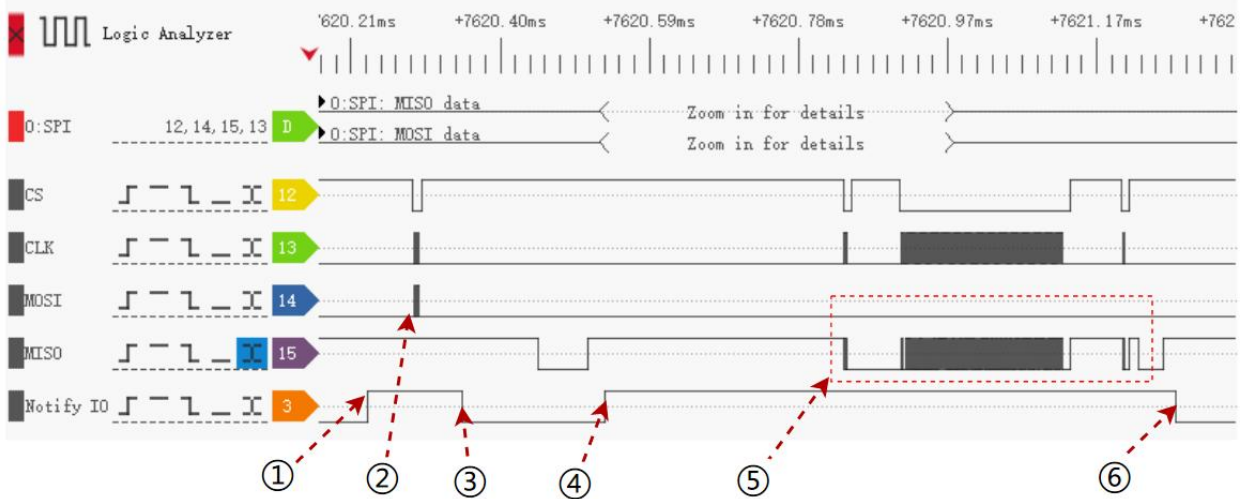


Figure 3-27 SPI Report-Data Timing

3.2.3.2 UART

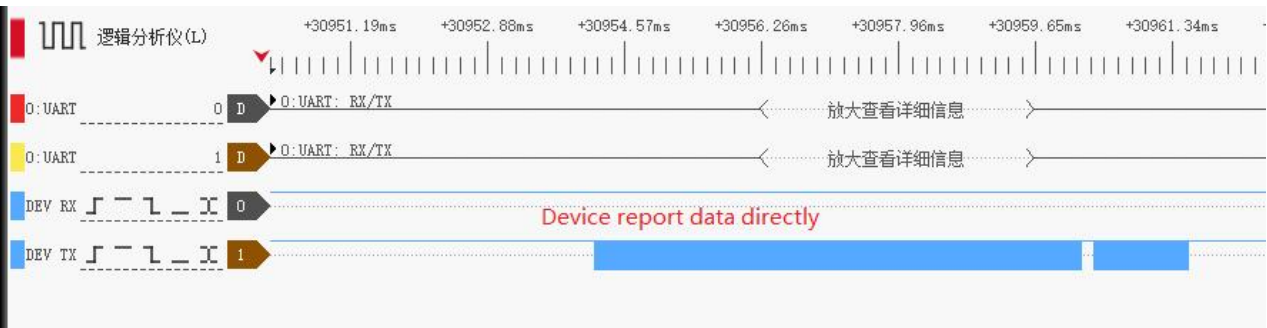


Figure 3-28 UART Report-Data Timing

3.2.3.3 I²C

Same as SPI.

3.2.3.4 Transport-Layer Retransmission

When physical bus signals are unstable, HIF provides transport-layer retransmission to ensure correct data transmission.

Transport-layer retransmission must be supported by both the Device and Host: enable CONFIG_HIF_RETRY_ENA on the Device and CFG_TL_RETRY_EN on the Host. See the Host and Device sections below for details.

3.2.4 Exception Recovery

3.2.4.1 The Host Cannot Receive the Message Header When Receiving Report Data

When the Host receives report data, the first byte is normally 0xA5 and the six received bytes conform to the message-header format. However, if an incorrect operation prevents the correct message header from being received, state synchronization should be performed by continuously receiving data. When 4 KB of 0xFF or 0x00 is received, state synchronization between the Host and Device is considered successful.

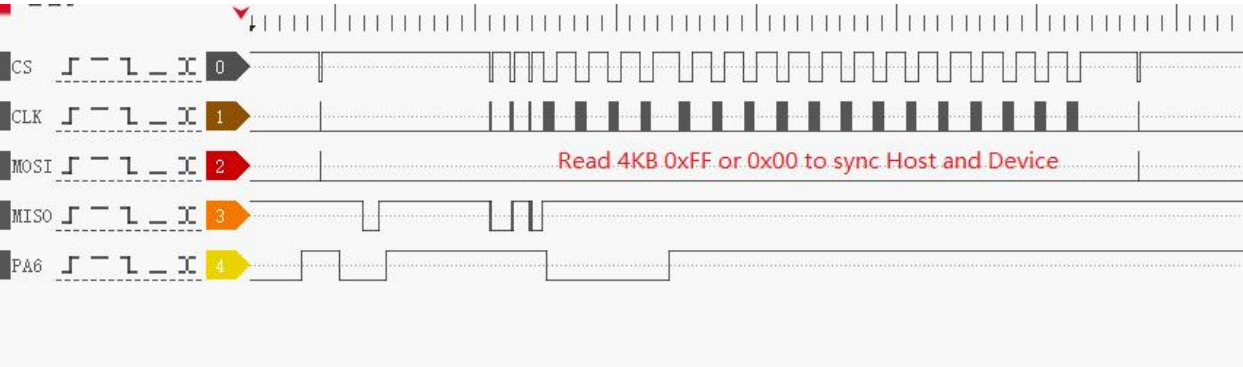


Figure 3-29 Host-Device State Synchronization

4 Software Framework

4.1 Protocol Architecture

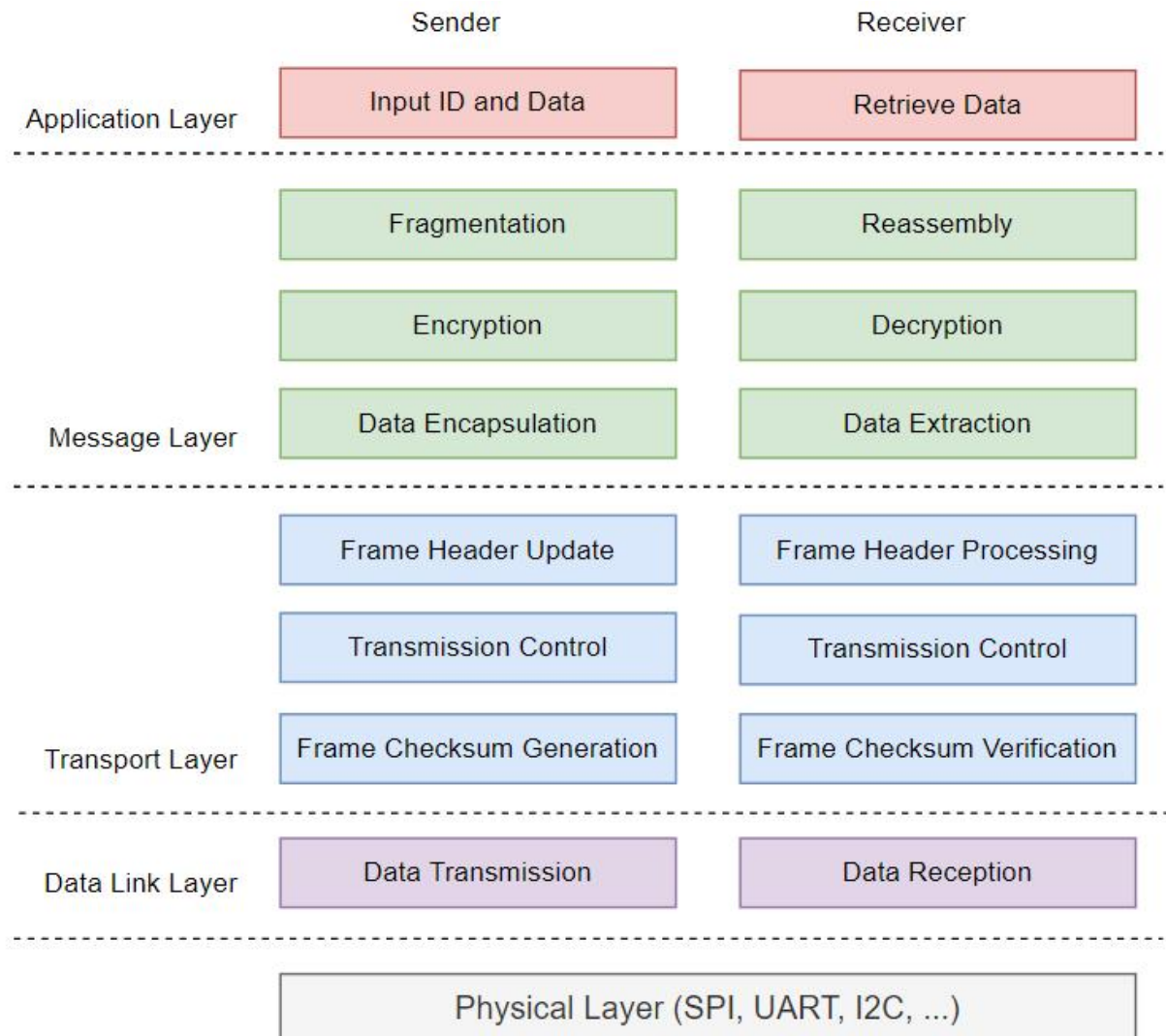


Figure 4-1 Protocol Architecture

4.1.1 Message Layer

Main functions:

Provides message transmission to the application layer:

- Encapsulate the ID and message data into a Frame;

- Message fragmentation;

- Encrypt (not supported in 2.1.0);

- Call the transport-layer interface to pass the Frame to the transport layer.

Provides data reception to the application layer:

- Receive a Frame from the transport layer;

- Decrypt (not supported in 2.1.0);

- Reassemble fragmented messages;

- Distribute messages to the application layer.

4.1.2 Transport Layer

The transport layer controls HIF Frame transmission and Notify I/O, and handles interactions such as wake-up, sleep, and Command-Response.

Transmit Frame:

- Receive the Frame passed from the message layer;
- Calculate control and checksum fields such as Seq, Check8, and CheckSum;
- Control interaction, retransmission, response, timeout handling, and other operations for all Frames;
- Pass the Frame to the link layer.

Receive Frame

- Receive the Frame passed from the link layer;
- Validate CheckSum;
- Extract MsgHeader and Payload;
- Pass MsgHeader and Payload to the message layer.

4.1.3 Link Layer

Manages and controls the physical layer and implements data transmission and reception. For received data, the link layer validates the Frame Header.

4.1.4 Physical Layer

The HIF in the current SDK version supports communication interfaces such as SPI, UART, and I²C. DMA is supported for SPI.

4.2 Software Architecture

4.2.1 Host

The HIF Host-side software architecture is shown in the following figure. It consists of four layers: hardware abstraction layer (HAL), link layer (LLC), middleware (MID), and application layer (APP). After porting the driver implementations (HAL interfaces), the vendor can directly use the APP-layer interfaces to implement functions.

The HAL provides operating-system, GPIO, storage-medium, bus COM, logging, and sleep operation functions. Bus COM must be adapted according to the specified format to abstract different buses into a set of bus-control interfaces (ops) usable by LLC.

LLC defines COM_BUS and COM_DEVICE internally to manage buses and Devices, respectively. Different Devices are ultimately abstracted as Devices with one of two notification types: event notification and data notification. A Device that reports passively uses event notification, whereas a Device that reports actively uses data notification.

Based on LLC event notification, data notification, and data-transmission functions, MID implements HOST HIF communication and the BROM download protocol.

APP integrates the functions provided by the lower layers, simplifies operation, and provides convenient APIs.

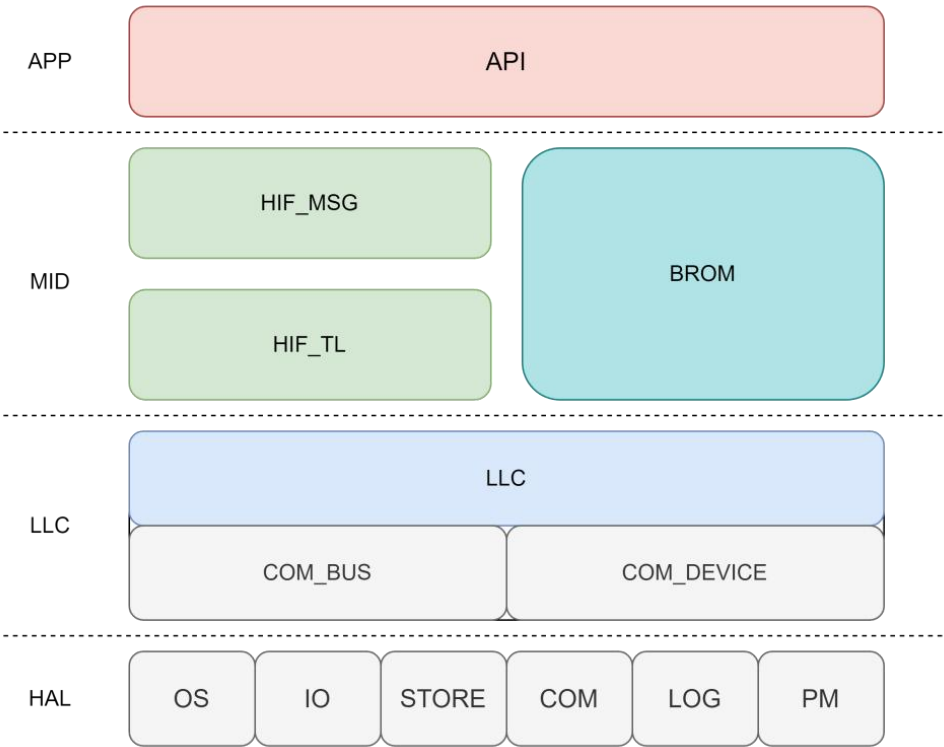


Figure 4-2 Host Software Architecture

4.2.2 Device

The HIF Device-side software architecture is shown in the following figure. The functions of each layer or module are generally consistent with the protocol architecture. HIF MEMORY is the Device-side HIF memory-resource management module.

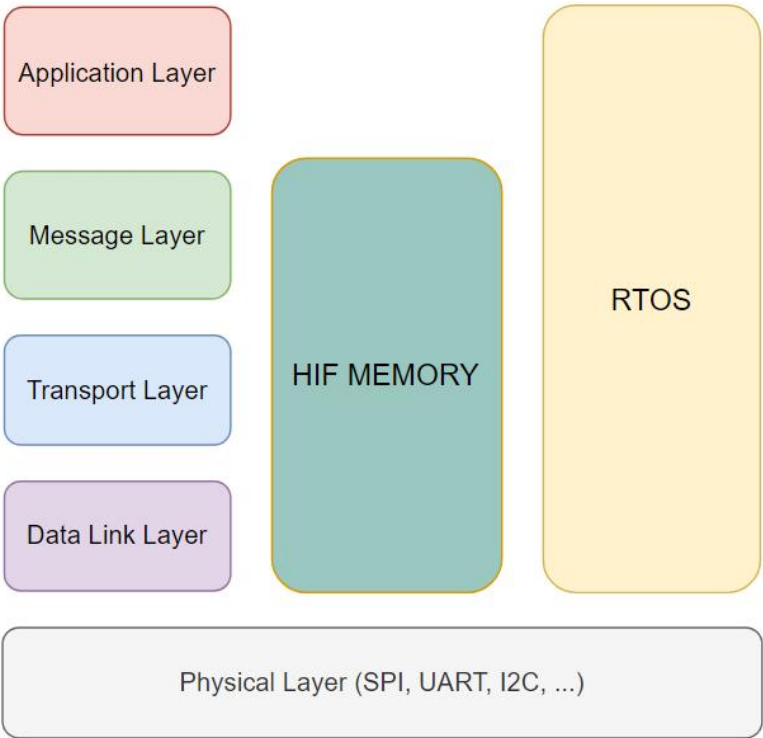


Figure 4-3 Device Software Architecture

5 Host Development

5.1 Host Driver Porting

The Host Driver supports communication with a single slave or multiple slaves. Multiple-slave communication requires only registering, configuring, and opening the additional slaves based on the single-slave configuration. Slaves on different buses can be used simultaneously.

If the vendor folder is named AAA, compile the following files. The host_driver folder path is "<home>/Software_Kit/02_SDK/psdf_sdk/project/subsys/hif/host_sample/host_driver". A path ending with "/" indicates a folder; otherwise, it indicates a file.

```
host_driver/host_config.h
host_driver/user_config.h
host_driver/cmd_api/
host_driver/porting/host_port.h
host_driver/porting/include/
host_driver/porting/AAA/
host_driver/protocol/
```

5.1.1 File Structure

```
<home>/Software_Kit/02_SDK/psdf_sdk/project/subsys/hif/host_sample/host_driver
├── host_config.h                /* host driver configuration file */
├── user_config.h                /* User configuration file that can override host_config.h
configuration items */
├── cmd_api                      /* Application-layer file directory */
│   ├── host_driver.c            /* host driver initialization, Device registration and opening,
general Command transmission, firmware download, and other functions */
│   ├── host_driver.h            /* Declaration */
│   └── rs6x_7x                  /* Directory for rs6x_7x Command transmission and
report-data processing files */
│       ├── host_mmw_api.h       /* Declaration */
│       ├── host_mmw_cmd_api.c   /* General Commands and radar analysis Commands */
│       └── host_mmw_report_api.c /* C1/C2/C3 data packet assembly and C6/C7/C8/CA/CB
data-packet callbacks */
├── porting                      /* Adaptation-layer file directory */
│   ├── host_port.h              /* Adaptation-layer header file */
│   ├── include                  /* Adaptation-layer interface-definition directory */
│       ├── hal_com.h            /* Communication interface-set definition */
│       ├── hal_io.h             /* I/O operation interface definition */
│       ├── hal_log.h            /* Logging interface */
│       └── hal_os.c              /* Virtual-function definitions for some system operation
```

interfaces */	
├─ hal_os.h	/* System operation interface definitions */
├─ hal_store.h	/* Storage-medium interface definitions */
├─ psic	/* MRS6130_P1812porting example directory */
├─ port.c	/* Provides porting_get_com_ops function implementation for use by LLC */
├─ port_io.c	/* I/O adaptation-layer interface implementation */
├─ port_log.h	/* Provides host_log_print function implementation */
├─ port_os.c	/* Provides system-operation interface implementations */
├─ port_store.c	/* Provides storage-medium interface implementations */
├─ port_com_no_alloc.c	/* COM adaptation layer without memory allocation,for testing */
├─ port_i2c.c	/* I ² C COM adaptation layer */
├─ port_spi.c	/* SPI COM adaptation layer */
├─ port_uart_com_irq_thread.c	/* COM adaptation layer for UART active reporting with COM_IRQ_THREAD notification */
├─ port_uart_com_isr.c	/* COM adaptation layer for UART active reporting with COM_ISR notification */
├─ port_uart_edge.c	/* COM adaptation layer for UART passive reporting with EDGE notification */
├─ protocol	/* Protocol-layer file directory */
├─ burn	/* Download-protocol file directory */
├─ host_brom.c	/* AS2525 download protocol */
├─ host_brom.h	/* AS2525 download protocol */
├─ host_brom_comm_struct.h	/* AS2525 download protocol command-structure definitions */
├─ host_burn.c	/* Firmware download framework */
├─ host_burn.h	/* Firmware download framework */
├─ host_img_struct.h	/* Firmware structure definitions */
├─ hif	/* HIF transport-layer and message-layer file directory */
├─ hif_type.h	/* HIF message-structure definitions */
├─ host_hif.c	/* HIF message layer (MSG) */
├─ host_hif.h	/* HIF message layer (MSG) */
├─ host_hif_tl.c	/* HIF transport layer (TL) */
├─ host_hif_tl.h	/* HIF transport layer (TL) */
├─ llc	/* Link-layer file directory */

└─ host_llc.c	/* Link layer (LLC) */
└─ host_llc.h	/* Link layer (LLC) */
└─ host_llc_com_bus.c	/* Bus abstraction layer */
└─ host_llc_com_bus.h	/* Bus abstraction layer */
└─ host_llc_com_device.c	/* Device abstraction layer */
└─ host_llc_com_device.h	/* Device abstraction layer */
└─ utils	/* Basic data types, linked lists, error codes, and common macros */
└─ host_slist.h	/* Linked List */
└─ host_std_includes.h	/* Basic data types */
└─ host_types.h	/* Error codes and common macros */

The host driver code package contains three folders and two files. The porting folder contains porting-related content corresponding to the HAL in the code framework. The protocol folder contains protocol-related content corresponding to LLC and MID. The cmd_api folder contains APIs available to vendors and corresponds to APP. The host_config.h and user_config.h files are global host driver configuration files. Configurations in user_config.h override those in host_config.h, and vendors can make adjustments in user_config.h.

The vendor must complete the porting work. The porting/psic folder contains the porting code for MRS6130_P1812. After completing the port, the vendor can use the APIs in the cmd_api folder to interact with the Device.

5.1.2 Porting Requirements

Interfaces to be ported are declared in the porting/include folder. A vendor-specific porting-code folder can be created under the porting directory.

If the vendor hardware requires memory alignment when using DMA or other features, modify the `_ALIGNAS_VARIABLE` macro in `host_config.h` at the root of the code package. This macro controls variable memory alignment. If alignment is not required, define this macro as an empty macro.

5.1.2.1 System Interfaces

5.1.2.1.1 Critical-Section Protection

The vendor must adjust the `host_os_critical_flag_t` variable as appropriate. It is defined as unsigned long by default. If no flag is required, the parameter does not need to be used in the function implementation.

Table 5-1 host_os_enter_critical

Interface	Description
Function	<code>host_os_critical_flag_t host_os_enter_critical(void);</code>
Functionality	Enter the critical section
Parameters	None
Considerations	<code>host_os_critical_flag_t</code> is unsigned long by default and can be modified
Return Value	<code>host_os_critical_flag_t</code> , used to restore the state when exiting the critical section

Table 5-2 host_os_exit_critical

Interface	Description
Function	<code>void host_os_exit_critical(host_os_critical_flag_t flag);</code>

Functionality	Exit the critical section
Parameters	host_os_critical_flag_t flag Return value from entering the critical section
Considerations	host_os_critical_flag_t is unsigned long by default and can be modified
Return Value	None

5.1.2.1.2 Timestamp

Table 5-3 host_os_timestamp_us

Interface	Description
Function	host_os_time_t host_os_timestamp_us(void);
Functionality	Obtain a timestamp, in microseconds
Parameters	None
Considerations	None
Return Value	host_os_time_t, timestamp; u32 is used by default and can be modified

5.1.2.1.3 Memory Operations

Table 5-4 host_os_malloc

Interface	Description
Function	void * host_os_malloc(uint32_t size);
Functionality	Allocate memory
Parameters	uint32_t size Size of memory to allocate, in Bytes
Considerations	None
Return Value	Memory address; returns NULL if no memory is available

Table 5-5 host_os_free

Interface	Description
Function	void host_os_free(void *pmem);
Functionality	Free memory
Parameters	void *pmem Address of memory to free
Considerations	None
Return Value	None

Table 5-6 host_os_memset

Interface	Description
Function	void * host_os_memset(void *addr, uint8_t val, uint32_t size);
Functionality	Set memory data
Parameters	void *addr

	Address of memory to set uint8_t val Value to set uint32_t size Length to set, in Bytes
Considerations	None
Return Value	Memory address, identical to the input memory address

Table 5-7 host_os_memcpy

Interface	Description
Function	void * host_os_memcpy(void *dst, const void *src, uint32_t size);
Functionality	Copy memory data
Parameters	void *dst Destination memory address const void *src Source memory address uint32_t size Copy length, in Bytes
Considerations	None
Return Value	Memory address, identical to the input destination memory address

Table 5-8 host_os_memcmp

Interface	Description
Function	int host_os_memcmp(const void *addr1, const void *addr2, uint32_t size);
Functionality	Compare memory data
Parameters	const void *addr1 Memory address 1 to compare const void *addr2 Memory address 2 to compare uint32_t size Comparison length, in Bytes
Considerations	None
Return Value	>0 Data at memory address 1 is greater =0

	Data at the two memory addresses is equal
	<0
	Data at memory address 2 is greater

5.1.2.1.4 Thread

Table 5-9 host_os_thread_create

Interface	Description
Function	host_os_thread_handle_t host_os_thread_create(const char *pname, uint32_t stack_size, host_os_thread_entry_t entry, void *arg, int32_t prio);
Functionality	Create a thread
Parameters	const char *pname Thread name uint32_t stack_size Thread stack size host_os_thread_entry_t entry Thread entry-function address, of type void (*)(void *) void *arg Parameter of the thread entry function int32_t prio Thread priority
Considerations	None
Return Value	host_os_thread_handle_t Thread handle; returns NULL on failure and another value on success

Table 5-10 host_os_thread_delete

Interface	Description
Function	int host_os_thread_delete(host_os_thread_handle_t thread_hdl);
Functionality	Delete a thread
Parameters	host_os_thread_handle_t Thread handle
Considerations	None
Return Value	HOST_ERRCODE_SUCCESS indicates success; any other value indicates failure.

Table 5-11 host_os_thread_is_valid

Interface	Description
Function	bool host_os_thread_is_valid(host_os_thread_handle_t thread_hdl);
Functionality	Determine whether a thread is valid by using its thread handle.
Parameters	host_os_thread_handle_t

	Thread handle
Notes	None
Return Value	true indicates valid; false indicates invalid.

5.1.2.1.5 Semaphore

Table 5-12 host_os_sem_create

Interface	Description
Function	host_os_sem_t host_os_sem_create(uint32_t initial_count, uint32_t limit);
Functionality	Create a semaphore.
Parameters	uint32_t initial_count Initial value uint32_t limit Maximum value
Notes	None
Return Value	host_os_sem_t Semaphore handle. NULL indicates failure; any other value indicates success.

Table 5-13 host_os_sem_delete

Interface	Description
Function	int host_os_sem_delete(host_os_sem_t psem);
Functionality	Delete a semaphore.
Parameters	host_os_sem_t psem Semaphore handle
Notes	None
Return Value	HOST_ERRCODE_SUCCESS indicates success; any other value indicates failure.

Table 5-14 host_os_sem_is_valid

Interface	Description
Function	int host_os_sem_is_valid(host_os_sem_t psem);
Functionality	Determine whether a semaphore is valid.
Parameters	host_os_sem_t psem Semaphore handle
Notes	None
Return Value	0 indicates invalid; any other value indicates valid. A return value of 1 is recommended to indicate valid.

Table 5-15 host_os_sem_take

Interface	Description
Function	int host_os_sem_take(host_os_sem_t psem, host_os_timeout_t timeout);
Functionality	Take a semaphore.
Parameters	host_os_sem_t psem Semaphore handle

	host_os_timeout_t timeout Timeout
Notes	None
Return Value	HOST_ERRCODE_SUCCESS indicates success; any other value indicates failure.

Table 5-16 host_os_sem_give

Interface	Description
Function	int host_os_sem_give(host_os_sem_t psem);
Functionality	Give a semaphore.
Parameters	host_os_sem_t psem Semaphore handle
Notes	None
Return Value	HOST_ERRCODE_SUCCESS indicates success; any other value indicates failure.

5.1.2.1.6 Delay Functions

Table 5-17 host_os_delayms

Interface	Description
Function	int host_os_delayms(uint32_t ms);
Functionality	Provide a millisecond-level delay.
Parameters	uint32_t ms Delay duration, in milliseconds.
Notes	A non-busy-wait implementation is required to allow system scheduling.
Return Value	HOST_ERRCODE_SUCCESS indicates success; any other value indicates failure.

Table 5-18 host_os_delayus

Interface	Description
Function	int host_os_delayus(uint32_t us);
Functionality	Provide a microsecond-level delay.
Parameters	uint32_t us Delay duration, in microseconds.
Notes	None
Return Value	HOST_ERRCODE_SUCCESS indicates success; any other value indicates failure.

5.1.2.1.7 Sleep Control

Table 5-19 host_os_pm_lock

Interface	Description
Function	void host_os_pm_lock(void);
Functionality	Lock/disable/prohibit sleep.
Parameters	None
Notes	Lock sleep directly.

Return Value	None
--------------	------

Table 5-20 host_os_pm_unlock

Interface	Description
Function	void host_os_pm_unlock(void);
Functionality	Unlock/enable/allow sleep.
Parameters	None
Notes	Unlock sleep directly.
Return Value	None

5.1.2.2 I/O Interface

The I/O interface is a GPIO control interface. In the Host Driver, it must control the reset and upgrade pins, read the level of the Notify I/O, and respond to edge or level interrupts from the Notify I/O (level interrupts are not currently supported).

When the I/O interface controls the RST I/O to reset the Device, a high output level enables Device operation by default, and a low output level disables Device operation. If the VCC reset scheme is used, ensure that a high level powers on the Device and a low level powers it off.

For an active-low power-on and active-high power-off scheme, when adapting the host_io_write interface, map a high RST I/O output to a low output and a low RST I/O output to a high output.

Table 5-21 host_io_mode_set

Interface	Description
Function	int host_io_mode_set(uint32_t io, host_io_mode_t mode, host_io_pull_t pull, bool is_wkio);
Functionality	Set the GPIO mode. Input, output, and interrupt modes, as well as pull-up, pull-down, and floating configurations, must be supported.
Parameters	uint32_t io GPIO number host_io_mode_t mode GPIO mode host_io_pull_t pull GPIO pull configuration bool is_wkio Whether WKIO is being configured true: Yes false: No
Notes	WKIO (I/O wake-up) must be supported if the sleep function is required.
Return Value	HOST_ERRCODE_SUCCESS indicates success; any other value indicates failure.

Table 5-22 host_io_irq_cfg

Interface	Description
Function	int host_io_irq_cfg(uint32_t io, host_io_irq_trig_t trig, host_io_irq_callback_t pcallback, void *arg, bool is_wkio);
Functionality	Set the GPIO interrupt trigger mode and callback function.

Parameters	uint32_t io GPIO number host_io_irq_trig_t trig Interrupt trigger mode <pre>typedef enum { HOST_IO_IRQ_EDGE_RISING, HOST_IO_IRQ_EDGE_FALLING, HOST_IO_IRQ_EDGE_BOTH, HOST_IO_IRQ_LEVEL_HIGH, HOST_IO_IRQ_LEVEL_LOW, } host_io_irq_trig_t;</pre> host_io_irq_callback_t pcallback Interrupt callback function of type void (*)(void *). void *arg Function argument bool is_wkio Whether WKIO is being configured true: Yes false: No
Notes	WKIO (I/O wake-up) must be supported if the sleep function is required.
Return Value	HOST_ERRCODE_SUCCESS indicates success; any other value indicates failure.

Table 5-23 host_io_irq_enable

Interface	Description
Function	int host_io_irq_enable(uint32_t io, bool is_wkio);
Functionality	Enable the GPIO interrupt.
Parameters	uint32_t io GPIO number to enable bool is_wkio Whether WKIO is being configured true: Yes false: No
Notes	WKIO (I/O wake-up) must be supported if the sleep function is required.
Return Value	HOST_ERRCODE_SUCCESS indicates success; any other value indicates failure.

Table 5-24 host_io_irq_disable

Interface	Description
Function	int host_io_irq_disable(uint32_t io, bool is_wkio);
Functionality	Disable the GPIO interrupt.

Parameters	uint32_t io GPIO number to enable bool is_wkio Whether WKIO is being configured true: Yes false: No
Notes	WKIO (I/O wake-up) must be supported if the sleep function is required.
Return Value	HOST_ERRCODE_SUCCESS indicates success; any other value indicates failure.

Table 5-25 host_io_write

Interface	Description
Function	void host_io_write(uint32_t io, host_io_value_t value, bool is_wkio);
Functionality	Set the GPIO output level.
Parameters	uint32_t io GPIO number host_io_value_t value Logic level typedef enum { HOST_IO_VALUE_LOW, HOST_IO_VALUE_HIGH, } host_io_value_t; bool is_wkio Whether WKIO is being configured true: Yes false: No
Notes	None
Return Value	None

Table 5-26 host_io_read

Interface	Description
Function	host_io_value_t host_io_read(uint32_t io, bool is_wkio);
Functionality	Read the GPIO input level.
Parameters	uint32_t io GPIO number
Notes	WKIO (I/O wake-up) must be supported if the sleep function is required.
Return Value	Logic level typedef enum { HOST_IO_VALUE_LOW, HOST_IO_VALUE_HIGH, } host_io_value_t;

	bool is_wkio Whether WKIO is being configured true: Yes false: No
--	--

5.1.2.3 Storage-Media Interface

The storage-media interface is used by the Host to obtain the Device firmware image.

Table 5-27 host_store_open

Interface	Description
Function	int host_store_open(void **hdl, void *path, uint32_t *len);
Functionality	Open a firmware image.
Parameters	void **hdl Firmware image pointer; output parameter. void *path Firmware image path uint32_t *len Pointer to the firmware image size; output parameter, in bytes.
Notes	If the interface returns HOST_ERRCODE_SUCCESS, values must be assigned to hdl and len, and hdl must not be NULL.
Return Value	HOST_ERRCODE_SUCCESS indicates success; any other value indicates failure.

Table 5-28 host_store_close

Interface	Description
Function	void host_store_close(void *hdl);
Functionality	Close a firmware image.
Parameters	void *hdl Firmware image
Notes	None
Return Value	HOST_ERRCODE_SUCCESS indicates success; any other value indicates failure.

Table 5-29 host_store_read

Interface	Description
Function	int host_store_read(void *hdl, uint32_t offset, void *pbuff, uint32_t size);
Functionality	Read firmware image data.
Parameters	void *hdl Firmware image uint32_t offset Offset, in bytes. void *pbuff Buffer for receiving data

	uint32_t size Data size, in bytes.
Notes	None
Return Value	HOST_ERRCODE_SUCCESS indicates success; any other value indicates failure.

5.1.2.4 Logging Interface

The logging interface must implement formatted printing. It is recommended to define the macro function `host_log_print(fmt, ...)` in a header file and define the `CFG_HOST_PORT_LOG_FILE` macro in `user_config.h`. Set this macro to the path of the file that defines/declares `host_log_print`, relative to `porting/include/hal_log.h`.

For example, if `host_log_print` is defined in `porting/psic/port_log.h`, set `CFG_HOST_PORT_LOG_FILE` to `"../psic/port_log.h"`.

Table 5-30 host_log_print

Interface	Description
Function	<code>int host_log_print(const char *, ...);</code>
Functionality	Print a string.
Parameters	const char * String to be formatted ... Formatting arguments
Notes	None
Return Value	The return value is not used.

5.1.2.5 COM Interface

5.1.2.5.1 Pre-Porting Considerations

The COM interface is the data-interaction interface between the Host and Device, such as 1-/2-/4-line SPI, UART, or I²C. The interface requirements differ for different data-reporting types and LLC notification types. These two types are described below.

The data-reporting types are active reporting, passive reporting, and active reporting (delayed). Active reporting (delayed) is not currently supported.

An active-reporting Device reports data directly to the Host when data is available. Because of bus limitations, only UART supports active reporting among the three buses described above. **Currently, an active-reporting Device exclusively occupies one bus; that is, only one Device can be attached to the bus.**

For passive reporting, the Device asserts the Notify I/O (when active high) when data is available. While the Notify I/O is asserted, the Host sends a POLL command to the Device. After receiving POLL, the Device deasserts the Notify I/O and prepares the data. When the data is ready, the Device asserts the Notify I/O again, and the Host reads the data. After the read is complete, the Device deasserts the Notify I/O. For an example waveform, see [3.2.3.1 HIF SPI Data Reporting](#).

The LLC notification types are edge notification (EDGE), COM interrupt/data-ready notification (COM_ISR), and COM data-arrival notification (COM_IRQ_THREAD).

Firmware download supports only 1-line SPI, I²C, and UART. If 2-/4-line SPI is used as HIF COM, initialize it as SPI/DSPI/QSPI in the initialization interface according to arg: initialize as SPI when arg is LLC_BUS_PARAM_SPI, as DSPI when arg is LLC_BUS_PARAM_DSPI, and as QSPI when arg is LLC_BUS_PARAM_QSPI.

5.1.2.5.2 Selecting the LLC Notification Type

For passive reporting, select EDGE directly.

For active reporting, data can be read in interrupt context (COM_ISR) or thread context (COM_IRQ_THREAD).

For COM_ISR and COM_IRQ_THREAD, COM_ISR must support setting an interrupt callback function. If this cannot be supported, only COM_IRQ_THREAD can be configured. For either type, the interrupt callback registered by the upper layer must be called when data is available. For COM_ISR, the function registered by the upper layer can be registered directly as the interrupt service function. For COM_IRQ_THREAD, the function registered by the upper layer is called when data is available to read.

For COM_ISR and COM_IRQ_THREAD, the threshold that generates an interrupt can be configured through interrupt_ctrl to provide more contiguous CPU execution time.

5.1.2.5.3 Receiving Data with the EDGE Notification Type

The following diagram shows reported-data reception using the EDGE notification type. In the diagram, TL Layer Thread is the transport-layer data-transfer thread, LLC Layer is the link layer, and HAL Layer is the adaptation layer. To identify the communication-port control interface used at each step, the interface name, such as com_ops.write, is shown in parentheses.

During data reception with EDGE, the com_ops.write and com_ops.read interfaces are called.

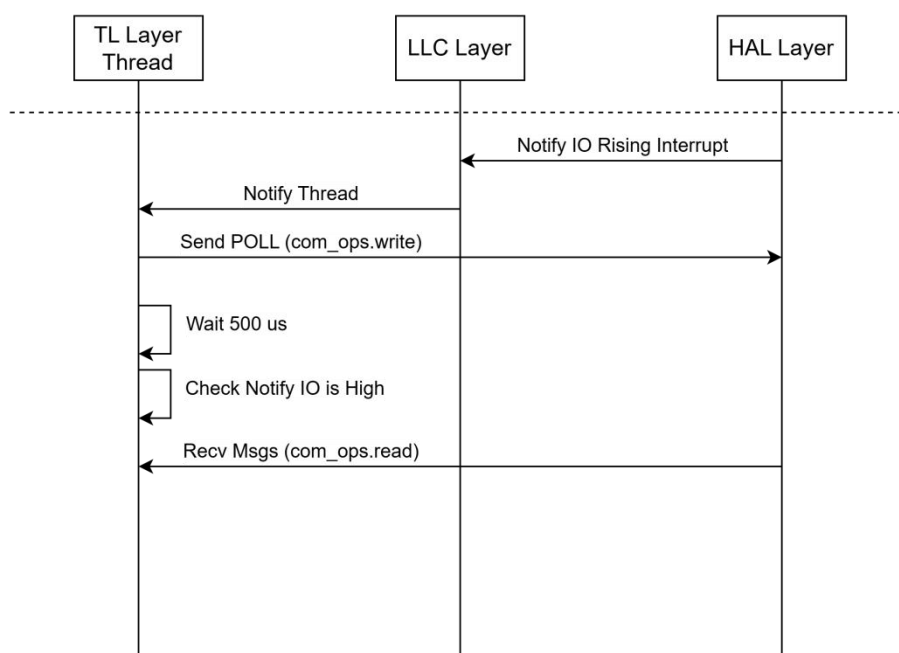


Figure 5-1 EDGE Notification-Type Reported-Data Reception Flow

5.1.2.5.4 Receiving Data with the COM_ISR Notification Type

The COM_ISR notification type receives data in interrupt context. During reception, the com_ops.read and com_ops.interrupt interfaces are called.

When the COM_ISR notification type receives reported data, the link-layer Cache may be either full or not full when the thread calls LLC_Rcv, resulting in slightly different reception flows. The case in which the Cache is not full when the thread calls LLC_Rcv is described first.

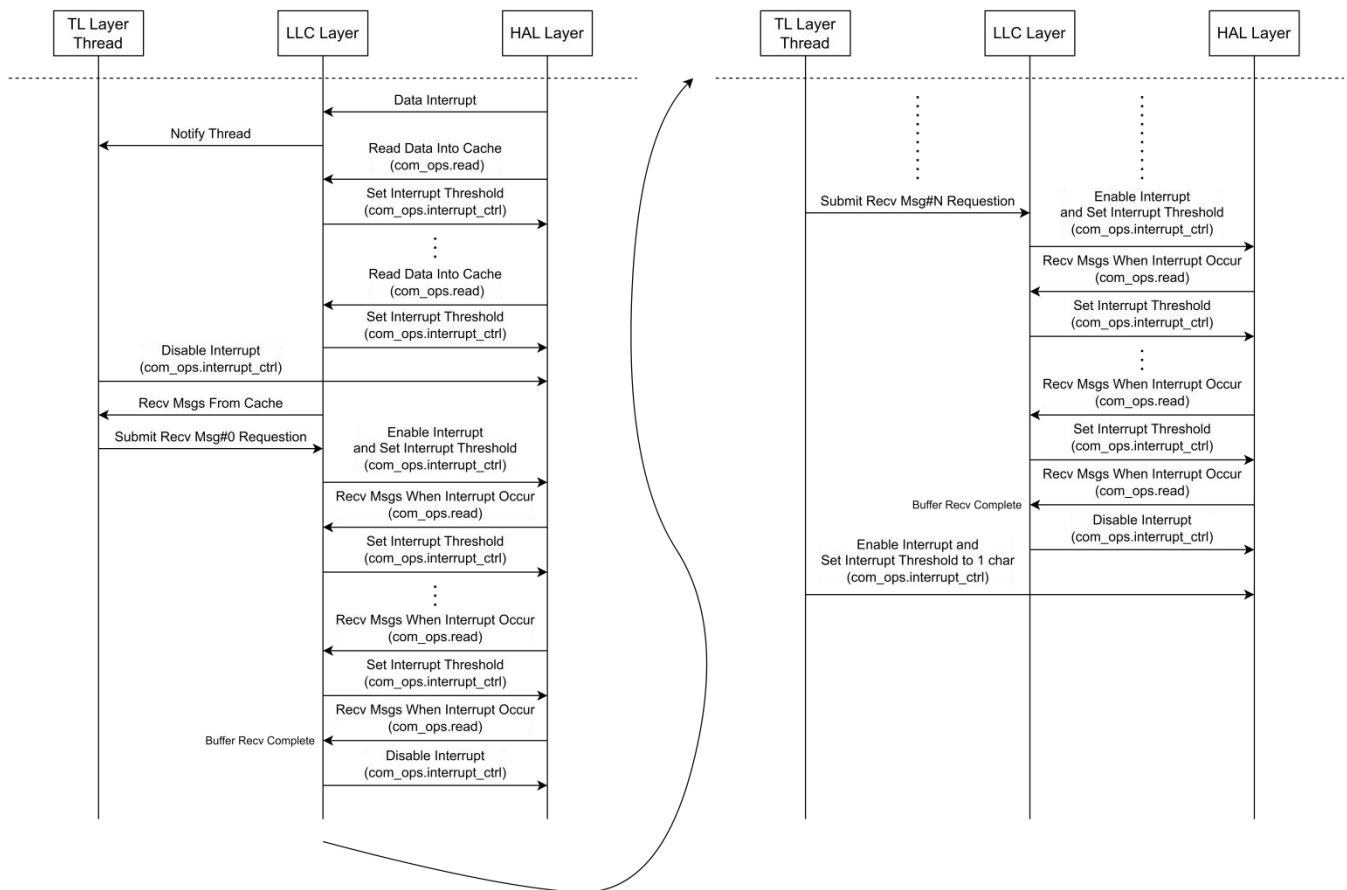


Figure 5-2 COM_ISR Notification-Type Reported-Data Reception Flow (Thread Calls LLC_Recv Before the Cache Is Full)

The following describes the case in which the Cache is already full when the thread calls LLC_Recv. If the Cache becomes full before the thread calls LLC_Recv, interrupts are disabled while waiting for the thread to call LLC_Recv to receive the data.

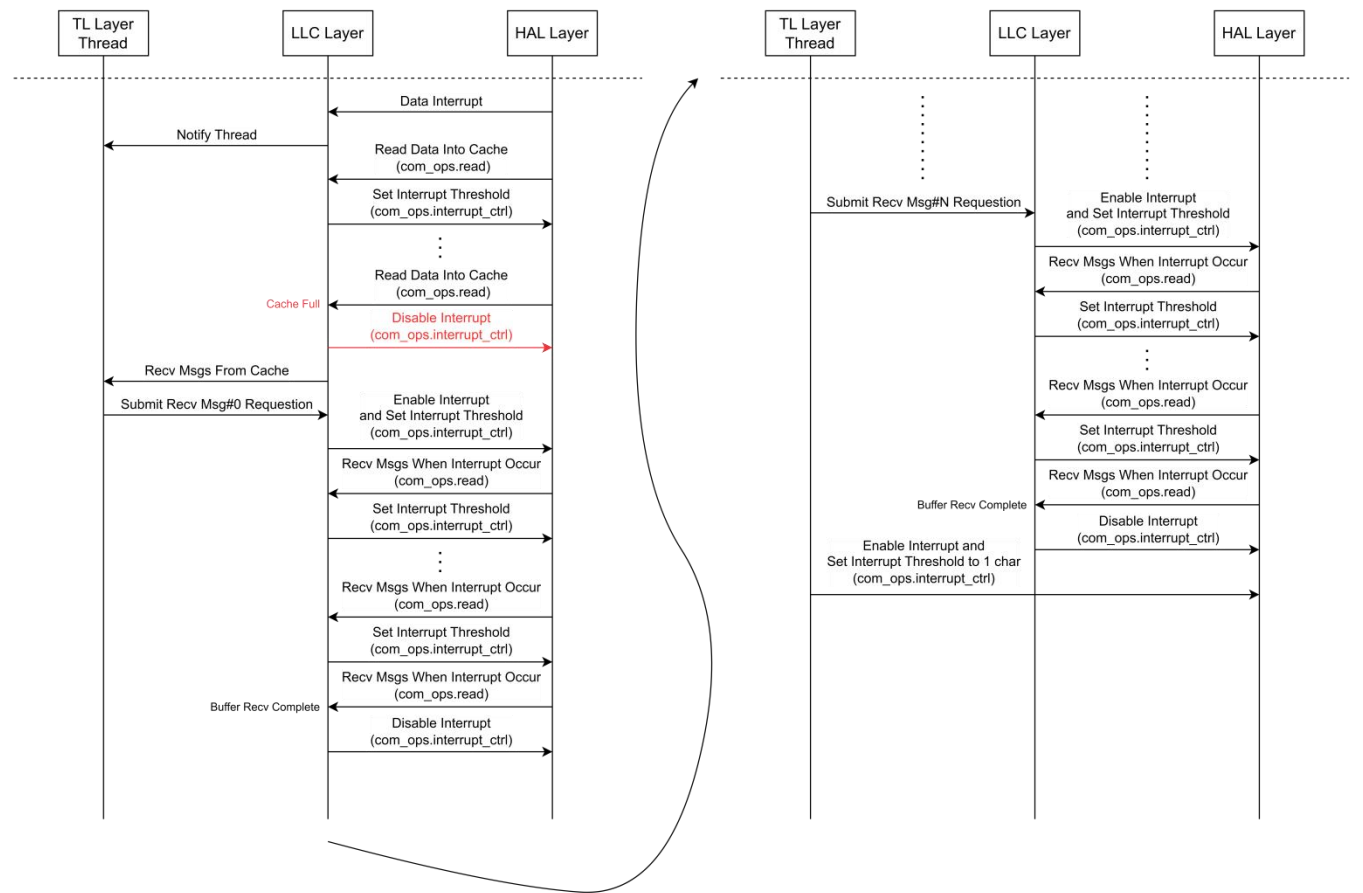


Figure 5-3 COM_ISR Notification-Type Reported-Data Reception Flow (Thread Calls LLC_Rcv After the Cache Is Full)

5.1.2.5.5 Receiving Data with the COM_IRQ_THREAD Notification Type

This section describes only the case in which the com_ops.interrupt_ctrl interface is not registered for the COM_IRQ_THREAD notification type.

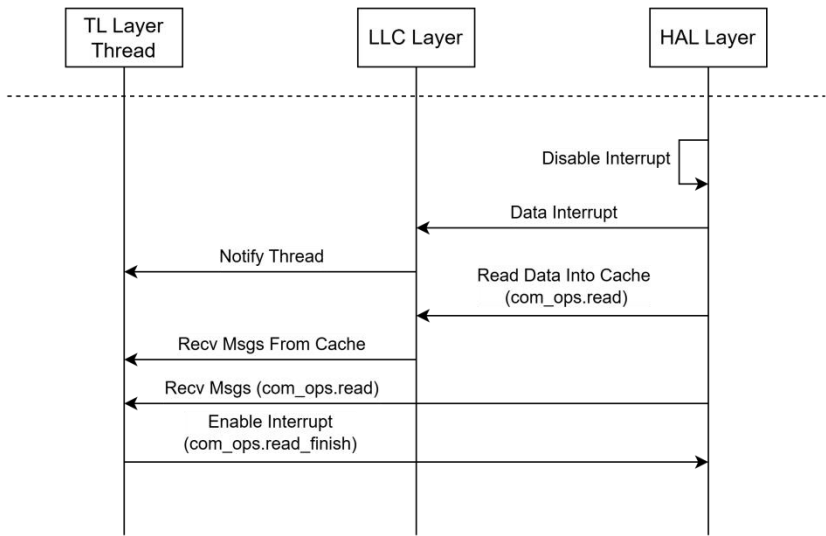


Figure 5-4 COM_IRQ_THREAD Notification-Type Reported-Data Reception Flow

5.1.2.5.6 Interface Adaptation Requirements

Table 5-31 Interface Requirements for Different LLC Notification Types

COM Interface \ LLC Notification Type	EDGE	COM_ISR	COM_IRQ_THREAD
	Notify I/O edge	HIF COM data interrupt (interrupt	HIF COM data ready (thread

	notification	read)	read)
init	✓	✓	✓
deinit	✓	✓	✓
open	✓	✓	✓
close	✓	✓	✓
write	✓	✓	✓
read	✓	✓	✓
read_finish	×	×	✓ Enable interrupts
interrupt_ctrl	×	✓ Enable/disable COM interrupts and set the interrupt threshold	Optional. Registering this interface enables full-cache data buffering.
interrupt_handle_regist	×	✓ When an interrupt is triggered, directly call the callback function registered by the LLC.	✓ When an interrupt is triggered, first disable the interrupt and then call the callback function registered by the LLC.
data_callback_regist	×	×	×
Special Requirements	None	None	Packet processing must take a short time.

A bus using the EDGE LLC notification type only needs to provide the following standard interfaces.

Table 5-32 host_com_init_t

Interface	Description
Function	typedef void * (*host_com_init_t)(uint8_t id, uint32_t speed, void *arg);
Functionality	Called when the bus is opened to initialize device-independent resources.
Parameters	uint8_t id Bus ID uint32_t speed Bus speed; for UART, this value indicates the baud rate. void *arg Bus parameter (unused)
Notes	None
Return Value	Vendor-defined handle

Table 5-33 host_com_deinit_t

Interface	Description
Function	typedef int (*host_com_deinit_t)(void *bus);

Functionality	Called when the bus is closed to deinitialize device-independent resources.
Parameters	void *bus Vendor-defined handle
Notes	None
Return Value	HOST_ERRCODE_SUCCESS indicates success; any other value indicates failure.

Table 5-34 host_com_open_t

Interface	Description
Function	typedef int (*host_com_open_t)(void *bus, uint32_t param);
Functionality	Called when the Device is opened to initialize device-specific resources.
Parameters	void *bus Vendor-defined handle uint32_t param Device parameter: the CS pin number for SPI, the attached bus ID for UART, or the Device address for I ² C. For example: If the CS pin number of an SPI Device is 0, param is 0. If a UART Device is attached to bus ID 1, param is 1. If the I ² C Device address is 0x3A, param is 0x3A.
Notes	None
Return Value	HOST_ERRCODE_SUCCESS indicates success; any other value indicates failure.

Table 5-35 host_com_close_t

Interface	Description
Function	typedef int (*host_com_close_t)(void *bus, uint32_t param);
Functionality	Called when the Device is closed to deinitialize device-specific resources.
Parameters	void *bus Vendor-defined handle uint32_t param Device parameter: the CS pin number for SPI, the attached bus ID for UART, or the Device address for I ² C.
Notes	None
Return Value	HOST_ERRCODE_SUCCESS indicates success; any other value indicates failure.

Table 5-36 host_com_write_t

Interface	Description
Function	typedef int (*host_com_write_t)(void *bus, uint32_t param, void *pbuff, uint32_t size);
Functionality	Write data to the specified Device.
Parameters	void *bus Vendor-defined handle

	uint32_t param Device parameter: the CS pin number for SPI, the attached bus ID for UART, or the Device address for I²C. void *pbuff Address of the buffer to be written uint32_t size Buffer size
Notes	None
Return Value	>0 indicates the amount of data written; any other value indicates failure.

Table 5-37 host_com_read_t

Interface	Description
Function	typedef int (*host_com_read_t)(void *bus, uint32_t param, void *pbuff, uint32_t size);
Functionality	Read data from the specified Device.
Parameters	void *bus Vendor-defined handle uint32_t param Device parameter: the CS pin number for SPI, the attached bus ID for UART, or the Device address for I²C. void *pbuff Address of the buffer for receiving data uint32_t size Buffer size
Notes	None
Return Value	>0 indicates the amount of data read; any other value indicates failure.

If the LLC notification type is COM_ISR, the interfaces above must be registered, and the interrupt_ctrl and interrupt_handle_regist interfaces must also be provided.

Table 5-38 host_com_interrupt_ctrl_t

Interface	Description
Function	typedef int (*host_com_interrupt_ctrl_t)(void *bus, bool en, uint32_t size);
Functionality	Control COM interrupts.
Parameters	void *bus Vendor-defined handle bool en true: Enable interrupts false: Disable interrupts

	uint32_t size Amount of data to receive, in bytes. This parameter can be used to dynamically set the FIFO threshold that triggers a COM interrupt.
Notes	None
Return Value	HOST_ERRCODE_SUCCESS indicates success; any other value indicates failure.

Table 5-39 host_com_interrupt_handle_regist_t

Interface	Description
Function	typedef int (*host_com_interrupt_handle_regist_t)(void *bus, host_com_interrupt_handler_t handler, void *arg);
Functionality	Notify the upper layer that data is available for reception.
Parameters	void *bus Vendor-defined handle host_com_interrupt_handler_t handler Callback function of type void (*)(void *). void *arg Function argument
Notes	None
Return Value	HOST_ERRCODE_SUCCESS indicates success; any other value indicates failure.

If the LLC notification type is COM_IRQ_THREAD, register the standard interfaces and additionally register read_finish and interrupt_handle_regist. **When an interrupt is triggered, first disable the interrupt and then call the registered interrupt handler.**

The COM_IRQ_THREAD notification type also supports registering the interrupt_ctrl interface. After this interface is registered, all space in LLC_HANDLE_S::rx_cache is used to buffer data before the thread executes the data-reception function.

Table 5-40 host_com_read_finish_t

Interface	Description
Function	typedef int (*host_com_read_finish_t)(void *bus, uint32_t param);
Functionality	Called by the upper layer when reception is complete to notify COM to wait for the next data arrival.
Parameters	void *bus Vendor-defined handle uint32_t param Device parameter: the CS pin number for SPI, the attached bus ID for UART, or the Device address for I ² C.
Notes	Enable COM interrupts.
Return Value	HOST_ERRCODE_SUCCESS indicates success; any other value indicates failure.

5.1.2.5.7 Definition of the Function for Obtaining the Bus-Control Interface Set ops

During vendor porting, define the function host_com_ops_t *porting_get_com_ops(host_bus_type_t type, uint8_t id) to allow the LLC layer to obtain the bus-operation interfaces. type is the bus type and id is the bus ID. The following definition can be used as a reference.

```

extern host_com_ops_t *porting_port_spi_get_com_ops(uint8_t id);
extern host_com_ops_t *porting_port_i2c_get_com_ops(uint8_t id);
extern host_com_ops_t *porting_port_uart_get_com_ops(uint8_t id);

/**
 * @brief provided to LLC
 */
host_com_ops_t *porting_get_com_ops(host_bus_type_t type, uint8_t id)
{
    switch (type) {
        case HOST_BUS_TYPE_SPI:
            return porting_port_spi_get_com_ops(id);
        case HOST_BUS_TYPE_I2C:
            return porting_port_i2c_get_com_ops(id);
        case HOST_BUS_TYPE_UART:
            return porting_port_uart_get_com_ops(id);
        default:
            return NULL;
    }
}

```

5.1.2.5.8 Example of Adding an SPI Bus

An SPI bus supports the EDGE notification type. The following is sample pseudocode for an SPI bus driver.

```

#include "../host_port.h"
#include "../include/hal_com.h"

#include "hal_spi.h"

static void * porting_port_spi_init(uint8_t id, uint32_t speed, void *arg);
static int porting_port_spi_deinit(void *bus);
static int porting_port_spi_open(void *bus, uint32_t cs_pin);
static int porting_port_spi_close(void *bus, uint32_t cs_pin);
static int porting_port_spi_write(void *bus, uint32_t cs_pin, void *pbuff, uint32_t size);
static int porting_port_spi_read(void *bus, uint32_t cs_pin, void *pbuff, uint32_t size);

static const host_com_ops_t spi_ops = {
    .init                = porting_port_spi_init,
    .deinit              = porting_port_spi_deinit,
    .open                = porting_port_spi_open,
    .close               = porting_port_spi_close,
    .write               = porting_port_spi_write,
    .read                = porting_port_spi_read,
    .read_finish         = NULL,
    .interrupt_handle_regist = NULL,
    .data_callback_regist  = NULL,
};

```

```
static void * porting_port_spi_init(uint8_t id, uint32_t speed, void *arg)
{}

static int porting_port_spi_deinit(void *bus)
{}

static int porting_port_spi_open(void *bus, uint32_t cs_pin)
{}

static int porting_port_spi_close(void *bus, uint32_t cs_pin)
{}

static int porting_port_spi_write(void *bus, uint32_t cs_pin, void *pbuff, uint32_t size)
{}

static int porting_port_spi_read(void *bus, uint32_t cs_pin, void *pbuff, uint32_t size)
{}

host_com_ops_t *porting_port_spi_get_com_ops(uint8_t id)
{
    if (id == 0) {
        return (host_com_ops_t *)&spi_ops;
    } else {
        return NULL;
    }
}
```

The example provides code for adding the SPI0 bus. In general, id directly represents the bus ID; that is, SPI0 has ID 0 and SPI1 has ID 1. If other requirements exist, a different mapping from id to bus can also be defined.

5.2 Host Driver Usage Flow

When using the Host Driver, only the APIs provided by the APP layer need to be called to implement data reporting, command transmission, and firmware download. The overall usage flow is described first, followed by the flow for each individual function.

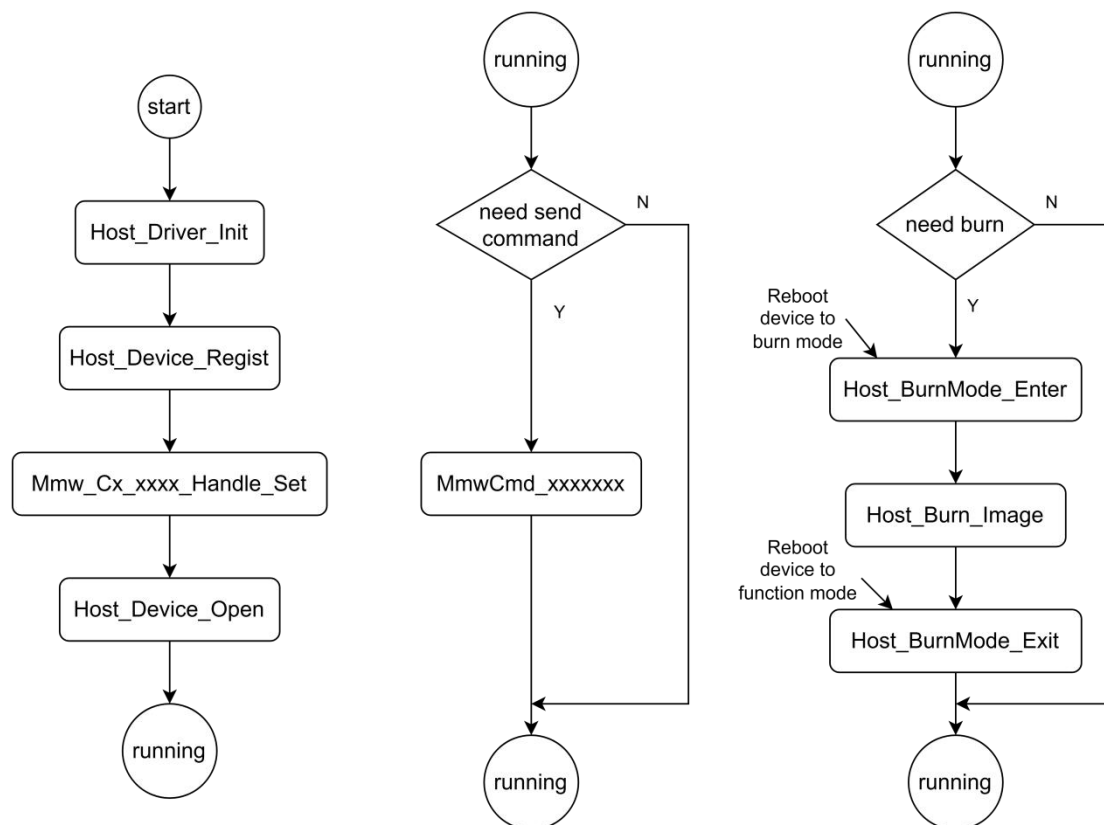


Figure 5-5 Overall Host Driver Usage Flow

5.2.1 Driver and Device Initialization

5.2.1.1 Initialization Flow

The first step is to initialize the Host Driver and register a Device.

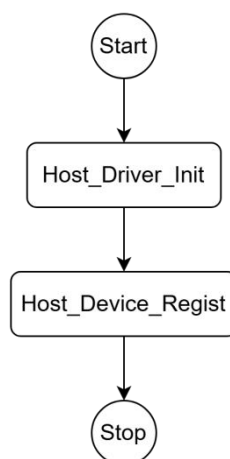


Figure 5-6 Host Driver and Device Initialization Flow

5.2.1.2 Device Parameter Configuration

Before using the Host Driver, understand the concepts of the data-reporting type and LLC notification type.

The data-reporting type specifies how the radar reports data. The three types are active reporting, passive reporting, and active reporting (delayed). Active reporting (delayed) is not currently supported.

A UART Device generally uses active reporting, in which the radar sends data directly to the Host when data is available. UART can also use passive reporting, but this is uncommon; only UART active reporting is described here.

For SPI or I²C, the Device can use only passive reporting because of the bus characteristics; that is, the Host initiates the data-interaction process.

The LLC notification type specifies how the PHY layer (HAL layer) notifies the LLC layer of data availability or the Notify I/O state.

For active reporting, data-arrival notification (COM_IRQ_THREAD) or interrupt/data-ready notification (COM_ISR) can be selected.

COM_IRQ_THREAD: When a data interrupt occurs, the PHY layer disables the interrupt and invokes the callback registered by the Host Driver. After the Host Driver completes the data interaction (in the TL-layer thread), it calls the read_finish interface of the PHY layer to enable the interrupt;

COM_ISR: When a data interrupt occurs, the PHY layer invokes the callback registered by the Host Driver. During data interaction, the Host Driver calls the interrupt_ctrl interface of the PHY layer to dynamically adjust the interrupt threshold.

For passive reporting, the Notify I/O state is reported to the layer above LLC. The notification type is edge notification (EDGE). This section assumes that the Notify I/O is active high, so EDGE notifies the rising edge.

5.2.1.2.1 Parameter Configuration for UART Active Reporting with COM_IRQ_THREAD Notification

```
DevHw_t devHw = {
    .bus_speed = 921600, // Bus speed
    .bus_type = LLC_BUS_TYPE_UART, // Bus type
    .bus_id = 1, // Bus ID; UART1 is used, so the ID is 1
    .upd_io = 4, // Upgrade I/O
    .rst_io = 5, // Reset I/O or VCC I/O
    .notify_io = 6, // Notify I/O
    .notify_type = LLC_NOTIFY_TYPE_COM_IRQ_THREAD, // COM_IRQ_THREAD notification
    .upload_type = LLC_UPLOAD_TYPE_ACTIVE, // Active reporting
    .param = 1, // Device parameter; for UART, this is the attached bus ID
};
```

The following initialization method is recommended:

```
DevHw_t devHw;
DevHw_Set_DefaultParam(&devHw, LLC_BUS_TYPE_UART, 0, 1, 921600, 4, 5, 6, 1);
// For the COM_ISR notification type, also add the following statement
//devHw.notify_type = LLC_NOTIFY_TYPE_COM_ISR;
```

5.2.1.2.2 Parameter Configuration for SPI Passive Reporting with EDGE Notification

```
DevHw_t devHw = {
    .bus_speed = 8000000, // Bus speed
    .bus_type = LLC_BUS_TYPE_SPI, // Bus type
    .bus_id = 0, // Bus ID; SPI0 is used, so the ID is 0
    .bus_param = LLC_BUS_PARAM_SPI, // 1-line SPI
    .upd_io = 4, // Upgrade I/O
    .rst_io = 5, // Reset I/O or VCC I/O
    .notify_io = 6, // Notify I/O
    .notify_type = LLC_NOTIFY_TYPE_EDGE, // EDGE notification
    .upload_type = LLC_UPLOAD_TYPE_PASSIVE, // Passive reporting
    .param = 0, // Device parameter; for SPI, this is the CS I/O
};
```

The following initialization method is recommended:

```
DevHw_t devHw;
DevHw_Set_DefaultParam(&devHw, LLC_BUS_TYPE_SPI, LLC_BUS_PARAM_SPI, 0, 8000000, 4, 5, 6, 0);
```

5.2.1.3 API Usage

Before initializing a Device, initialize the Host Driver by calling the following API.

Table 5-41 Host_Driver_Init

API	Description
Function	int Host_Driver_Init(void);
Functionality	Initialize the Host Driver.
Parameters	None
Notes	None
Return Value	HOST_ERRCODE_SUCCESS indicates success; any other value indicates failure.

To deinitialize the Host Driver, call the following API after closing all Devices.

Table 5-42 Host_Driver_Deinit

API	Description
Function	void Host_Driver_Deinit(void);
Functionality	Deinitialize the Host Driver.
Parameters	None
Notes	None
Return Value	None

After initializing the Host Driver, use the following APIs to register, unregister, open, and close Devices.

Table 5-43 Host_Device_Regist

API	Description
Function	int Host_Device_Regist(DEV_HANDLE *pDevHdl_out, DevHw_t *hw_cfg, HifCfg_t *hif_cfg);
Functionality	Register a Device.
Parameters	DEV_HANDLE *pDevHdl_out Pointer to the Device handle; output parameter. DevHw_t *hw_cfg Physical configuration parameters of the Device, including bus information, bus event policy, Device information, and other settings. HifCfg_t *hif_cfg HIF control parameters of the Device
Notes	None
Return Value	HOST_ERRCODE_SUCCESS indicates success; any other value indicates failure.

For the DevHw_t configuration, refer to Device Parameter Configuration above.

HifCfg_t can be initialized to all zeros so that the Host Driver automatically converts invalid settings to their defaults, or it can be configured manually. The configuration members are described below.

```
typedef struct {
```

```

uint8_t poll_enable; // Enable POLL commands: 1 = enabled, 0 = disabled
uint8_t app_retry_enable; // Enable application-layer retransmission: 1 = enabled, 0 = disabled
uint8_t tl_retry_enable; // Enable transport-layer retransmission: 1 = enabled, 0 = disabled
uint8_t fragment_enable; // Enable fragmentation: 1 = enabled, 0 = disabled
uint8_t fragment_retry_enable; // Enable fragment retransmission: 1 = enabled, 0 = disabled

uint16_t burst_num; // Number of bursts per data reception
uint16_t cmd_buf_len; // Buffer size used to transmit commands
uint16_t poll_buf_len; // Buffer size used to transmit POLL
} HifCfg_t;

```

Table 5-44 Host_Device_Unregist

API	Description
Function	void Host_Device_Unregist(DEV_HANDLE DevHdl);
Functionality	Unregister a Device.
Parameters	DEV_HANDLE DevHdl Device handle
Notes	None
Return Value	None

Table 5-45 Host_Device_Open

API	Description
Function	int Host_Device_Open(DEV_HANDLE DevHdl);
Functionality	Open a Device.
Parameters	DEV_HANDLE DevHdl Device handle
Notes	None
Return Value	HOST_ERRCODE_SUCCESS indicates success; any other value indicates failure.

Table 5-46 Host_Device_Close

API	Description
Function	int Host_Device_Close(DEV_HANDLE DevHdl);
Functionality	Close a Device.
Parameters	DEV_HANDLE DevHdl Device handle
Notes	None
Return Value	HOST_ERRCODE_SUCCESS indicates success; any other value indicates failure.

5.2.2 Data Reporting

The Host Driver provides two methods for receiving data reported by the Device. One reassembles the data before invoking the user callback, as for C1, C2, and C3 data. The other invokes the user callback without processing the data, as for C6, C7, C8, CA, and CB data.

5.2.2.1 C1/C2/C3 Data Usage Flow

For C1/C2/C3 data (for the data definitions, see [3.1.3.2 Reported Data Structure](#)), the Device reports fragmented data. The Host Driver APP layer provides data-reassembly functions through `Mmw_C1_CubeData_Handle_Set`, `Mmw_C2_FFTData_Handle_Set`, and `Mmw_C3_Objects_Handle_Set`. The callbacks registered through these three APIs are invoked after the corresponding data has been reassembled.

The callback function setup flow is as follows:

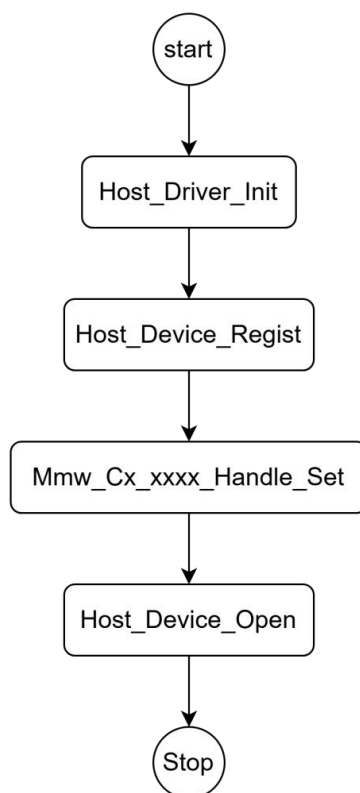


Figure 5-7 C1/C2/C3 Reported-Data Callback Setup Flow

As shown above, after this flow is completed, the Host Driver passes received C1/C2/C3 data to the user through the configured callback processing function. **Note that the buffer storing the data is allocated by the Host Driver APP layer and released after the user processing function returns. Therefore, if the data must remain accessible after the callback returns, copy the data to a backup buffer.**

The user callback function is defined as follows:

```
typedef void (*Report_CB)(uint32_t msg_id, uint8_t *data, uint32_t data_len, void *cb_arg);
```

5.2.2.1.1 Data structure passed to the C1 callback

```
typedef struct {
    uint16_t imag;
    uint16_t real;
} Frame_FFT_t;

typedef struct {
    uint32_t frameIdx;
    uint32_t frameLen; /* unit=DWORD */
    uint32_t dataOffset; /* unit=DWORD */
    Frame_FFT_t data[0];
} FrameData_C1_t;
```

```
#define HOST_MMW_GET_C1_DATA_LENGTH(data_len) \
    (((data_len) - sizeof(FrameData_C1_t)) / sizeof(Frame_FFT_t))
```

In the callback function, the data can be parsed as follows:

```
void C1_Report_CB(uint32_t msg_id, uint8_t *data, uint32_t data_len, void *cb_arg)
{
    FrameData_C1_t *frame = (FrameData_C1_t *)data;
    uint32_t length = HOST_MMW_GET_C1_DATA_LENGTH(data_len);

    for (uint32_t i = 0; i < length; i++) {
        function(&frame->data[i]);
    }
}
```

5.2.2.1.2 Data structure passed to the C2 callback

```
typedef struct {
    uint32_t frameIdx;
    uint32_t frameLen; /* unit=BYTE */
    uint32_t dataOffset; /* unit=BYTE */

    uint32_t type : 8;
    uint32_t total_length : 24;
    uint8_t tx_num;
    uint8_t rx_num;
    uint8_t rev;
    uint8_t rsv;
    uint16_t range_bin_num;
    uint16_t dop_bin_num;

    Frame_FFT_t data[0];
} FrameData_C2_t;
#define HOST_MMW_GET_C2_DATA_LENGTH(data_len) \
    (((data_len) - sizeof(FrameData_C2_t)) / sizeof(Frame_FFT_t))
```

5.2.2.1.3 Data structure passed to the C3 callback

```
typedef struct {
    union {
        struct {
            uint16_t range;
            uint16_t azimuth;
            uint16_t elevation;
            uint16_t velocity;
            uint16_t snr;
            uint16_t reserved;
        } polar;
        struct {
            uint16_t x;

```

```

        uint16_t y;
        uint16_t z;
        uint16_t velocity;
        uint16_t snr;
        uint16_t reserved;
    } coord;
} type;
} Frame_Point_t;

typedef struct {
    uint32_t frameIdx;
    uint32_t frameLen; /* unit=DWORD */
    uint32_t dataOffset; /* unit=DWORD */

    uint8_t type;
    uint8_t flag;
    uint16_t total_length;

    Frame_Point_t data[0];
} FrameData_C3_t;
#define HOST_MMW_GET_C3_DATA_LENGTH(data_len) \
    (((data_len) - sizeof(FrameData_C3_t)) / sizeof(Frame_Point_t))

```

5.2.2.2 Other Data Processing Flows

To customize packet reassembly and parsing, call `HIF_Report_Handle_Regist` to set a callback function. The registered callback is invoked when a packet or a complete fragmented data unit is received.

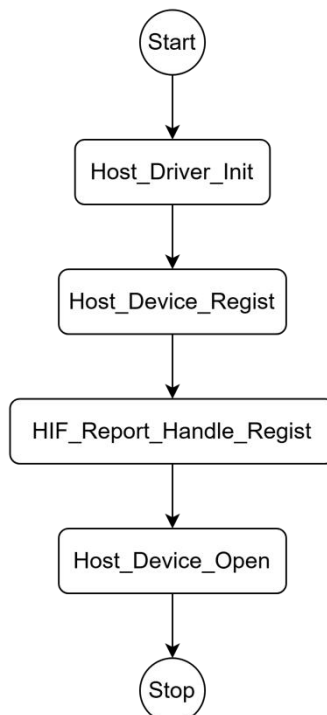


Figure 5-8 Custom Data Processing Scheme

Use `HIF_Report_Handle_Regist` to register a packet callback function. The function is declared as follows:

```
int HIF_Report_Handle_Regist(HIF_HANDLE hdl, uint8_t msg_id, Msg_Report_CB cb, void *arg);
```

The callback function type is as follows:

```
typedef void (*Msg_Report_CB)(uint32_t msg_id, uint8_t *payload, uint32_t payload_len, void *arg);
```

Here, msg_id is the registered message ID, payload is the packet Payload, payload_len is the packet length in bytes, and arg is the arg parameter specified when the callback is registered.

An example of using this function is shown below.

```
static DEV_HANDLE devHandle = NULL;

static void my_report_cb(uint32_t msg_id, uint8_t *data, uint32_t data_len, void *cb_arg)
{
    // Process and use the data
}

int main()
{
    DevHw_t devHw;
    HifCfg_t hifCfg = {
        .cmd_buf_len = 1024,
    };
    DevHw_Set_DefaultParam(&devHw, LLC_BUS_TYPE_SPI, LLC_BUS_PARAM_SPI, 0, 8000000, 4, 5, 6, 0);

    Host_Driver_Init();
    Host_Device_Regist(&devHandle, &devHw, &hifCfg);
    HIF_Report_Handle_Regist(Host_HIF_Handle_Get(devHandle), 0xE0, my_report_cb, devHandle);
    Host_Device_Open(devHandle);

    return 0;
}
```

5.2.2.3 API Usage

To process reported data, register the callback function for the corresponding message ID with the Device before opening the Device. The callback type is shown below.

Table 5-47 Report_CB

Callback function	Description
Function	typedef void (*Report_CB)(uint32_t msg_id, uint8_t *data, uint32_t data_len, void *cb_arg);
Functionality	After a frame/packet of data has been collected, provide the data to the user through a callback.
Parameters	uint32_t msg_id Message ID uint8_t *data Data uint32_t data_len Data length

	void *cb_arg Callback argument
Notes	The data parameter may be released after the callback returns. For C1, C2, and C3, the callback is invoked once per frame; for other messages, it is invoked once per packet.
Return Value	None

The Host Driver provides data-processing APIs for C1, C2, C3, C6, C7, C8, CA, and CB. The APIs for processing these Cx messages are defined below.

Table 5-48 Mmw_Cx_FUNCTION_Handle_Set

Callback function	Description
Function	int Mmw_Cx_FUNCTION_Handle_Set(DEV_HANDLE DevHdl, Report_CB cb, void *cb_arg);
Functionality	Set the Cx data callback. C1, C2, and C3 use frame callbacks; the others use packet callbacks.
Parameters	DEV_HANDLE DevHdl Device handle Report_CB cb Callback function void *cb_arg Callback argument
Notes	None
Return Value	HOST_ERRCODE_SUCCESS indicates success; any other value indicates failure.

The mapping between Cx and FUNCTION is as follows.

Table 5-49 Mapping Between Cx and FUNCTION

Message ID	FUNCTION
C1	CubeData
C2	FFTData
C3	Objects
C6	GeneralData
C7	PlotData
C8	MotionSensor
CA	MotionSensor
CB	MotionSensor

To set a processing callback for another message ID, use the following general message-ID callback setup API.

Table 5-50 Mmw_Report_Handle_Set_General

Callback function	Description
Function	int Mmw_Report_Handle_Set_General(DEV_HANDLE DevHdl, Report_CB cb, void *cb_arg, void *buffer, uint32_t buffer_len, uint8_t msg_id, void *proc_handle);
Functionality	Set the callback for a general message ID. This is a packet callback.

Parameters	DEV_HANDLE DevHdl Device handle Report_CB cb Callback function, stored in the message ID control object void *cb_arg Callback argument, stored in the message ID control object void *buffer Buffer for receiving data uint32_t buffer_len Buffer size uint8_t msg_id Message ID void *proc_handle Packet callback function of type void (*)(uint32_t msg_id, uint8_t *payload, uint32_t payload_len, void *arg), where msg_id is the message ID, payload is the packet Payload, payload_len is the Payload length, and arg is the message ID control object.
Notes	For each packet, proc_handle, the registered user callback cb and its argument cb_arg can be obtained from the message ID control object.
Return Value	HOST_ERRCODE_SUCCESS indicates success; any other value indicates failure.

The message ID control object is structured as follows.

```

#define REPORT_FLAG_BUFFER_USER      BIT(0)
#define REPORT_FLAG_BUFFER_HOLD     BIT(1)
typedef struct {
    DEV_HANDLE DevHdl;
    Report_CB  report_cb;
    void      *cb_arg;
    uint8_t   *buffer;
    uint32_t   bufferLen;

    uint8_t    msgId;
    uint8_t    flag;
    uint8_t    drop;
    #if (CFG_APP_RETRY_EN)
        uint8_t    retry_max;
        uint8_t    retry_start;
        uint8_t    retry_end;
        void      *retry_node;
    #else
        uint8_t    reserved;
    #endif
}

```

```

uint32_t   frameIdx;
uint32_t   frameLen;    /* unit=byte */
uint32_t   frameOffset; /* unit=byte */
} Report_Local_t;

```

To release all reporting-data processing functions and their allocated resources, call the following API.

Table 5-51 Mmw_Report_Handle_FreeAll

Callback function	Description
Function	void Mmw_Report_Handle_FreeAll(DEV_HANDLE DevHdl);
Functionality	Release all reporting-data processing functions and their allocated resources. Only resources allocated after registration through Mmw_Cx_FUNCTION_Handle_Set are released.
Parameters	DEV_HANDLE DevHdl Device handle
Notes	Only processing functions and allocated resources associated with a dedicated message-ID processing-function registration interface can be released.
Return Value	None

To release a custom reporting-data processing function and its allocated resources, use the following code.

```
Mmw_Report_Handle_Set_General(DevHdl, NULL, NULL, NULL, 0, MSG_ID, NULL);
```

5.2.3 Command Transmission

5.2.3.1 Usage Flow

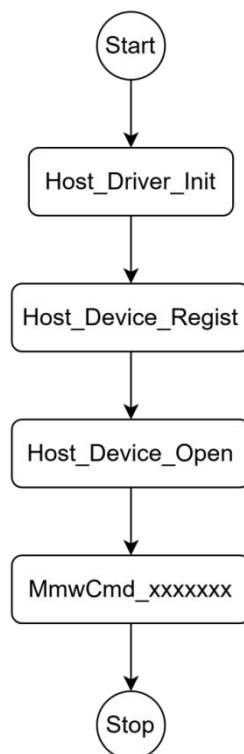


Figure 5-9 Command Transmission Flow

Commands must be transmitted after the Device is opened. The following uses obtaining the SDK version as an example to describe the command transmission flow.

```
static DEV_HANDLE devHandle = NULL;
```

```

int main()
{
    DevHw_t devHw;
    HifCfg_t hifCfg = {
        .cmd_buf_len = 1024,
    };
    DevHw_Set_DefaultParam(&devHw, LLC_BUS_TYPE_SPI, LLC_BUS_PARAM_SPI, 0, 8000000, 4, 5, 6, 0);

    Host_Driver_Init();
    Host_Device_Regist(&devHandle, &devHw, &hifCfg);
    Host_Device_Open(devHandle);
    HostCmd_Version_Get(devHandle, 0);

    return 0;
}

```

5.2.3.2 API Usage

Command control is divided into system command configuration and radar command configuration. **System command configuration is general**, independent of the specific radar model and firmware type, whereas **radar command configuration must be selected according to the radar model and firmware type**.

5.2.3.2.1 System Command Configuration

The Host Driver provides APIs for the obtain-version command, reboot-Device command, and general commands in cmd_api/host_driver.

Table 5-52 HostCmd_Version_Get

API	Description
Function	int HostCmd_Version_Get(DEV_HANDLE DevHdl, uint32_t user_version);
Functionality	Obtain the version.
Parameters	DEV_HANDLE DevHdl Device handle uint32_t user_version User-version selector. A value of 1 obtains the custom version; a value of 0 obtains the code version.
Notes	None
Return Value	0 indicates success; any other value indicates failure.

Table 5-53 HostCmd_Device_Reboot

API	Description
Function	int HostCmd_Device_Reboot(DEV_HANDLE DevHdl, uint32_t reboot);
Functionality	Reboot the Device.
Parameters	DEV_HANDLE DevHdl Device handle uint32_t reboot Reboot mode. A value of 1 specifies a software reboot; other values are not supported.

Notes	None
Return Value	0 indicates success; any other value indicates failure.

Table 5-54 Host_General_Command

API	Description
Function	int Host_General_Command(DEV_HANDLE DevHdl, uint32_t msg_id, void *param, uint32_t param_len, void *resp_buff, uint32_t buffer_len, uint32_t *data_len);
Functionality	Transmit a general command.
Parameters	DEV_HANDLE DevHdl Device handle uint32_t msg_id Message ID void *param Command Payload uint32_t param_len Command Payload length void *resp_buff Buffer for receiving the Response Payload uint32_t buffer_len Length of the buffer for receiving the Response Payload uint32_t *data_len Length of the received Response Payload. NULL can be passed if this length information is not required.
Notes	None
Return Value	0 indicates success; any other value indicates failure.

When transmitting a general command, use the following form as a reference. MSG_ID is the message ID of the command, and X and Y are the lengths of the command Payload and command Response Payload, respectively. Radar command configuration can be implemented in this form.

```
int HostCmd_xxxx(DEV_HANDLE DevHdl)
{
    int ret = 0;
    uint8_t cmd_payload[X];
    uint8_t resp_payload[Y];
    uint32_t resp_data_len;

    // set cmd_payload
    .....

    ret = Host_General_Command(DevHdl, MSG_ID, cmd_payload, X, resp_payload, Y, &resp_data_len);
}
```

```

    if (ret != HOST_ERRCODE_SUCCESS) {
        return ret;
    } else {
        return resp_payload[0];
    }
}

```

5.2.3.2.2 Radar Command Configuration

The Host Driver provides selected radar command configurations in `cmd_api/host_driver/rs6x_7x`. Radar command configuration is implemented based on the general command-transmission API.

To use command configuration for the radar analysis tool firmware (`radar_analysis`), first transmit the following command to that firmware to enable its HIF configuration and radar-parameter acquisition functions.

```
mmwc hif_regist
```

The following describes the general radar command-configuration APIs (excluding radar analysis).

Table 5-55 MmwCmd_General_StartCtrl_Cfg

API	Description
Function	<code>int MmwCmd_General_StartCtrl_Cfg(DEV_HANDLE DevHdl, uint32_t enable);</code>
Functionality	Start or stop the radar.
Parameters	DEV_HANDLE DevHdl Device handle uint32_t enable Start/stop control word. A value of 1 starts the radar; a value of 0 stops it.
Notes	None
Return Value	0 indicates success; any other value indicates failure.

Table 5-56 MmwCmd_General_MimoMode_Cfg

API	Description
Function	<code>int MmwCmd_General_MimoMode_Cfg(DEV_HANDLE DevHdl, uint32_t mimo_mode);</code>
Functionality	Configure the radar MIMO mode.
Parameters	DEV_HANDLE DevHdl Device handle uint32_t mimo_mode MIMO mode #define HOST_HIF_R3DATABOX_TLV_MIMO_1T1R 0x00 #define HOST_HIF_R3DATABOX_TLV_MIMO_1T3R 0x01 #define HOST_HIF_R3DATABOX_TLV_MIMO_2T3R 0x02 #define HOST_HIF_R3DATABOX_TLV_MIMO_1T4R 0x03 #define HOST_HIF_R3DATABOX_TLV_MIMO_2T4R 0x04 #define HOST_HIF_R3DATABOX_TLV_MIMO_1T2R 0x05
Notes	None
Return Value	0 indicates success; any other value indicates failure.

Table 5-57 MmwCmd_General_FrameType_Cfg

API	Description
Function	int MmwCmd_General_FrameType_Cfg(DEV_HANDLE DevHdl, uint32_t frame_type);
Functionality	Configure the radar frame type.
Parameters	DEV_HANDLE DevHdl Device handle uint32_t frame_type Radar frame type #define HOST_HIF_R3DATABOX_TLV_FRAME_TYPE_1D 0x00 #define HOST_HIF_R3DATABOX_TLV_FRAME_TYPE_2D 0x01
Notes	None
Return Value	0 indicates success; any other value indicates failure.

Table 5-58 MmwCmd_General_StartFreq_Cfg

API	Description
Function	int MmwCmd_General_StartFreq_Cfg(DEV_HANDLE DevHdl, uint32_t start_freq);
Functionality	Configure the radar start frequency.
Parameters	DEV_HANDLE DevHdl Device handle uint32_t start_freq Start frequency
Notes	None
Return Value	0 indicates success; any other value indicates failure.

Table 5-59 MmwCmd_General_TriggerRange_Cfg

API	Description
Function	int MmwCmd_General_TriggerRange_Cfg(DEV_HANDLE DevHdl, uint32_t trigger_range);
Functionality	Configure the radar trigger range.
Parameters	DEV_HANDLE DevHdl Device handle uint32_t trigger_range Trigger range
Notes	None
Return Value	0 indicates success; any other value indicates failure.

Table 5-60 MmwCmd_General_RangeResolution_Cfg

API	Description
Function	int MmwCmd_General_RangeResolution_Cfg(DEV_HANDLE DevHdl, uint32_t range_resolution);
Functionality	Configure the radar range resolution.

Parameters	DEV_HANDLE DevHdl Device handle uint32_t range_resolution Range resolution
Notes	None
Return Value	0 indicates success; any other value indicates failure.

Table 5-61 MmwCmd_General_MaxVelocity_Cfg

API	Description
Function	int MmwCmd_General_MaxVelocity_Cfg(DEV_HANDLE DevHdl, uint32_t max_velocity);
Functionality	Configure the radar maximum detectable velocity.
Parameters	DEV_HANDLE DevHdl Device handle uint32_t max_velocity Maximum detectable velocity
Notes	None
Return Value	0 indicates success; any other value indicates failure.

Table 5-62 MmwCmd_General_VelResolution_Cfg

API	Description
Function	int MmwCmd_General_VelResolution_Cfg(DEV_HANDLE DevHdl, uint32_t vel_resolution);
Functionality	Configure the radar velocity resolution.
Parameters	DEV_HANDLE DevHdl Device handle uint32_t vel_resolution Velocity resolution
Notes	None
Return Value	0 indicates success; any other value indicates failure.

Table 5-63 MmwCmd_General_FramePeriod_Cfg

API	Description
Function	int MmwCmd_General_FramePeriod_Cfg(DEV_HANDLE DevHdl, uint32_t period_ms);
Functionality	Configure the radar frame period.
Parameters	DEV_HANDLE DevHdl Device handle uint32_t period_ms Frame period
Notes	None

Return Value	0 indicates success; any other value indicates failure.
--------------	---

The radar analysis tool command APIs are as follows.

Table 5-64 MmwCmd_RadarAnalysis_Start

API	Description
Function	int MmwCmd_RadarAnalysis_Start(DEV_HANDLE DevHdl);
Functionality	Start the radar.
Parameters	DEV_HANDLE DevHdl Device handle
Notes	None
Return Value	0 indicates success; any other value indicates failure.

Table 5-65 MmwCmd_RadarAnalysis_Stop

API	Description
Function	int MmwCmd_RadarAnalysis_Stop(DEV_HANDLE DevHdl);
Functionality	Stop the radar.
Parameters	DEV_HANDLE DevHdl Device handle
Notes	None
Return Value	0 indicates success; any other value indicates failure.

Table 5-66 MmwCmd_RadarAnalysis_Mode_Cfg

API	Description
Function	int MmwCmd_RadarAnalysis_Mode_Cfg(DEV_HANDLE DevHdl, uint32_t txrx, uint32_t mode);
Functionality	Configure the radar MIMO mode and frame type.
Parameters	DEV_HANDLE DevHdl Device handle uint32_t txrx MIMO mode uint32_t mode Frame type
Notes	None
Return Value	0 indicates success; any other value indicates failure.

Table 5-67 MmwCmd_RadarAnalysis_Freq_Cfg

API	Description
Function	int MmwCmd_RadarAnalysis_Freq_Cfg(DEV_HANDLE DevHdl, uint32_t start_freq, uint32_t max_freq);
Functionality	Configure the radar start frequency and maximum frequency.
Parameters	DEV_HANDLE DevHdl Device handle

	uint32_t start_freq Start frequency uint32_t max_freq Maximum frequency
Notes	None
Return Value	0 indicates success; any other value indicates failure.

Table 5-68 MmwCmd_RadarAnalysis_Range_Cfg

API	Description
Function	int MmwCmd_RadarAnalysis_Range_Cfg(DEV_HANDLE DevHdl, uint32_t range, uint32_t resol);
Functionality	Configure the radar detection range and range resolution.
Parameters	DEV_HANDLE DevHdl Device handle uint32_t range Detection range uint32_t resol Range resolution
Notes	None
Return Value	0 indicates success; any other value indicates failure.

Table 5-69 MmwCmd_RadarAnalysis_Veloc_Cfg

API	Description
Function	int MmwCmd_RadarAnalysis_Veloc_Cfg(DEV_HANDLE DevHdl, uint32_t veloc, uint32_t resol);
Functionality	Configure the radar detection velocity and velocity resolution.
Parameters	DEV_HANDLE DevHdl Device handle uint32_t veloc Detection velocity uint32_t resol Velocity resolution
Notes	None
Return Value	0 indicates success; any other value indicates failure.

Table 5-70 MmwCmd_RadarAnalysis_Frame_Cfg

API	Description
Function	int MmwCmd_RadarAnalysis_Frame_Cfg(DEV_HANDLE DevHdl, uint32_t period, uint32_t num);

Functionality	Configure the radar frame period and number of frames to transmit.
Parameters	DEV_HANDLE DevHdl Device handle uint32_t period Frame period uint32_t num Number of frames to transmit. A value of 0 indicates continuous frame transmission.
Notes	None
Return Value	0 indicates success; any other value indicates failure.

Table 5-71 MmwCmd_RadarAnalysis_Intv_Cfg

API	Description
Function	int MmwCmd_RadarAnalysis_Intv_Cfg(DEV_HANDLE DevHdl, uint32_t period, uint32_t intv_num);
Functionality	Configure the radar interval period and count.
Parameters	DEV_HANDLE DevHdl Device handle uint32_t period Interval period uint32_t intv_num Number of intervals
Notes	Used only in 1D FFT mode.
Return Value	0 indicates success; any other value indicates failure.

Table 5-72 MmwCmd_RadarAnalysis_ChirpNum_Cfg

API	Description
Function	int MmwCmd_RadarAnalysis_ChirpNum_Cfg(DEV_HANDLE DevHdl, uint32_t num);
Functionality	Configure the number of chirps in each radar interval.
Parameters	DEV_HANDLE DevHdl Device handle uint32_t num Number of chirps per interval
Notes	None
Return Value	0 indicates success; any other value indicates failure.

Table 5-73 MmwCmd_RadarAnalysis_Report_Cfg

API	Description
Function	int MmwCmd_RadarAnalysis_Report_Cfg(DEV_HANDLE DevHdl, uint32_t report_type, uint32_t report_num);

Functionality	Configure the type and amount of reported data.
Parameters	DEV_HANDLE DevHdl Device handle uint32_t report_type Data type: 0 indicates CUBE data, 4 indicates point-cloud data, and 255 disables reporting. uint32_t report_num Data count, indicating the number of frames to report
Notes	None
Return Value	0 indicates success; any other value indicates failure.

Table 5-74 MmwCmd_RadarAnalysis_DopFft_Cfg

API	Description
Function	int MmwCmd_RadarAnalysis_DopFft_Cfg(DEV_HANDLE DevHdl, uint32_t flag);
Functionality	Enable or disable DOP FFT in 2D FFT mode.
Parameters	DEV_HANDLE DevHdl Device handle uint32_t report_type Enable flag. A value of 1 enables the function; a value of 0 disables it.
Notes	None
Return Value	0 indicates success; any other value indicates failure.

5.2.4 Firmware Download

5.2.4.1 Usage Flow

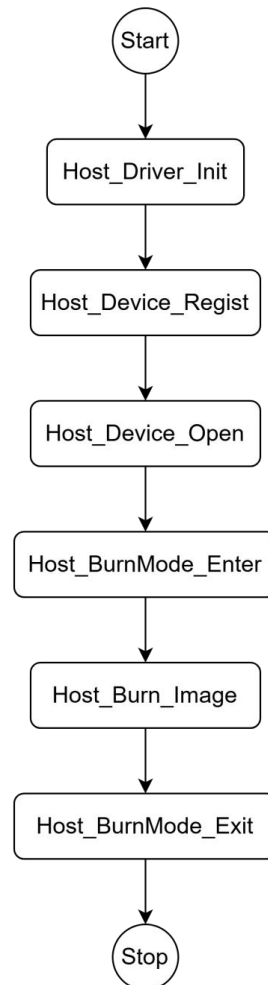


Figure 5-10 Firmware Download Flow

Firmware download must be performed after the Device is opened. An example is shown below:

```
static DEV_HANDLE devHandle = NULL;

int main()
{
    DevHw_t devHw;
    HifCfg_t hifCfg = {
        .cmd_buf_len = 1024,
    };
    DevHw_Set_DefaultParam(&devHw, LLC_BUS_TYPE_SPI, LLC_BUS_PARAM_SPI, 0, 8000000, 4, 5, 6, 0);

    Host_Driver_Init();
    Host_Device_Regist(&devHandle, &devHw, &hifCfg);
    Host_Device_Open(devHandle);
    Host_BurnMode_Enter(devHandle);
    Host_Burn_Image(devHandle, "image1");
    Host_BurnMode_Exit(devHandle);
}
```

```

    return 0;
}

```

The string "image1" is passed to the path parameter of the storage-media interface `int host_store_open(void **hdl, void *path, uint32_t *len)`. This interface returns the firmware image handle and firmware image length.

5.2.4.2 API Usage

To download firmware to the radar, first use the Host Driver to open a Device object, and then call the following API to place the radar in burn mode.

Table 5-75 Host_BurnMode_Enter

API	Description
Function	<code>int Host_BurnMode_Enter(DEV_HANDLE DevHdl);</code>
Functionality	Place a Device in burn mode.
Parameters	DEV_HANDLE DevHdl Device handle
Notes	None
Return Value	HOST_ERRCODE_SUCCESS indicates success; any other value indicates failure.

After entering burn mode, call the following API to download the firmware image. The path parameter is related to the ported `hal_store` interface and is vendor-defined; it can be used to select different firmware images.

Table 5-76 Host_Burn_Image

API	Description
Function	<code>int Host_Burn_Image(DEV_HANDLE DevHdl, void *path);</code>
Functionality	Burn a firmware image to a slave Device.
Parameters	DEV_HANDLE DevHdl Device handle void *path Firmware image
Notes	None
Return Value	HOST_ERRCODE_SUCCESS indicates success; any other value indicates failure.

After firmware download is complete, call the following API to make the radar exit burn mode and enter functional mode.

Table 5-77 Host_BurnMode_Exit

API	Description
Function	<code>void Host_BurnMode_Exit(DEV_HANDLE DevHdl);</code>
Functionality	Exit burn mode on a slave Device.
Parameters	DEV_HANDLE DevHdl Device handle
Notes	None
Return Value	None

Note that the Host Driver allows only one radar to be in burn mode at a time.

5.3 Configuration and Compilation

The `host_config.h` file in the root directory of the code package manages the global Host Driver configuration. `user_config.h` overrides the configuration macros in `host_config.h`. These settings can be modified for customization.

5.3.1 Feature Trimming

Global macros support code/feature trimming for four functions: transport-layer retransmission, message-layer fragmentation, message-layer retransmission, and application-layer retransmission. If message-layer fragmentation is trimmed, message-layer retransmission must also be trimmed.

Table 5-78 Global Macros for Feature Trimming

Macro	Description
CFG_TL_RETRY_EN	Enable transport-layer retransmission; can be used for code-size trimming.
CFG_MSG_FRAGMENT_EN	Enable message-layer fragmentation; can be used for code-size trimming.
CFG_MSG_FRAGMENT_RETRY_EN	Enable message-layer fragment retransmission; can be used for code-size trimming. If this function is enabled, message-layer fragmentation must also be enabled.
CFG_APP_RETRY_EN	Enable application-layer retransmission; can be used for code-size trimming.

5.3.2 Adaptation-Layer Feature Switches

The adaptation-layer feature-switch macros control logging, storage-media, communication-interface, OS, sleep, and I/O functions. The logging configuration also includes the log level and the path to the prototype/declaration of the log-printing interface.

Table 5-79 Global Macros for Enabling/Disabling Adaptation-Layer Features

Macro	Description
CFG_HOST_PORT_LOG_EN	Enable logging
CFG_HOST_PORT_LOG_FILE	host_log_print definition/declaration path relative to <code>porting/include/hal_log.h</code>
CFG_HOST_LOG_LEVEL	Log level: <code>HOST_LOG_LEVEL_xxx</code>
CFG_HOST_PORT_STORE_EN	Enable the storage-media interface
CFG_HOST_PORT_COM_EN	Enable COM; this function must be enabled.
CFG_HOST_PORT_COM_SPI_EN	Enable SPI COM
CFG_HOST_PORT_COM_I2C_EN	Enable I ² C COM
CFG_HOST_PORT_COM_UART_EN	Enable UART COM
CFG_HOST_PORT_OS_EN	Enable the OS interface; this function must be enabled.
CFG_HOST_PORT_PM_EN	Enable sleep
CFG_HOST_PORT_IO_EN	Enable the I/O interface; this function must be enabled.

5.3.3 Firmware Download Function

The detailed download-function macros configure Flash read/write, SRAM read/write, and Device PC-pointer setting. They can enable readback verification (supported only for UART firmware download), configure the number of retries for firmware-download commands and Response-packet reads, and configure checksum verification.

Table 5-80 Detailed Firmware Download Configuration

Macro	Description
CFG_HOST_BURN_DOWNLOAD_READ_CHECK_EN	Enable readback verification. When enabled, readback verification is performed during UART firmware download.
CFG_HOST_BURN_FLASH_EN	Enable firmware download to Flash
CFG_HOST_BURN_FLASH_READ_EN	Enable reading Device Flash
CFG_HOST_BURN_SRAM_EN	Enable firmware download to SRAM; not supported in SDK 2.1.0.
CFG_HOST_BURN_RUN_EN	Enable setting the Device PC pointer in burn mode; not supported in SDK 2.1.0.
CFG_HOST_BURN_RETRY_CNT	Number of command retries in burn mode
CFG_HOST_BURN_RSP_RETRY_CNT	Number of retries for reading Response packets in burn mode
CFG_HOST_BURN_CHECKSUM_EN	Enable checksum verification in burn mode
CFG_HOST_DRIVER_SPI_SPEED_IN_DOWNLOAD	Speed used for firmware download over SPI; must be lower than 8 MHz.

5.3.4 Message-Layer Functions

The detailed message-layer configuration macros configure exception detection for message-layer fragmentation, the maximum retained buffer size, and other settings.

Table 5-81 Global Macros for Configuring the Message Layer

Macro	Description
SUPPORT_HOST_HIF_FRAGMENT_PENDING_INFO_ABNORMAL_DETECT	Enable exception detection for message-layer fragmentation to clear invalid fragments.
CFG_HOST_HIF_FRAGMENT_PENDING_INFO_ABNORMAL_BURST_THR	Exception-detection threshold for message-layer fragmentation. If a fragmented stream is incomplete and the number of consecutively received bursts that do not belong to that stream reaches the value specified by this macro, the stream is cleared.
SUPPORT_HOST_HIF_FRAGMENT_PENDING_INFO_ABNORMAL_DETECT_TIMELY	Enable immediate clearing of abnormal fragments. When enabled, clearing is performed during data reception; otherwise, it is performed during data processing. This function is disabled by default to maintain data-reception efficiency.
ABNORMAL_FRAGMENT_INFO_LOG	Callback macro invoked before a clear operation, similar to a destructor.
CFG_HOST_HIF_FRAGMENT_FREE_INFO_NUM_MAX	Maximum number of idle fragmented-stream control blocks retained for each message ID of each Device. This can reduce memory fragmentation and improve fragmented-data-stream reception efficiency. -1 indicates no maximum limit, 0 indicates that none are retained, and >0 specifies the number retained.
CFG_HOST_HIF_FRAGMENT_HOLD_BUFFER_SIZE_MAX	Maximum size of a buffer previously used by a fragmented stream that can be retained for each message ID of each Device. After a fragmented stream is complete, the size of its buffer is compared with that of the retained buffer. If the fragmented-stream buffer is larger and the retained buffer is not in use, the retained buffer is released and the fragmented-stream buffer is retained. This macro limits buffer replacement during this process. If the fragmented-stream buffer meets the replacement conditions but exceeds the size

	limited by this macro, replacement is not performed. 0 indicates no size limit; >0 specifies the maximum retained size.
--	--

5.3.5 Other

Table 5-82 Other Configuration Macros

Macro	Description
_ALIGNAS_VARIABLE	Variable memory alignment. Modify this macro if variables have memory-alignment requirements.
_WEAK_FUNCTION	Virtual function definition
HOST_OS_TIMEOUT_FOREVER	OS wait forever
HOST_OS_TIMEOUT_NO_WAIT	OS no-wait
HOST_OS_THREAD_PRIO_NORMAL	Normal thread priority
HOST_OS_THREAD_PRIO_LOW	Low thread priority
HOST_OS_THREAD_PRIO_HIGH	High thread priority
HOST_HIF_POLL_ACK_TLV_IS_VALID	Macro for validating the POLL-ACK TLV. Do not modify this macro.
CFG_LLC_DEVICE_RX_CACHE_SIZE	Cache size, in bytes. When the notification type is COM_ISR or COM_IRQ_THREAD, a Cache is created to buffer data. COM_ISR uses the entire Cache to buffer data. COM_IRQ_THREAD uses the entire Cache only when the interrupt_ctrl interface is registered.
HOST_DEVICE_STATE_INFO	Enable obtaining Device status and information
HOST_HIF_CASE_GENERAL_EN	Enable general radar commands
HOST_HIF_CASE_RADAR_ANALYSIS_EN	Enable radar analysis commands

5.4 Simple Porting Test Code

After porting the HAL, write simple test code to verify that the porting was successful. The following application code demonstrates how to use UART and SPI Devices.

5.4.1 Data Reporting

5.4.1.1 UART

First, consider a UART Device using the COM_IRQ_THREAD notification type.

```
#include "host_driver.h"

static DEV_HANDLE devHandle = NULL;

static void Msg_C6_Report_CB(uint32_t msg_id, uint8_t *payload, uint32_t payload_len, void *arg)
{
    static uint64_t counter = 0;
    HOST_LOG_INF("[%llu] device(%p) recv 0x%02X msg\n", counter++, arg, msg_id);
    HOST_LOG_INF("payload len = %u\n", payload_len);
    HOST_LOG_INF_HEX(payload, ((payload_len > 16) ? 16 : payload_len), "PAYLOAD");
}
```

```

}

int main()
{
    int status = HOST_ERRCODE_SUCCESS;
    DevHw_t devHw;
    HifCfg_t hifCfg = {
        .cmd_buf_len = 1024,
    };
    DevHw_Set_DefaultParam(&devHw, LLC_BUS_TYPE_UART, 0, 1, 921600, 4, 5, 6, 1);

    HOST_LOG_PRINT("\n\n\n");
    HOST_LOG_INF("start to test application\n\n\n");

    status = Host_Driver_Init();
    if (status != HOST_ERRCODE_SUCCESS) {
        HOST_LOG_ERR("init host driver fail(%d)\n", status);
        goto out;
    }
    HOST_LOG_INF("init host driver succ\n");
    status = Host_Device_Regist(&devHandle, &devHw, &hifCfg);
    if (status != HOST_ERRCODE_SUCCESS) {
        HOST_LOG_ERR("regist device(%p) fail(%d)\n", devHandle, status);
        goto out;
    }
    HOST_LOG_INF("regist device(%p) succ\n", devHandle);
    status = HIF_Report_Handle_Regist(Host_HIF_Handle_Get(devHandle), 0xC6, Msg_C6_Report_CB,
    devHandle); // Change the second argument from 0xC6 to 0xC1 to receive 0xC1 data
    if (status != HOST_ERRCODE_SUCCESS) {
        HOST_LOG_ERR("regist 0xC6 report handle fail(%d)\n", status);
        goto out;
    }
    HOST_LOG_INF("regist 0xC6 report handle succ\n");
    status = Host_Device_Open(devHandle);
    if (status != HOST_ERRCODE_SUCCESS) {
        HOST_LOG_ERR("open device(%p) fail(%d)\n", devHandle, status);
        goto out;
    }
    HOST_LOG_INF("open device(%p) succ\n", devHandle);

out:
    HOST_LOG_PRINT("\n\n\n");
    HOST_LOG_INF("test application done\n\n\n");
}

```

5.4.1.2 SPI

For a 1-line SPI Device, the EDGE notification type is used as an example.

```

#include "host_driver.h"

static DEV_HANDLE devHandle = NULL;

static void Msg_C6_Report_CB(uint32_t msg_id, uint8_t *payload, uint32_t payload_len, void *arg)
{
    static uint64_t counter = 0;
    HOST_LOG_INF("[%llu] device(%p) recv 0x%02X msg\n", counter++, arg, msg_id);
    HOST_LOG_INF("payload len = %u\n", payload_len);
    HOST_LOG_INF_HEX(payload, ((payload_len > 16) ? 16 : payload_len), "PAYLOAD");
}

int main()
{
    int status = HOST_ERRCODE_SUCCESS;
    DevHw_t devHw;
    HifCfg_t hifCfg = {
        .cmd_buf_len = 1024,
    };
    DevHw_Set_DefaultParam(&devHw, LLC_BUS_TYPE_SPI, LLC_BUS_PARAM_SPI, 0, 8000000, 4, 5, 6, 0);

    HOST_LOG_PRINT("\n\n");
    HOST_LOG_INF("start to test application\n\n");

    status = Host_Driver_Init();
    if (status != HOST_ERRCODE_SUCCESS) {
        HOST_LOG_ERR("init host driver fail(%d)\n", status);
        goto out;
    }
    HOST_LOG_INF("init host driver succ\n");
    status = Host_Device_Regist(&devHandle, &devHw, &hifCfg);
    if (status != HOST_ERRCODE_SUCCESS) {
        HOST_LOG_ERR("regist device(%p) fail(%d)\n", devHandle, status);
        goto out;
    }
    HOST_LOG_INF("regist device(%p) succ\n", devHandle);
    status = HIF_Report_Handle_Regist(Host_HIF_Handle_Get(devHandle), 0xC6, Msg_C6_Report_CB, devHandle); // Change the second argument from 0xC6 to 0xC1 to receive 0xC1 data
    if (status != HOST_ERRCODE_SUCCESS) {
        HOST_LOG_ERR("regist 0xC6 report handle fail(%d)\n", status);
        goto out;
    }
    HOST_LOG_INF("regist 0xC6 report handle succ\n");
    status = Host_Device_Open(devHandle);
    if (status != HOST_ERRCODE_SUCCESS) {
        HOST_LOG_ERR("open device(%p) fail(%d)\n", devHandle, status);
    }
}

```

```
        goto out;
    }
    HOST_LOG_INF("open device(%p) succ\n", devHandle);

out:
    HOST_LOG_PRINT("\n\n");
    HOST_LOG_INF("test application done\n\n");
}
```

An example of the log during normal operation is shown below.

```
[I][0] device(10505708) recv 0xC6 msg
[I]payload len = 535
[I]PAYLOAD (16)
01 00 00 06 02 67 61 69 6E 20 66 61 63 74 6F 72

[I][1] device(10505708) recv 0xC6 msg
[I]payload len = 120
[I]PAYLOAD (16)
03 03 01 08 00 6D 6F 74 69 6F 6E 5F 70 6F 69 6E

[I][2] device(10505708) recv 0xC6 msg
[I]payload len = 535
[I]PAYLOAD (16)
01 00 00 06 02 67 61 69 6E 20 66 61 63 74 6F 72

[I][3] device(10505708) recv 0xC6 msg
[I]payload len = 72
[I]PAYLOAD (16)
03 03 01 04 00 6D 6F 74 69 6F 6E 5F 70 6F 69 6E

[I][4] device(10505708) recv 0xC6 msg
[I]payload len = 535
[I]PAYLOAD (16)
01 00 00 06 02 67 61 69 6E 20 66 61 63 74 6F 72

[I][5] device(10505708) recv 0xC6 msg
[I]payload len = 84
[I]PAYLOAD (16)
03 03 01 05 00 6D 6F 74 69 6F 6E 5F 70 6F 69 6E

[I][6] device(10505708) recv 0xC6 msg
[I]payload len = 535
[I]PAYLOAD (16)
01 00 00 06 02 67 61 69 6E 20 66 61 63 74 6F 72

[INF][7] device(10505708) recv 0xC6 msg
[INF]payload len = 72
```

```
[INF]PAYLOAD (16)
03 03 01 04 00 6D 6F 74 69 6F 6E 5F 70 6F 69 6E

[!][8] device(10505708) recv 0xC6 msg
[!]payload len = 535
[!]PAYLOAD (16)
01 00 00 06 02 67 61 69 6E 20 66 61 63 74 6F 72

[!][9] device(10505708) recv 0xC6 msg
[!]payload len = 84
[!]PAYLOAD (16)
03 03 01 05 00 6D 6F 74 69 6F 6E 5F 70 6F 69 6E
```

5.4.2 Configuration Transmission

5.4.2.1 UART

UART Device, COM_IRQ_THREAD notification type.

```
#include "host_driver.h"
#include "host_mmw_api.h"

static DEV_HANDLE devHandle = NULL;

int main()
{
    int status = HOST_ERRCODE_SUCCESS;
    DevHw_t devHw;
    HifCfg_t hifCfg = {
        .cmd_buf_len = 1024,
    };
    DevHw_Set_DefaultParam(&devHw, LLC_BUS_TYPE_UART, 0, 1, 921600, 4, 5, 6, 1);

    HOST_LOG_PRINT("\n\n");
    HOST_LOG_INF("start to test application\n\n");

    status = Host_Driver_Init();
    if (status != HOST_ERRCODE_SUCCESS) {
        HOST_LOG_ERR("init host driver fail(%d)\n", status);
        goto out;
    }
    HOST_LOG_INF("init host driver succ\n");
    status = Host_Device_Regist(&devHandle, &devHw, &hifCfg);
    if (status != HOST_ERRCODE_SUCCESS) {
        HOST_LOG_ERR("regist device(%p) fail(%d)\n", devHandle, status);
        goto out;
    }
    HOST_LOG_INF("regist device(%p) succ\n", devHandle);
    status = Host_Device_Open(devHandle);
```

```

    if (status != HOST_ERRCODE_SUCCESS) {
        HOST_LOG_ERR("open device(%p) fail(%d)\n", devHandle, status);
        goto out;
    }
    HOST_LOG_INF("open device(%p) succ\n", devHandle);

    host_os_delayms(20);

    HOST_LOG_INF("start to test command...\n");
    status = HostCmd_Version_Get(devHandle, 0);
    if (status != HOST_ERRCODE_SUCCESS) {
        HOST_LOG_ERR("get version fail(%d)\n", status);
        goto out;
    }
    HOST_LOG_INF("end test command!\n");

out:
    HOST_LOG_PRINT("\n\n");
    HOST_LOG_INF("test application done\n\n");
}

```

An example LOG during normal operation is shown below.

```

[!]start to test command...
[!][app_code] ir_type:0 ir_minor:0
im_minor:0 im_project:0 im_number:0
iv_major:2 iv_minor:0 iv_revision:7
[!]end test command!

```

5.5 Typical Debugging Process

5.5.1 Data Path

If no data is reported or no command can be sent after porting the adaptation layer, check the initialization process, the LLC send and receive functions, and the transport-layer thread execution flow.

5.5.1.1 Initialization Process

During initialization, Host_Device_Open ultimately calls com_ops.init and com_ops.open to initialize and open the communication port. Add LOG statements to these two interfaces and observe whether they are called when Host_Device_Open is invoked.

5.5.1.2 LLC_Send and LLC_Recv Paths

LLC_Send and LLC_Recv connect to com_ops.write and com_ops.read, respectively, which are the write and read interfaces of the communication port. Add LOG statements to these two interfaces and observe whether they are called when data is reported.

5.5.2 Data Reporting

5.5.2.1 HIF_TL_DataNotify_CB

For both active and passive reporting, HIF_TL_DataNotify_CB is called to notify the TL layer to receive data. LOG statements can be added to this function. For active reporting, the Device sends data to the Host communication port, and the Host calls this function after receiving the data. For passive reporting, this function is called on a rising edge at the Device Notify I/O pin connected to the Host.

5.5.2.2 com_ops.interrupt_ctrl

If thread scheduling is slow when the COM_ISR notification type is used, the interrupt threshold may have failed to be set or may be too small. Check whether the size parameter passed when com_ops.interrupt_ctrl is called is consistently 1 or 0. Under normal conditions, the size parameter should decrease from a larger value to a smaller value.

For example, when receiving a 3000-byte Payload, the size parameter may change as follows: 2967, 2933, 2899, ..., 56, 24, 8, 6, 4, 2, 1.

5.5.2.3 Protocol Logic

For active reporting, the Host generally does not need to send data to the Device while receiving data.

For passive reporting, data reception must comply with the data reporting protocol, such as [SPI data reporting](#). If no POLL command is sent, observe whether the HIF_TL_Transport_Task thread calls HIF_TL_SendPoll. If the Host does not receive data, observe whether the HIF_TL_Transport_Task thread calls HIF_TL_RecvMsg. If data is received but the user-defined callback does not receive it, observe whether the HIF_TL_Report_Task thread is executed.

5.5.3 Command Sending

5.5.3.1 Path for Sending a Command Request to the HIF_TL_Transport_Task Thread

The Host_General_Command function for sending commands calls HIF_Cmd_Request to send a command-transmission request to the HIF_TL_Transport_Task thread. If the Host does not receive a Response from the Device after a command-sending API provided by the application layer is called, check whether this path operates correctly.

For active reporting, first use a logic analyzer to check whether the Device sends a Response. For passive reporting, first use a logic analyzer to check whether the Notify I/O (PA6 by default) is driven high.

After confirming the preceding step, confirm that [HIF_TL_DataNotify_CB](#) is called correctly.

5.5.3.2 Protocol Logic

If the command is sent normally but the Device does not respond correctly, first check whether the transmitted command is correct and whether the status code in the Response returned by the Device is 0. For the format, see the description of common Command-Response formats in the appendix.

5.5.4 Firmware Download

5.5.4.1 Synchronization Failure

When Host_BurnMode_Enter is used to place the Device in download mode, the Device is synchronized. If synchronization fails, confirm that the RST I/O and UPD I/O timing is correct, that the Device is enabled when RST_IO outputs a high level, and that after the Host sends 0x7E the Device responds with 0x59 (AS2525) or 0x79 (AS2531). If the response is 0x79, check whether the Host continues by sending 0x55 0xFF and the Device then responds with 0x79.

5.6 Notes

5.6.1 Automatic C1, C2, and C3 Packet Assembly at the Application Layer

During automatic packet assembly at the application layer, the Buffer used for the previous Frame is retained and used when data for the next Frame is received. Therefore, use or back up the data before the next Frame arrives.

5.6.2 Sleep Function

To enable the sleep function, ensure that all required peripherals and other resources are restored before the WKIO interrupt callback is invoked.

6 Device Development

6.1 Code Directory Structure

The inc and src folders contain the main HIF protocol code. For a new project, add the code in both folders.

The cmd folder contains commands such as executing arbitrary functions and read/write memory. Add them as required.

```
<home>/psdf_sdk/subsys/hif
├── cmd                                /* Command code directory */
│   ├── hif_cmd.c                     /* Command registration/initialization */
│   ├── hif_cmd..h                   /* Command declarations and configuration */
│   ├── cmd                           /* Command directory */
│   │   ├── hif_cmd_0x23_fexec.c      /* Execute an arbitrary function */
│   │   ├── hif_cmd_0x24_readMem.c    /* read memory */
│   │   └── hif_cmd_0x25_writeMem.c   /* write memory */
├── inc                               /* HIF protocol header-file directory */
│   ├── hif_checksum.h                /* Check Sum calculation API */
│   ├── hif_config.h                 /* Feature, resource tailoring configuration and default-parameter
macro definitions */
│   ├── hif_dll.h                     /* Data link layer; processes data transmission and reception by Frame
*/
│   ├── hif_io.h                      /* I/O control: the Device notifies or wakes the Host, and the Host
wakes the Device through I/O */
│   ├── hif_log.h                     /* Debugging: controls level-based printing and the Trace I/O
debugging method */
│   ├── hif_mem.h                     /* Memory-pool management and linked-list operation interfaces */
│   ├── hif_ml.h                      /* Msg Level, message layer, primarily containing the fragmentation
function */
│   ├── hif_msg.h                     /* Definitions of HIF formats, IDs, status codes, and related items */
│   ├── hif_phy_can.h                 /* PHY-layer CAN communication interface; check whether the
corresponding version supports it */
│   ├── hif_phy_dma.h                 /* PHY-layer DMA interface */
│   ├── hif_phy_i2c.h                 /* PHY-layer I2C communication interface; check whether the
corresponding version supports it */
│   ├── hif_phy_spi.h                 /* PHY-layer SPI communication interface; check whether the
corresponding version supports it */
│   ├── hif_phy_uart.h                /* PHY-layer UART communication interface; check whether the
corresponding version supports it */
│   └── hif_phy.h                     /* Common PHY-layer data types and DLL interfaces */
```

```

├──hif_pm.h           /* Sleep and wakeup management */
├──hif_priv.h         /* Internally shared data types and interface definitions */
├──hif_tl.h           /* Frame transport timeout control */
├──hif_type.h         /* Data-type and error-code definitions */
├──hif.h              /* External configuration data structures and interface definitions */
├──src                /* Source-file directory */
│   ├──hif_checksum.c
│   ├──hif_dll.c
│   ├──hif_io.c
│   ├──hif_mem.c
│   ├──hif_ml.c
│   ├──hif_msg.c       /* Msg registration and allocation, and implementation of common
Msg processing */
│   ├──hif_phy_can.c
│   ├──hif_phy_dma.c
│   ├──hif_phy_i2c.c
│   ├──hif_phy_spi.c
│   ├──hif_phy_uart.c
│   ├──hif_phy.c
│   ├──hif_pm.c
│   ├──hif_tl.c
│   └──hif.c           /* Main HIF logic implementation and external interface
encapsulation */

```

6.2 Usage Process

Basic usage:

1. Create a project and add the preceding files to the project;
2. Initialize HIF;
3. Register a receive callback (optional, as required). After the Device receives data from the Host, it invokes the corresponding callback based on the ID.
4. Report data.

Example:

```

/**
 * @brief Message(ID: 0x18) receive callback
 * @param msg Message Header
 * @return int
 */
int hif_msghdl_0x18_callback(HIF_MsgHdr_t *msg)
{

```

```

/* ack status; for optional statuses, see API: HIF_MsgResp */
uint8_t cmd_status = HIF_CMD_STATUS_SUCCESS;
.....
return HIF_MsgResp(msg, respDataTestLen, cmd_status);
}

int main(void)
{
    int status = HAL_STATUS_OK;

    /* 2. Initialize */
    HIF_CfgParam_t hifCfgParam;
    HIF_DefaultInitParam(&hifCfgParam, HIF_COM_TYPE_SPI, 56000000);

    status = HIF_Init(&hifCfgParam);
    if (status != HAL_STATUS_OK) {
        LOG_PRINT("HIF_Init FAIL (%d)\n", status);
        return status;
    }

    /* 3. Register the receive callback */
    status = HIF_MsgHdl_Regist(0x18, hif_msghdl_0x18_callback);
    if (status != HAL_STATUS_OK) {
        LOG_PRINT("HIF_MsgHdl_Regist 0x18 FAIL (%d)\n", status);
    }

    /* 4. Send data. pData is the data address, dataLen is the data length, and NULL indicates that the
    send callback is empty. */
    status = HIF_MsgReport(0xC6, pData, dataLen, NULL);

    return 0;
}

```

6.3 Configuration Description

HIF functions are configured/controlled through the HIF_CfgParam_t structure.

The default configuration is in hif_config.h.

6.3.1 Configuration Structure and Global Variable

The global variable HIF_CfgParam_t hifCfg stores the real-time HIF configuration parameters. These parameters can be configured using default, static, or dynamic configuration.

The HIF_CfgParam_t structure is defined as follows:

```

typedef struct {
    /* com
    */
    HIF_ComType_t      comType;
    uint32_t           comId;
    uint32_t           comRate;
    /* uart :
    *         start bit, data bit, stop bit, parity
    *
    * i2c :

```

```

*      address
* spi :
*      line:
*          0: stand
*          1: dual
*          2: quad
*/
uint32_t          comParam;

HIF_ReportMode_t  reportMode;

/* device notify host
*/
HIF_NotifyType_t   notifyType;
HIF_IOLevel_t     notifyIoLevel;
uint32_t          notifyIoNum;

/* host wakeup device
*/
HIF_WakeType_t     wakeType;
HIF_IOLevel_t     wakeIoLevel;
uint32_t          wakeIoNum;
uint32_t          wakeComParam;

/* timeout
*/
/* begin receive frame magic to all frame
*/
uint32_t          recvFrameTo;
/*
* from the start of sending to successful sending.
*/
uint32_t          sendFrameTo;
/* configuration phase
* after waking up the device but before sending any cmd,
* if a timeout occurs, this is marked as a false wake-up.
*/
uint32_t          sendDelay;
/* configuration phase
* a series of cmd were sent, but the sleep cmd was not sent for a long time.
* after this timeout, the system will auto enter sleep mode.
*
* calc start from the resoinse to the last cmd.
*/
uint32_t          sleepTo;

uint32_t          splitSize;
uint32_t          splitTotalSize;
uint32_t          splitEna;
uint32_t          splitRetryEna;

uint32_t          reportInitStatus;

```

uint32_t	retryEna;
uint32_t	burstEna;
uint32_t	sendMode;
uint32_t	recvMode;
uint32_t	cmdrspBuffSize;
uint32_t	frameNodeSize;
uint32_t	dataNodeSize;
uint32_t	semNodeSize;
uint32_t	taskPrio;
uint32_t	taskStackSize;
uint32_t	cascadeThresh;
uint32_t	msgNodeSize;
uint32_t	sendNodeLevel;
} HIF_CfgParam_t;	

6.3.2 Default Configuration

That is, the default HIF configuration in hif_config.h.

6.3.2.1 Basic Configuration

Table 6-1 HIF Basic Configuration

Configuration Item	Description	Default
CONFIG_HIF	0 disables HIF and excludes the code from compilation; 1 enables HIF.	0
CONFIG_HIF_MODE	HIF mode configuration: 0 - Ultra-low-power mode; 1 - Standard mode. (The following configuration descriptions are based on standard mode.)	1
CONFIG_HIF_TASK_STACK_SIZE	HIF Task stack configuration.	1024
CONFIG_HIF_TASK_PRIO	HIF Task priority.	14
CONFIG_HIF_MEM_CMD_RSP_SIZE	Receive command Buffer size: maximum received command length, in Bytes.	512
CONFIG_HIF_MSG_PAYLOAD_MAX_SIZE	Maximum Payload length of a reported Frame. The maximum valid value is 4095.	3839
CONFIG_HIF_MEM_FRAME_NODE_SIZE	Frame memory-pool size.	16
CONFIG_HIF_MEM_DATA_NODE_SIZE	Data Node memory-pool size.	64
CONFIG_HIF_MEM_SEM_NODE_SIZE	Sem semaphore memory-pool size (used for blocking data reporting).	4

Configuration Item	Description	Default
CONFIG_HIF_PHY_TYPE	Supported physical communication methods: bit 0 indicates UART, bit 1 indicates I ² C, bit 2 indicates SPI, and bit 3 indicates CAN. A bit set to 1 indicates support for the corresponding communication method. For example, a value of 5 indicates support for UART and SPI.	0x07

6.3.2.2 Feature Configuration

This section describes configuration items for optional HIF features.

Table 6-2 HIF Feature Configuration Items

Configuration Item	Description	Default
CONFIG_HIF_RETRY_ENA	Transport-layer retransmission enable: 0 - Transport-layer retransmission is not supported; 1 - Transport-layer retransmission is supported.	0
CONFIG_HIF_SPLIT_ENA	Fragmentation enable: 0 - Fragmentation is not supported; 1 - Fragmentation is supported.	0
CONFIG_HIF_MEM_MSG_NODE_SIZE	HIF Msg memory-pool size. Used for fragmentation and valid when fragmentation is enabled.	4
CONFIG_HIF_SEND_SPLIT_RETRY_ENA	Fragment retransmission enable: 0 - Fragment retransmission is not supported; 1 - Fragment retransmission is supported. Fragmentation must be enabled before fragment retransmission can be used.	0
CONFIG_HIF_PM	HIF PM enable: 0 - PM is not supported; 1 - PM is supported. This configuration follows CONFIG_PM; that is, HIF supports PM when the system supports PM.	CONFIG_PM

6.3.3 Other Configuration

More detailed feature configurations include whether UART transmission uses DMA or interrupt transfer, the default I²C communication address, and the default GPIO and level for Notify I/O. Modification is not recommended.

6.3.4 Configuration and Usage

6.3.4.1 Using the Default Configuration

Initialize/use HIF with the default configuration described in the preceding section. This is the simplest and most direct method.

For example, SPI, UART, and I²C are initialized with the default configuration as follows:

```
/* 1. Initialize HIF with the default configuration using "SPI+56 MHz" */
HIF_CfgParam_t hifCfgParam;
HIF_DefaultInitParam(&hifCfgParam, HIF_COM_TYPE_SPI, 56000000);

status = HIF_Init(&hifCfgParam);
```

```

    if (status != HAL_STATUS_OK) {
        LOG_PRINT("HIF_Init FAIL (%d)\n", status);
        return status;
    }

    /* 2. Initialize HIF with the default configuration using "UART+921600baud" */
    HIF_CfgParam_t hifCfgParam;
    HIF_DefaultInitParam(&hifCfgParam, HIF_COM_TYPE_UART, 921600);

    status = HIF_Init(&hifCfgParam);
    if (status != HAL_STATUS_OK) {
        LOG_PRINT("HIF_Init FAIL (%d)\n", status);
        return status;
    }

    /* 3. Initialize HIF with the default configuration using "I2C+400Khz" */
    HIF_CfgParam_t hifCfgParam;
    HIF_DefaultInitParam(&hifCfgParam, HIF_COM_TYPE_IIC, 400000);

    status = HIF_Init(&hifCfgParam);
    if (status != HAL_STATUS_OK) {
        LOG_PRINT("HIF_Init FAIL (%d)\n", status);
        return status;
    }

```

6.3.4.2 Static Project Configuration

Modifying the contents of hif_config.h is not recommended.

If different Projects require different HIF features or configurations, such as retransmission, fragmentation, or HIF memory-pool size configuration, redefine the configuration in the project's prj_config.h using the same macro name. The macros in the project's prj_config.h override the corresponding macros in hif_config.h, allowing part of the HIF configuration to be changed for different requirements or features.

Example:

The following prj_config.h redefines several HIF configurations:

```

#ifndef _PRJ_CONFIG_H
#define _PRJ_CONFIG_H

#define CONFIG_XIP 1

#define CONFIG_PM 1
/* Enable HIF */
#define CONFIG_HIF 1
/* Modify the hif_Task priority */
#define CONFIG_HIF_TASK_PRIO 8
/* Modify the hif_Task stack size */
#define CONFIG_HIF_TASK_STACK_SIZE 2048
/* Modify the maximum Payload length of a reported Frame */
#define CONFIG_HIF_MSG_PAYLOAD_MAX_SIZE 1024
/* Modify the receive Buffer size (accommodates the Frame Header, Payload, and CheckSum) */
#define CONFIG_HIF_CMD_RSP_SIZE_MIN 512
/* Enable fragmentation */
#define CONFIG_HIF_SPLIT_ENA 1

```

```
#endif /* _PRJ_CONFIG_H */
```

The project then initializes HIF using the preceding configuration items and the other configuration items in `hif_config.h`.

Example:

```
HIF_CfgParam_t hifCfgParam;
HIF_DefaultInitParam(&hifCfgParam, HIF_COM_TYPE_SPI, 56000000);

status = HIF_Init(&hifCfgParam);
if (status != HAL_STATUS_OK) {
    LOG_PRINT("HIF_Init FAIL (%d)\n", status);
    return status;
}
```

6.3.4.3 Dynamic Configuration

After HIF initialization, individual parameters can still be modified through `HIF_ExtControl()`, which is suitable for runtime parameter adjustment.

Example:

```
/* Change the send timeout to 50 ms */
uint32_t sendTo = 50;
status = HIF_ExtControl(HIF_SET_SEND_FRAME_TO, &sendTo, sizeof(uint32_t));
if (status != 0) {
    return status;
}
```

6.4 API Usage Instructions

6.4.1 Status Codes

Table 6-3 Status Codes

Status	Value	Meaning
HIF_ERRCODE_SUCCESS	0	Command executed successfully
HIF_ERRCODE_NOT_READY	-1	Invalid state; not ready
HIF_ERRCODE_INVALID_PARAM	-2	Invalid parameter
HIF_ERRCODE_NO_BUFFER	-3	Insufficient buffer
HIF_ERRCODE_IO_ERROR	-4	Transport error
HIF_ERRCODE_TIMEOUT	-5	Timeout
HIF_ERRCODE_ABORTED	-6	Forced termination
HIF_ERRCODE_TOO_LONG	-7	Data-length overflow
HIF_ERRCODE_FAIL	-8	General failure

6.4.2 API

6.4.2.1 HIF_DefaultInitParam

Interface	Description
Function	<pre>int HIF_DefaultInitParam(HIF_CfgParam_t * cfgParam, HIF_ComType_t comType, uint32_t comRate);</pre>
Functionality	Set the default HIF parameter configuration.
Parameters	HIF_CfgParam_t * cfgParam HIF parameter configuration structure. HIF_ComType_t comType Communication type, such as SPI or UART. uint32_t comRate Communication rate of the physical interface.
Notes	None.
Return Value	Returns zero on success and a nonzero value on failure. For details, see Status Codes.

6.4.2.2 HIF_Init

Interface	Description
Function	<pre>int HIF_Init(HIF_CfgParam_t *pInitParam);</pre>
Functionality	Initialize HIF.
Parameters	HIF_CfgParam_t * pInitParam HIF parameter configuration structure.
Notes	pInitParam can be NULL. If it is NULL, call HIF_SET_CFG_PARAM of HIF_ExtControl before this function to configure the parameters in HIF; otherwise, initialization fails. If pInitParam is not NULL, initialization uses the parameters in pInitParam regardless of whether parameters have been configured through HIF_ExtControl.
Return Value	Returns zero on success and a nonzero value on failure. For details, see Status Codes.

6.4.2.3 HIF_ExtControl

Interface	Description
Function	<pre>int HIF_ExtControl(HIF_Attr_t attr, void * arg, uint32_t argSize);</pre>
Functionality	Get or set HIF configuration parameters.

Interface	Description
Parameters	HIF_Attr_t attr Configuration type. HIF_SET_CFG_PARAM and HIF_GET_CFG_PARAM set or get the complete parameter set; the other types set or get an individual parameter. void * arg Configuration parameter buff. uint32_t argSize Configuration parameter length.
Notes	For HIF_SET_CFG_PARAM and HIF_GET_CFG_PARAM, the parameter length is sizeof(HIF_Attr_t). For other types, the parameter length is fixed at 4. This function can be called before or after HIF_Init.
Return Value	Returns zero on success and a nonzero value on failure. For details, see Status Codes.

6.4.2.4 HIF_MsgReport_ListPushData

Interface	Description
Function	<pre>int HIF_MsgReport_ListPushData(HIF_Data_ListHead_t * pDataListHead, void *data, uint16_t len);</pre>
Functionality	Push data into the linked list, at the head of the list.
Parameters	HIF_Data_ListHead_t * pDataListHead Data linked list. void *data Data address. uint16_t len Data length.
Notes	For the first node of a Report, initialize the doubly headed linked list pDataListHead; that is, initialize both pDataListHead->head and pDataListHead->tail to NULL. On failure, use HIF_MsgReport_ListPopData() to release HIF internal resources.
Return Value	Returns zero on success and a nonzero value on failure. For details, see Status Codes.

6.4.2.5 HIF_MsgReport_ListPopData

Interface	Description
Function	<pre>uint8_t *HIF_MsgReport_ListPopData(HIF_Data_ListHead_t * pDataListHead);</pre>
Functionality	Pop the data address from the head node and advance the linked-list head.

Interface	Description
Parameters	HIF_Data_ListHead_t * pDataListHead Data linked list.
Notes	None.
Return Value	User data buffer. NULL indicates that the current linked list is empty.

6.4.2.6 HIF_MsgReport_ListStart

Interface	Description
Function	int HIF_MsgReport_ListStart(uint8_t msgId, HIF_Data_ListHead_t * pDataListHead, HIF_MsgTrans_Callback sentCb);
Functionality	Start linked-list data transmission.
Parameters	uint8_t msgId Message ID. HIF_Data_ListHead_t * pDataListHead Report data doubly headed linked list. Must not be NULL. HIF_MsgTrans_Callback sentCb Send callback. If NULL, blocking transmission is used and this interface returns after transmission ends, whether successfully or unsuccessfully. If not NULL, this interface returns immediately and calls sentCb after data transmission ends. In sentCb, call HIF_MsgReport_ListPopData() to pop the transmitted data address.
Notes	When sentCb == NULL, after the function returns, use HIF_MsgReport_ListPopData() to release the hif buffer. When sentCb != NULL: If the function returns failure, use HIF_MsgReport_ListPopData() to release the hif buffer. If the function returns success, call HIF_MsgReport_ListPopData() in sentCb to pop the transmitted data address.
Return Value	Returns zero on success and a nonzero value on failure. For details, see Status Codes.

6.4.2.7 HIF_MsgReport

Interface	Description
-----------	-------------

Interface	Description
Function	<pre>int HIF_MsgReport(uint8_t msgId, void * pData, uint32_t len, HIF_MsgTrans_Callback sentCb);</pre>
Functionality	Report data without a linked list.
Parameters	uint8_t msgId Message ID. void *data User data. uint32_t len Data length. HIF_MsgTrans_Callback sentCb Send callback. If NULL, blocking transmission is used and this interface returns after transmission ends, whether successfully or unsuccessfully. If not NULL, this interface returns immediately and calls sentCb after data transmission ends. In sentCb, call HIF_MsgReport_ListPopData() to pop the transmitted data address.
Notes	When sentCb != NULL, if the return value indicates success, do not modify the user data or release *data before the callback is invoked.
Return Value	Returns zero on success and a nonzero value on failure. For details, see Status Codes.

6.4.2.8 HIF_MsgHdl_Regist

Interface	Description
Function	<pre>int HIF_MsgHdl_Regist(uint8_t msgId, HIF_MsgHdl_t msgHdl);</pre>
Functionality	Register a message-processing function.
Parameters	uint8_t msgId Message ID. HIF_MsgHdl_t msgHdl Message-processing function pointer.
Notes	None.
Return Value	Returns zero on success and a nonzero value on failure. For details, see Status Codes.

6.4.2.9 HIF_MsgResp

Interface	Description
Function	int HIF_MsgResp(HIF_MsgHdr_t *pMsg, uint16_t dataLen, uint8_t status);
Functionality	Reply to the Host from a message-processing function.
Parameters	HIF_MsgHdr_t *pMsg Message Header. The Cmd ID must match cmd. uint16_t dataLen Payload length, excluding status. uint8_t status Cmd execution status. <pre> /*! @brief General Ack Data */ #define HIF_CMD_STATUS_SUCCESS 0 /**< Cmd Success */ #define HIF_CMD_STATUS_UNSUPPORT 1 /**< Unsupport Cmd */ #define HIF_CMD_STATUS_VERSION 2 /**< Version not match */ #define HIF_CMD_STATUS_TOOLONG 3 /**< Message too long */ #define HIF_CMD_STATUS_CHECK_FAILED 4 /**< Head or Data check failed */ #define HIF_CMD_STATUS_PARAM 5 /**< Invalid Param */ #define HIF_CMD_STATUS_REPLAY 6 /**< Replay message */ #define HIF_CMD_STATUS_AUTH 7 /**< Auth failed */ #define HIF_CMD_STATUS_LATE 8 /**< Result need report later */ #define HIF_CMD_STATUS_BUSY 9 /**< Device is busy now */ #define HIF_CMD_STATUS_STATE 10 /**< Invalid Device State to process Cmd */ #define HIF_CMD_STATUS_SYSERR 11 /**< System error in Cmd Handler */ #define HIF_CMD_STATUS_IO 12 /**< General error */ </pre>
Notes	None.
Return Value	Returns zero on success and a nonzero value on failure. For details, see Status Codes.

6.4.2.10 HIF_InitStatusReport

Interface	Description
Function	int HIF_InitStatusReport(uint8_t status);
Functionality	Report the system startup status.
Parameters	uint8_t status System startup status.
Notes	Call this function before HIF_Init.
Return Value	Returns zero on success and a nonzero value on failure. For details, see Status Codes.

6.4.2.11 HIF_MsgReport_Lock

Interface	Description
Function	void HIF_MsgReport_Lock(void);
Functionality	Prevent the system from reporting data.
Parameters	None
Notes	Prevent data reporting while parameters are being configured, and remove the prevention after configuration is complete.
Return Value	None

6.4.2.12 HIF_MsgReport_Unlock

Interface	Description
Function	void HIF_MsgReport_Unlock(void);
Functionality	Remove the prevention of system data reporting.
Parameters	None
Notes	Prevent data reporting while parameters are being configured, and remove the prevention after configuration is complete.
Return Value	None

6.4.2.13 HIF_PM_WakeupDevice

Interface	Description
Function	void HIF_PM_WakeupDevice(void);
Functionality	Wake HIF.
Parameters	None
Notes	None
Return Value	None

6.4.2.14 HIF_PM_SuspendDevice

Interface	Description
Function	void HIF_PM_SuspendDevice(void);
Functionality	Suspend HIF.
Parameters	None
Notes	None

Interface	Description
Return Value	None

6.4.3 Data Reporting

Data reporting provides two types of interfaces: linked-list data reporting and non-linked-list data reporting.

Linked-list data reporting: When several noncontiguous data blocks must be combined into one packet for transmission, the conventional method is to copy the data into a transmit Buffer for packet assembly. However, this increases the Data Size and data-copy time, and the impact is more severe when multiple packets are queued. Linked-list data reporting links the data to be transmitted in a linked list, reducing transmit-buffer backup and copy time.

Example Scenario 1:

There are three Buffers at noncontiguous addresses: Buffer_1, Buffer_2, and Buffer_3. To combine these three Buffers into data and transmit it as the Payload of one HIF Frame, use linked-list data reporting.



Figure 6-1 Linked-List Data Reporting Example Scenario 1

Example Scenario 2:

BufferA is divided into three blocks: BufferA_1, BufferA_2, and BufferA_3. Data Headers Header_1, Header_2, and Header_3 must be added before the respective Buffer blocks to form the following data structure, which is then transmitted as the Payload of one HIF Frame. Linked-list data reporting can be used.

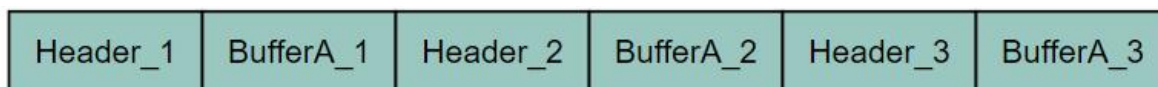


Figure 6-2 Linked-List Data Reporting Example Scenario 2

Non-linked-list data reporting: When all data to be reported resides in the same buffer, the data at that address can be reported directly.

Reporting methods are divided into blocking and callback methods.

Blocking: When the reporting interface is called to report data, no callback (NULL) is passed. After the interface transfers the data into HIF, it does not return until HIF finishes transmitting the data.

Callback/non-blocking: A callback can be passed when the reporting interface is called to report data. After the interface transfers the data into HIF, it returns immediately. HIF invokes the callback after it finishes transmitting the data. This method is suitable for highly concurrent scenarios.

6.4.3.1 Blocking Linked-List Transmission

Related interfaces: *HIF_MsgReport_ListPushData*, *HIF_MsgReport_ListPopData*, *HIF_MsgReport_ListStart*.

Notes:

If *HIF_MsgReport_ListPushData* fails to add data to the linked list, stop and call *HIF_MsgReport_ListPopData* to release the HIF internal resources in the linked list.

HIF_MsgReport_ListStart returns, call *HIF_MsgReport_ListPopData* to release the HIF internal resources in the linked list.

For other notes, see the descriptions of the related APIs.

Example:

```

int status = 0;
/* Initialize the Report data linked list. Note: The head and tail must both be assigned NULL. */
HIF_Data_ListHead_t dataList = {NULL, NULL};

do {

```

```

/* Add Report data pRptDataA to the transmit queue */
status = HIF_MsgReport_ListPushData(&dataList, pRptDataA, rptLenA);
if (status != 0) {
    /* Always check the return value. On failure, stop adding data and call
HIF_MsgReport_ListPopData to release the HIF internal resources in the linked list; otherwise, a memory
leak occurs. */
    break;
}

/* Add Report data pRptDataB to the transmit queue */
status = HIF_MsgReport_ListPushData(&dataList, pRptDataB, rptLenB);
if (status != 0) {
    break;
}

/* Add Report data pRptDataC to the transmit queue */
status = HIF_MsgReport_ListPushData(&dataList, pRptDataC, rptLenC);
if (status != 0) {
    break;
}

/* Start Report data transmission. A NULL third parameter indicates blocking transmission; the
function returns only after transmission is complete. */
status = HIF_MsgReport_ListStart(0xC6, &dataList, NULL);
} while (0);

/* Always reclaim resources; otherwise, memory leaks occur. */
uint8_t *pBuff = NULL;
do {
    /* Repeatedly pop buffs. HIF internal resources are released while the buffs are popped. */
    pBuff = HIF_MsgReport_ListPopData(&dataList);
    if (pBuff == NULL) {
        /* NULL, indicates that all addresses in the linked list have been popped. */
        break;
    }

    /* The application manages pBuff, for example by calling free. */
    .....
} while (1);

```

6.4.3.2 Callback-Based Linked-List Transmission

Related interfaces: [HIF_MsgReport_ListPushData](#), [HIF_MsgReport_ListPopData](#), [HIF_MsgReport_ListStart](#).

Notes:

If `HIF_MsgReport_ListPushData` fails to add data to the linked list, stop and call `HIF_MsgReport_ListPopData` to release the HIF internal resources in the linked list.

If `HIF_MsgReport_ListStart` returns failure, call `HIF_MsgReport_ListPopData` to release the HIF internal resources in the linked list.

For other notes, see the descriptions of the related APIs.

Example:

```

int status = 0;
/* Initialize the Report data linked list. Note: The head and tail must both be assigned NULL. */

```

```

HIF_Data_ListHead_t dataList = {NULL, NULL};

do {
    /* Add Report data pRptDataA to the transmit queue */
    status = HIF_MsgReport_ListPushData(&dataList, pRptDataA, rptLenA);
    if (status != 0) {
        /* Always check the return value. On failure, stop adding data and call
HIF_MsgReport_ListPopData to release the HIF internal resources in the linked list; otherwise, a memory
leak occurs. */
        break;
    }

    /* Add Report data pRptDataB to the transmit queue */
    status = HIF_MsgReport_ListPushData(&dataList, pRptDataB, rptLenB);
    if (status != 0) {
        break;
    }

    /* Add Report data pRptDataC to the transmit queue */
    status = HIF_MsgReport_ListPushData(&dataList, pRptDataC, rptLenC);
    if (status != 0) {
        break;
    }

    /* Start Report data transmission. A report_callback third parameter indicates callback-based
transmission. */
    status = HIF_MsgReport_ListStart(0xC6, &dataList, report_callback);
} while (0);

/* Release resources on failure. */
if (status != 0) {
    /* Always reclaim resources; otherwise, memory leaks occur. */
    uint8_t *pBuff = NULL;
    do {
        /* Repeatedly pop buffs. HIF internal resources are released while the buffs are popped. */
        pBuff = HIF_MsgReport_ListPopData(&dataList);
        if (pBuff == NULL) {
            /* NULL indicates that all addresses in the linked list have been popped. */
            break;
        }

        /* The application manages pBuff, for example by calling free. */
        .....
    } while (1);
}

/**
 * @brief Callback function
 * @param msgId Message ID
 * @param pDataListHead Linked list passed when HIF_MsgReport_ListStart is called
 * @param status Data transmission status
 */
void report_callback(uint8_t msgId, HIF_Data_ListHead_t * pDataListHead, int8_t status)
{

```

```

/* Release resources.
   If the application-layer buffers in linked list pDataListHead do not need to be managed, the
   following release code may be omitted. The HIF driver automatically reclaims HIF internal resources. */
uint8_t *pBuff = NULL;
do {
    pBuff = HIF_MsgReport_ListPopData(&dataList);
    if (pBuff == NULL) {
        break;
    }

    /* The application manages pBuff, for example by calling free. */
    .....
} while (1);
}

```

6.4.3.3 Blocking Non-Linked-List Transmission

Related interface: [HIF_MsgReport](#).

Example:

```

/* The callback is NULL; blocking transmission is used. */
status = HIF_MsgReport(0xC6, pData, dataLen, NULL);

```

6.4.3.4 Callback-Based Non-Linked-List Transmission

Related interface: [HIF_MsgReport](#).

Example:

```

/* The callback is report_callback, callback/non-blocking transmission is used. */
status = HIF_MsgReport(0xC6, pData, dataLen, report_callback);

/**
 * @brief Callback function
 * @param msgId Message ID
 * @param pDataListHead Call HIF_MsgReport_ListPopData(pDataListHead) to pop the data address
 * passed to HIF_MsgReport
 * @param status Data transmission status
 */
void report_callback(uint8_t msgId, HIF_Data_ListHead_t * pDataListHead, int8_t status)
{
    /* Linked list pDataListHead contains exactly one node, the head node, which stores the data address
    passed to HIF_MsgReport. Call HIF_MsgReport_ListPopData(pDataListHead) to pop this address.
    The code below for popping the data address is optional. */
    uint8_t *pBuff = NULL;
    do {
        pBuff = HIF_MsgReport_ListPopData(&dataList);
        if (pBuff == NULL) {
            break;
        }

        /* The application manages pBuff, for example by calling free. */
        .....
    } while (1);
}

```

6.4.4 Command-Response

For the Device, receiving a message from the Host means receiving a Command. The Device uses HIF_MsgHdl_Regist to register a receive message-processing callback function (the processing function must reply to the message). When the Device receives data with the corresponding ID, it invokes the corresponding callback.

The general development process is as follows:

1. Create a message (ID) processing function callback Callback;
2. In the callback, obtain the data sent by the Host, process it, and send a reply;
3. Register the callback.

6.4.4.1 Message-Processing Callback Prototype

```
/**
 * @brief Received-message handler type
 * @param msg Message header
 * @attention The address of user data is (msg + 1)
 */
typedef int (* HIF_MsgHdl_t)(HIF_MsgHdr_t *msg);
```

6.4.4.2 Response Description

Message reply interface: HIF_MsgResp.

The reply message follows the HIF Frame format. See the preceding section [HIF Protocol Frame Format](#) under [Command-Response](#).

In HIF_MsgResp, the second parameter dataLen is the length of the reply data, and the third parameter status is the Ack Status. The first parameter can use the input parameter of the message-processing function.

```
/**
 * @brief Call this function to respond to host in a message-received handler
 * @param pMsg Message handler
 * @param dataLen The length of data responded to host
 * @param status The ack status responded to host
 * @return int
 */
int HIF_MsgResp(
    HIF_MsgHdr_t *pMsg,
    uint16_t dataLen,
    uint8_t status);
```

6.4.4.3 Receive and Reply Example

```
/**
 * @brief Message(ID: 0x18) callback
 * @param msg Message Header
 * @return int
 */
int hif_msghdl_0x18_callback(HIF_MsgHdr_t *msg)
{
    /* ack status; for optional statuses, see API: HIF_MsgResp */
    uint8_t cmd_status = HIF_CMD_STATUS_SUCCESS;

    /* Address of the received data (the Payload address in the HIF Frame structure) */
    uint8_t *payloadAddr = NULL;

    /* Length of the reply data */
    uint16_t respDataTestLen = 0;
```

```

for (uint8_t i = 0; i < RESP_DATA_TEST_LEN; i++) {
    respDataTest[i] = i;
}

/* Calculate the Payload address */
payloadAddr = (uint8_t *) (msg + 1);
/* When an extended Header is supported, offset by the extended Header length */
if (msg->flag & HIF_MSG_FLAG_EXTEND_BIT) {
    payloadAddr += sizeof(HIF_MsgExtHdr_t);
}

/* Status check. This example validates the reply-data length (users can define other checks or
statuses).
The +1 accounts for the one Byte occupied by ack status. */
if ((RESP_DATA_TEST_LEN + 1) > CONFIG_HIF_CMD_PAYLOAD_MAX_SIZE) {
    respDataTestLen = 0;
    cmd_status = HIF_CMD_STATUS_TOOLONG;
} else {
    /* Ack Status occupies the first Byte of the Payload in the HIF Frame format, so reply data
respDataTest starts at payloadAddr + 1 */
    memcpy(payloadAddr + 1, respDataTest, RESP_DATA_TEST_LEN);
    respDataTestLen = RESP_DATA_TEST_LEN;
}

/* The first parameter can use the input parameter of the message-processing function unless there
is a special requirement, such as modifying fields in the Message Header;
the second parameter dataLen is the length of the reply data;
the third parameter status is the Ack Status. */
return HIF_MsgResp(msg, respDataTestLen, cmd_status);
}

.....

/* Register hif_msghdl_0x18_callback as the message-processing function for 0x18.
After successful registration, HIF calls hif_msghdl_0x18_callback when it receives a message with ID
0x18. */
status = HIF_MsgHdl_Regist(0x18, hif_msghdl_0x18_callback);

```

6.5 Radar Data Reporting Process

This section describes the radar-data process before an HIF API is called. For the HIF reporting process, see the preceding section.

The general process for raw radar data is as follows:

1. The radar IP generates and preprocesses radar data;
2. The radar interrupt is triggered, generating a radar-data Ready event;
3. The radar thread processes the radar-data Ready event and executes the application-layer radar-data processing callback registered by the user;
4. The radar-data processing callback performs data acquisition, algorithm processing, and other operations to obtain different data types, such as 0xC2, 0xC3, and 0xC6, and then reports them to the Host through the HIF interface.

Related interfaces:

mmw_callback_handle - Radar interrupt;

mmw_ctrl_task - Radar thread;

mmw_data_handle - Radar-data processing callback registered through mmw_ctrl_callback_cfg, for example: mmw_ctrl_callback_cfg(radar_framework_2d_frame_cb, MMW_DATA_TYPE_2DFFT, NULL).

6.5.1 0xC2 Radar FFT Data

0xC2 is the MsgID for radar FFT data in the HIF protocol. For the 0xC2 structure and field information, see the preceding section [HIF Protocol Frame Format](#).

0xC2 reporting process:

1. The radar thread calls the radar-data processing callback: radar_framework_2d_frame_cb;
2. The callback obtains and processes the radar data, and then calls radar_framework_report_cube;
3. radar_framework_report_cube assembles the radar data into a packet (0xC2), and then calls radar_framework_report_cube_split to report the radar data to the Host.

6.5.2 0xC3 Radar Point-Cloud Data

0xC3 is the MsgID for radar point-cloud data in the HIF protocol. For the 0xC3 structure and field information, see the preceding section [HIF Protocol Frame Format](#).

0xC3 reporting process:

1. The radar thread calls the radar-data processing callback: radar_framework_2d_frame_cb;
2. Call radar_framework_motion_point_process (motion point cloud) and mmw_presence_point_process (presence point cloud) to obtain point-cloud data;
3. Call radar_framework_report_point_cloud_data to assemble the point-cloud data into a packet according to the 0xC3 structure:
 - (1) 0xC3 Frame Header MMW_FRAME_UPLOAD: frame_idx, frame_len, offset, and related fields;
 - (2) Motion point-cloud data ptr_point_cloud_buffer;
 - (3) Presence point-cloud data ptr_presence_point_cloud_buffer;
4. Then call HIF_MsgReport to report the 0xC3 data to the Host.

6.5.3 0xC6 "PSIC Debug Protocol" Data

0xC6 is the MsgID for PSIC Debug Protocol data in the HIF protocol. For the 0xC6 structure and field information, see the preceding section [HIF Protocol Frame Format](#).

0xC6 reporting process:

1. The radar thread calls the radar-data processing callback radar_framework_2d_frame_cb;
2. Call radar_framework_report_frameInfo:
 - (1) Obtain radar data: txrx_mode, work_mode, range_mm, resol_mm, velocity_mm, veloc_resol, period_ms, frame_num, rfft_num, and dfft_num;
 - (2) Call radar_framework_report_debug_data;
3. radar_framework_report_debug_data constructs a data packet according to the 0xC6 data structure:
 - (1) Debug Payload Header: Dim, Data Formats, Len;
 - (2) Payload Name;
 - (3) Payload Data;
4. After constructing the 0xC6 data packet, send it to the Host through HIF.

7 Custom Msg IDs

This section describes how to implement custom Report data or custom radar HIF commands.

7.1 Permitted Custom Msg ID Ranges

HIF supports adding custom MsgIDs, including custom command IDs and custom Report IDs. The command-ID range available to users is 0x90–0x9F, and the Report-ID range is 0xE0–0xEF.

7.2 Custom Commands

A custom command ID must be supported and defined on both the Host and Device. The Host creates a command with the corresponding MsgID, fills in the Payload information, and calls `Host_General_Command` to send the command. The Device creates a callback for processing the corresponding received MsgID. After msg initialization, call `HIF_MsgHdl_Regist` to register the corresponding MsgID callback.

7.2.1 Host-Side Development Steps

Example: Create a configuration command with MsgID 0x90, payload_length 3, payload[0] = 0x10, and payload[1] = 0x11,

where payload[2] is the parameter and the return-value resp_length is 1. The Device returns 0x00 after successful reception.

That is, send a 3-Byte Payload and receive a 1-Byte resp.

A: Configure the msgid and Payload information;

B: Call `Host_General_Command`;

C: Check the return value and the Device resp;

D: Declare the command function.

Create the function in `host_mmw_cmd_api.c`.

```
/* DevHdl is the Device handle obtained when Host_Device_Open is called. */
int MmwCmd_R3_User_Define_Cfg(DEV_HANDLE DevHdl, uint8_t user_value)
{
    int ret = 0;
    uint32_t msg_id = 0x90;    //Configure command msgid = 0x90;
    uint32_t param_len = 3;    //Configure payload_length = 3;
    uint32_t buffer_len = 1;    //Configure resp_length = 1;
    uint8_t param[3];          //payload buffer
    uint8_t resp_buff[1];      //resp buffer
    param[0] = 0x10;            //Assign payload[0]
    param[1] = 0x11;            //Assign payload[1]
    param[2] = user_value;      //Assign payload[2]
    /* The Host sends the command to the Device through Host_General_Command. */
    ret = Host_General_Command(DevHdl, msg_id, param, param_len, resp_buff, buffer_len, NULL);
    if (ret != HOST_ERRCODE_SUCCESS) {
        return ret;            //Check whether transmission succeeded; if not, return the status value
    } else {
        return resp_buff[0];    //Return resp after successful transmission
    }
}
```

Declare the function in `host_mmw_api.h`.

```
int MmwCmd_R3_User_Define_Cfg(DEV_HANDLE DevHdl, uint8_t user_value);
```

7.2.2 Device-Side Development Steps

Example: Create a command with MsgID 0x90 that receives payload_length 3. When payload[0] = 0x10 and payload[1] = 0x11, print payload[2]. Return HIF_CMD_STATUS_SUCCESS (0) if printing succeeds, or HIF_CMD_STATUS_UN SUPPORT (1) if it fails.

- A: Create a MsgID command callback;
- B: Obtain the Payload information;
- C: Add custom user logic;
- D: Call HIF_MsgResp to reply to the Host;
- E: Call HIF_MsgHdl_Regist to register the callback.

```
int r3_user_define_handler(HIF_MsgHdr_t *msg)
{
    int ret = HIF_CMD_STATUS_SUCCESS;
    if (msg->length != 3) { //Check whether the received payload_length is 3
        return HIF_CMD_STATUS_PARAM;
    }
    uint8_t *pId = (uint8_t*)(msg + 1); //Obtain the Payload
    if (pId[0] == 0x10 && pId[1] == 0x11) { //Check whether the Payload meets the conditions
        printf("payload[2]:0x%02x\n", (pId[2]));
    } else {
        ret = HIF_CMD_STATUS_UN SUPPORT; //Return HIF_CMD_STATUS_UN SUPPORT if reception
fails
    }
    return HIF_MsgResp(msg, 0, ret); //Return HIF_CMD_STATUS_SUCCESS if reception succeeds
}

HIF_MsgHdl_Regist(0x90, r3_user_define_handler); //Register the 0x90 callback
```

If the available command IDs are insufficient, use a subid (sub-ID) to extend control, using the first Byte of the Payload as the sub-ID and processing it separately in the callback for the corresponding MsgID.

7.3 Custom Report Data

A custom Report ID must be supported by both the Host and Device. The Host creates a callback and registers it through HIF_Report_Handle_Regist, and the Device calls HIF_MsgReport to send the data.

7.3.1 Host-Side Development Steps

See [Custom Data Processing Solution](#).

7.3.2 Device-Side Development Steps

Example: Create a Report-ID processing function with MsgID 0xE0.

See [Device Development](#) in the preceding section under [Data Reporting](#).

8 Q&A

8.1 Communication Issues

8.1.1 A checksum validation problem occurs when the HOST sends a message with a Payload of 1024 bytes or more

SDK V2.1.0 supports sending only HIF messages whose total length, including the message Header, Payload, checksum32, and other fields, is less than 512 bytes to the radar (Device). If the length is greater than or equal to 512 bytes (configurable), the radar discards the message.

When receiving a command, the radar determines that transmitted data belongs to the same message if it is sent during the same CS assertion period.

8.1.2 Why does the Host driver receive one packet but then continuously miss multiple packets when active reporting uses COM_IRQ_THREAD notification?

The COM_IRQ_THREAD notification type receives data in a thread. If the callback for each packet takes too long, data reception is delayed and data is missed, resulting in multiple consecutive packets being missed.

In addition to COM_IRQ_THREAD, register the interrupt_ctrl interface. After this interface is registered, the entire LLC_HANDLE_S::rx_cache space is used to buffer data. interrupt_ctrl must support enabling and disabling interrupts and setting the interrupt threshold, which specifies how many received Bytes trigger an interrupt.

Alternatively, use the COM_ISR notification type, which supports using the entire LLC_HANDLE_S::rx_cache space to buffer data.

8.1.3 Unable to Receive Report Packets That Pass checksum Validation

If cache is enabled on the Host, invalidate and clean operations on the cache may inadvertently operate on other memory addresses.

Modify the ported host_os_malloc and host_os_free so that allocated memory is aligned to N Bytes, where N depends on the Host. For example, if the Host cache operates on at least 32 bytes at a time, N is 32. The return value of host_os_malloc and the actual allocated length must both be multiples of 32. For example, when 33 bytes are requested, the actual allocated address may be 0x10507715; host_os_malloc should return 0x10507720, and the final address of the allocated memory space is 0x1050775F.

In summary, if cache is enabled and N is 32, the following requirements must be met:

- The host_os_malloc return value is 32-Byte aligned;

- The final address of the allocated memory space plus 1 is 32-Byte aligned.

8.1.4 Commands and Responses

A Command generally refers to an instruction sent from the Host to the Device.

A Response generally refers to the Device's reply to a received Command. Whether a Response is returned is determined by the Req field in the Command Header.

The Device does not support concurrent Command reception and processing. **It receives the next Command only after the previous Command has been processed or a Response has been sent.**

8.2 Configuration Issues

8.2.1 Modifying Tool/Host PC Configuration Through the Host

There are two ways to modify a configuration through the Host: identify the TLV by modifying parameters in the Host PC software, or directly consult the TLV description document.

Modify one parameter in the Host PC software and compare the TLV byte streams before and after the modification. The differing group identifies the TLV corresponding to that parameter. Then consult the TLV

description document to determine the purpose of the TLV and whether it contains multiple configuration items, that is, whether one TLV configures multiple parameters.

Consulting the TLV description document directly provides a more accurate understanding of each TLV's function and configuration method.

8.3 Abnormal Radar Operation

8.3.1 The radar analysis sample suddenly stops reporting when the Frame period is set to a very small value

When the Frame period is very small, the radar detection interrupt may be triggered before data reception is complete. After this interrupt is triggered, the system attempts to allocate a large amount of memory for calculations, result storage, and other operations. If reception of the previous Frame is incomplete, the radar does not release the memory occupied by that data, causing memory allocation to fail and the system to hang.

The minimum Frame period can be calculated using the following formula:

$$T = t_{\text{radar}} + t_{\text{report}} + t_{\text{delay}}$$

where t_{radar} is the time from radar transmission to interrupt t_{radar} generation, which can be approximated as the transmission time, in ms;

t_{report} it is the data reporting time, in ms;

t_{delay} it is the reporting delay, in ms.

t_{radar} It is calculated as follows:

$$t_{\text{radar}} = \text{ceiling}\left(\frac{150}{V_{\text{res}} f_{\text{start}}}\right)$$

where $\text{ceiling}(x)$ indicates x rounding up;

V_{res} it is the velocity resolution, in m/s;

f_{start} it is the start frequency, in GHz.

The SPI reporting formula t_{report} is as follows:

$$t_{\text{report}} = \text{ceiling}\left(\frac{85 R_{\text{max}} V_{\text{max}} N_{\text{antCh}}}{65536 R_{\text{res}} V_{\text{res}}}\right)$$

where N_{antCh} is the number of antenna channels, calculated as the product of $N_{\text{antCh}} = N_{\text{tx}} N_{\text{rx}}$ the transmit-channel count and receive-channel count;

R_{max} it is the maximum measurement range, in m;

V_{max} it is the maximum detection velocity, in m/s;

R_{res} it is the range resolution, in m.

t_{report} It uses transmission of 256 kB in 85 ms as an example and must be converted according to the actual conditions. Note that the transmission time per Byte is not the time required for SPI or UART to transmit one Byte; it is the average time per Byte over the entire 256-kB data transfer and is generally longer than the time for SPI or UART to transmit one Byte.

The UART reporting formula t_{report} is as follows:

$$t_{\text{report}} = \text{ceiling}\left(\frac{48000 N_{\text{antCh}} R_{\text{max}} V_{\text{max}}}{\text{baudrate} \times R_{\text{res}} V_{\text{res}}}\right)$$

where it is the baudrate UART baud rate.

t_{delay} It is 0 for both active and passive reporting, and has a value only for active reporting (delayed).

Appendix

Existing Msg ID Definitions

MSG ID	Meaning
0x00	Get the chip model/ID and firmware version number
0x01	Get Device status information
0x02	Get feature-support information
0x03	Reserved
0x04	Restart the Device
0x05	Device sleep/wakeup
0x06	Device miscellaneous control
0x07	Start/stop radar operation
0x08	Reserved
0x09	HIF hardware-interface mode parameters
0x0A	HIF interaction-mode configuration
0x0B	Host status synchronization
0x0C	Transport control
0x0D	Reserved
0x0E	Firmware download control
0x0F	Firmware data-packet transmission
0x10	GPIO control
0x11	Storage enable control
0x12	EEPROM write
0x13	EEPROM read
0x14	Get Flash ID
0x15	Flash erase
0x16	Flash write
0x17	Flash read
0x18	eFuse parameter configuration
0x19	Get eFuse parameters
0x1A	Get information status
0x1B	Write image data
0x1C	Request command
0x1D	Shell function
0x1E	Debug control configuration

MSG ID	Meaning
0x1F	Read Debug information
0x20-0x2F	Reserved for internal use
0x30	Production-test mode switching
0x31	System power supply
0x32	System I/O
0x33	Radar performance evaluation
0x34	Radar calibration
0x35	Radar scenario functions
0x36	Radar scenario power consumption
0x37	Save calibration parameters
0x38	Get calibration parameters
0x39-0x3F	Reserved
0x40	Read radar parameters
0x41	Configure radar parameters
0x42	Get AS2528 radar parameters
0x43-0x4F	Reserved
0x50-0x5F	Reserved
0x60	Send configuration
0x61	Get data
0x62-0x64	Reserved
0x65	Human-presence sensor control on the Device
0x66-0x6F	Reserved
0x70-0x7F	Reserved
0x80-0x8F	Reserved
0x90-0x9F	Register customer command-ID configuration
0xA0-0xAF	Reserved
0xB0-0xBF	Reserved
0xC0	ADC data
0xC1	FFT data
0xC2	Multiple types of FFT data
0xC3	Point clouds/targets
0xC4	R7 gesture data
0xC5	Low-power human-presence data Report
0xC6	Motion Sensor

MSG ID	Meaning
0xC7	Motion Sensor Plot
0xC8, 0xCA-0xCB	R4 Report of range, azimuth, elevation, uid, and snr
0xC9	MRS2 Report of range
0xCC-0xCF	Reserved
0xD0-0xDF	Reserved
0xE0-0xEF	Register customer Report-ID configuration
0xF0	System print
0xF1	System compressed print
0xF2-0xFD	Reserved
0xFE	System heartbeat
0xFF	System startup Report

Common Command-Response Data Formats

VERSION [0x00]

Get the Device version number.

MsgID: 0x00

Payload: 1 Byte. 0x40 indicates the version of the customer configuration file customer-cfg.img, and 0x42 indicates the versions of the Code and customer configuration file customer-cfg.img.

Code version:

```
struct code_version_t {
    uint8_t  ir_type;
    uint16_t ir_minor;

    uint16_t im_minor;
    uint8_t  im_project;
    uint16_t im_number;

    uint8_t  iv_major;
    uint8_t  iv_minor;
    uint16_t iv_revision;
};
```

Customer configuration file customer-cfg.img version:

```
struct version_t {
    uint8_t major;
    uint8_t minor;
    uint16_t revision;
    uint32_t build_num; // Indicates the generation date of the cfg file
};
```

The 32-bit build_num date format is as follows:

Seconds: [0:4], in units of 2 seconds. $\text{Sec} = (\text{build_num} \& 0x1F) * 2$

Minutes: [5:10]. $\text{Min} = (\text{build_num} \gg 5) \& 0x3F$

Hours: [11:15]. $\text{Hour} = (\text{build_num} \gg 11) \& 0x1F$

Day: [16:20]. $\text{Day} = (\text{build_num} \gg 16) \& 0x1F$

Month: [21:24]. $\text{Mon} = (\text{build_num} \gg 21) \& 0xF$

Year: [25:31], with 2000 as the start year. $\text{Year} = (\text{build_num} \gg 25) \& 0x7F + 2000$

$\text{build_num} = ((\text{Year} - 2000) \ll 25) | (\text{Mon} \ll 21) | (\text{Day} \ll 16) | (\text{Hour} \ll 11) | (\text{Min} \ll 5) | (\text{Sec}/2)$

Customer configuration file version:

CMD:

A5 24 15 00 01 20 40 AA FF FE DF

00 gets the version number, and 40 gets the customer-cfg.img version.

RSP:

A5 1F 12 00 09 20 00 01 04 05 00 AD 7C 6A 33 BA 51 76 70

00 gets the version number, 00 is the returned status, and 01 04 05 00 AD 7C 6A 33 is the version number.

Code and customer configuration file versions:

CMD:

A5 04 15 00 01 40 42 A8 FF FE BF

00 gets the version number, 42 gets the app code and customer-cfg.img versions.

RSP:

A5 03 12 00 15 30 00 00 00 00 00 00 00 02 00 03 00 01 04 05 00 AD 7C 6A 33 BA 4F 6A 5D

00 gets the version number, 00 is the returned status, and 00 00 00 00 00 00 00 00 02 00 03 00 01 04 05 00 AD 7C 6A 33 is the version number.

TICK [0x01]

Get the Device heartbeat clock

CMD:

A5 D3 15 01 01 70 40 AA FE FE 8F

01 indicates getting Device information, 40 indicates that the specific content to get is Tick.

RSP:

A5 22 12 01 05 20 00 2F F7 51 04 E9 CF 03 8E

01 indicates getting Device information, 00 indicates the status code, and 2F F7 51 04 indicates the tick value 0x0451F72F.

REBOOT[0x04]

Set the Payload for radar reset and restart to 1.

CMD

```
A5 40 15 04 01 00 01 E9 FB FE FF
```

04 configures radar reset and restart, and 01 resets and restarts the radar.

RSP

```
A5 E3 12 04 01 60 00 ED FB FE 9F
```

04 configures radar parameters, 00 indicates success status.

POWER_STATE_SET [0x05]

While the Host is sending or obtaining configuration, the Device does not enter sleep after being woken. The Device is permitted to enter sleep only after receiving this command.

CMD

```
A5 CF 15 05 01 70 01 E9 FA FE 8F
```

05 sets the Device status, and 01 permits sleep.

RSP

```
A5 02 12 05 01 00 00 ED FA FE BF
```

05 indicates getting Device configuration information, and 00 indicates the status code.

START_CTRL(0x07)

Set the Payload to 1 to start radar detection, or to 0 to stop radar detection and enter configuration mode.

CMD

```
A5 3D 15 07 01 00 01 E9 F8 FE FF
```

07 configures radar detection, and 01 starts radar detection.

RSP

```
A5 10 12 07 01 30 00 ED F8 FE CF
```

07 configures radar parameters, and 00 indicates success status.

POLL_CMD [0x0C]

When passive reporting is used on the communication port, Notify I/O (generally PA6) is driven high, Host must send a poll command to the Device.

CMD:

```
A5 35 15 0C 04 00 02 00 01 00 E8 F3 FA FF
```

CFG_GET [0x42]

The Host gets the radar configuration through this command (some firmware uses another ID). The transmitted configuration uses TLV format.

For radar configuration-get TLVs, see Table 8-1 Radar Configuration TLVs.

CMD

```
A5 FF 15 42 04 00 12 02 00 00 D8 BB FB FF
```

42 gets radar parameters, 12 is the type, 02 is the length, and 00 00 is the Value. This TLV gets the radar operating start frequency.

RSP

```
A5 01 12 42 05 00 00 12 02 78 E6 07 AB F8 87
```

42 gets radar parameters, 00 indicates success status, 12 is the type, 02 is the length, and 78 E6 is the Value. This TLV indicates that the obtained radar start frequency is E678 (59000).

CFG_SET [0x60]

The Host sends all radar configurations through this command, and the transmitted configuration uses TLV format.

Table 8-1 Radar Configuration TLVs (Partial)

Type	Length	Value	Meaning	Valid Value Range
0x10	1	1	Configure the radar antenna operating mode (send the corresponding parameter when selecting a mode: 1T1R: 0, 1T3R: 1, 2T3R: 2, 1T4R: 3, 2T4R: 4, 1T2R: 5). The 6130 series supports only 1T3R.	0-5
0x11	1	1	Configure the radar operating Frame mode (0 for a 1D Frame and 1 for a 2D Frame).	0/1
0x12	2	59000	Configure the radar operating start frequency. Unit: MHz (megahertz)	[57-61]GHz
0x13	4	8000	Configure the maximum radar detection range. Unit: mm (millimeters)	[1.3-32]m
0x14	2	40	Configure the radar range resolution. Unit: mm (millimeters)	[0.02-0.5]m
0x15	2	1600	Configure the maximum unambiguous velocity. Unit: mm/s (millimeters per second)	[0.1-20]m/s
0x16	2	100	Configure the velocity resolution. Unit: mm/s (millimeters per second)	[0.01-10]m/s
0x17	2	500	Configure the radar operating Frame period. Unit: ms (milliseconds)	[0.05-1]s

CMD

```
A5 D1 15 60 04 10 17 02 2C 01 D3 9D CF EE
```

60 configures radar parameters, 17 is the type, 02 is the length, and 2C 01 is the Value. This TLV sets the radar operating Frame period to 300 ms.

RSP

```
A5 E7 12 60 01 00 00 ED 9F FE FF
```

60 configures radar parameters, and 00 indicates success status.

Disclaimer and Copyright Notice

The information provided by our company in this document is accurate to the best of our knowledge, but we do not guarantee that the content is completely error-free. All liability for any damages arising from the use of this document (including but not limited to indirect, incidental, or special losses) or any improper use resulting from non-compliance with the instructions herein (including but not limited to over-voltage, over-frequency, or over-temperature operations) is explicitly disclaimed.

The content in this document, including URL references, may differ from the actual content due to product version upgrades or other reasons, and is subject to change without notice.

THIS DOCUMENT IS PROVIDED "AS IS" WITH NO WARRANTIES. NO WARRANTY IS PROVIDED TO THIS DOCUMENT FOR ITS MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR ANY PARTICULAR PURPOSE, NOR DOES ANY WARRANTY OTHERWISE ARISING OUT OF ANY PROPOSAL, SPECIFICATION OR SAMPLE.

All liability, including liability for infringement of any patent rights, relating to use of information in this document is disclaimed. No licenses express or implied, by estoppel or otherwise, to any intellectual property rights are granted herein.

Copyright © 2026 Possumic Technology Co., Ltd. All rights reserved.